

验收成绩	报告成绩	总评成绩

武汉大学计算机学院 本科生实验报告

操作系统实践 A

专 业 名 称 ： 计算机科学与技术

课 程 名 称 ： 操作系统设计

指 导 教 师 ： 蔡朝晖

学 生 学 号 ： 2023302111419

学 生 姓 名 ： 林豪

二〇二五年十二月

实验 1：RISC-V 引导与裸机启动

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

0. 实验任务和目标

本实验任务如下：

1. 通过参考 xv6 的启动机制，理解并实现最小操作系统的引导过程，实现操作系统的启动
2. 学习在 xv6 操作系统中如何使用串口驱动uart，从而在QEMU中输出对应启动信息，最终在QEMU中输出测试信息
3. 额外任务：通过测试代码理解锁的用处，从而理解操作系统中锁的作用，加强进程与线程中有关于锁的印象

1. 系统设计

1.1 架构设计说明

1.1.1 xv6的启动流程

通过阅读代码，可知xv6的启动流程如下：

1. 当操作系统启动时，CPU将会重置到一个预定义的物理地址（即0x80000000），从而开始运行entry.s。这个文件确定栈指针地址，对每一个CPU核分配对应的栈空间，之后跳转到start.c进行接下来的操作。
2. 在系统跳转到start.c后，start.c对于存储器以及中断进行管理。例如禁止分页等。在此之后，start.c将mepc设为main函数的地址，并使用mret功能跳转到main函数，进行下一步的操作。
3. CPU跳转到main函数中。对于多核CPU，其0号核负责所有模块的初始化操作。之后，系统根据当前核的不同输出不同的提示信息。最后所有核所运行的线程均进入死循环。

本系统架构采用与xv6相同的启动过程，使用entry.s, start.c, main.c的顺序进行程序的启动

1.1.2 xv6的串口机制与输出

串口（UART），全称 Universal Asynchronous Receiver/Transmitter，中文叫“通用异步收发器”，负责将串行数据以及并行数据的转换，并实现串行数据的输入核输出。在操作系统内，uart实现了系统以及输入输出设备的交互功能。例如在屏幕上输出信息，处理用户的输入等。

本操作系统使用uart NS16550A作为串口，进而通过提供控制台的功能从而进行输出输出的处理。同时，uart设备采用内存映射I/O的方式连接到CPU。在xv6默认运行的QEMU virt机器中，NS16550A兼容的UART控制器被映射到了物理地址 **0x10000000**。

这个UART设备有一组寄存器，每个寄存器都有其对应的偏移地址：

偏移量	寄存器缩写	名称	作用描述
0	RHR	接收保持寄存器	读操作：从该寄存器读取接收到的字节
0	THR	发送保持寄存器	写操作：向该寄存器写入要发送的字节
1	IER	中断使能寄存器	控制是否在特定事件（如数据到达）时产生中断
5	LSR	线路状态寄存器	最重要的寄存器，提供状态信息，例如bit 5（THRE）表示发送保持寄存器是否为空（可写入新数据），bit 0（DR）表示是否有数据到达（可读取）

注：由于寄存器被映射到内存，所以可以通过指针访问这些寄存器。如(uint8*)(0x10000000 + 5)就指向LSR寄存器。

在系统启动时，我们需要初始化UART硬件，从而便于使用：

- 配置波特率（设置时钟分频器寄存器）。
- 配置数据格式（数据位、停止位等）。
- 使能接收中断（IER），这样当键盘有数据输入时，UART会向CPU发出中断请求，CPU就可以及时处理输入，而不需要不断轮询。

而当操作系统需要输出一个字符时，系统将检查uart状态并将字符放入缓冲队列中，uart通过如下流程进行输出：

1. 读取线路状态寄存器（LSR），检查 THRE（发送保持寄存器空）位是否为1。
2. 如果为空，说明UART已经准备好接收一个新的字节进行发送。此时，程序将字符 c 写入到 发送保持寄存器（THR）的地址。
3. 一旦数据写入THR，UART硬件会自动接管，开始将字节转换成串行数据流并从TX引脚发送出去。
4. 如果寄存器不为空（即UART还在发送上一个字节），驱动会等待（轮询）或者进入睡眠，等待发送完成中断（在xv6中，发送部分采用了轮询而非中断以简化设计）。

通过如上uart的输出字符功能，我们可以进一步的实现如uart输出字符串等功能，从而实现对于系统的输入输出，典型的利用即使用printf函数进行C语言下的输出。

1.2 关键数据结构

1.2.1 汇编代码启动程序入口entry.S

作为汇编代码的启动程序入口，其代码如下：

```

.section .text
.global _entry
_entry:
    # set up a stack for C.
    # stack0 is declared in start.c,
    # with a 4096-byte stack per CPU.
    # sp = stack0 + (hartid * 4096)
    la sp, stack0
    li a0, 1024*4
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # jump to start() in start.c
    call start
spin:
    j spin

```

这段代码为CPU的每一个核设置了栈空间，就是为了让每个核在执行线程时拥有对应的空间。在此处，栈空间设置为每个核4096字节。栈指针和栈空间的设置过程如下：

- 将栈指针sp设为stack0：`la sp, stack0`，可知`la`为加载地址的伪指令，且stack0在start.c中定义：`__attribute__((aligned(16))) char stack0[4096 * NCPU];`
- 将a0设为1024*4：`li a0, 1024*4`，将a1设为当前CPU的线程号：`csrr a1, mhartid`。其中，`csrr`指令用于读取状态寄存器的值输入到寄存器中，`mhartid`表示当前CPU线程的编号，标识这个CPU的核编号(0, 1.....)
- 计算当前CPU栈底地址，并将其输入到sp寄存器中，即 $sp = stack0 + (hartid * 4096)$

在此之后，使用`call start`进行跳转到对应的C语言函数`start`，如果调用意外结束，则跳转到spin死循环中，防止程序意外结束。

1.2.2 链接脚本kernel.ld

在将源文件编译为目标文件`.o`、`.d`之后，`kernel.ld`负责控制文件之间的链接。其代码如下：

```

OUTPUT_ARCH( "riscv" )
ENTRY( _entry )

SECTIONS
{
    /*
     * ensure that entry.S / _entry is at 0x80000000,
     * where qemu's -kernel jumps.
     */
    . = 0x80000000;

    .text : {
        *(.text .text.*)
        . = ALIGN(0x1000);
        _trampoline = .;
    }

```

```

    *(trampsec)
    . = ALIGN(0x1000);
    ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
    PROVIDE(etext = .);
}

.rodata : {
    . = ALIGN(16);
    *(.srodata .srodata.*) /* do not need to distinguish this from .rodata */
    . = ALIGN(16);
    *(.rodata .rodata.*)
}

.data : {
    . = ALIGN(16);
    *(.sdata .sdata.*) /* do not need to distinguish this from .data */
    . = ALIGN(16);
    *(.data .data.*)
}

.bss : {
    . = ALIGN(16);
    *(.sbss .sbss.*) /* do not need to distinguish this from .bss */
    . = ALIGN(16);
    *(.bss .bss.*)
}

PROVIDE(end = .);
}

```

每个段的功能如下：

- `OUTPUT_ARCH("riscv")`：指定输出内核镜像的目标架构为 RISC-V。这确保了链接器会生成符合 RISC-V 指令集规范的二进制代码
- `ENTRY(_entry)`：指定程序的入口点为符号 `_entry`。从而在操作系统开始执行的第一条代码地址为 `_entry` 的地址，从而使得 `entry.s` 成为第一个启动的程序
- `. = 0x80000000`：设置了当前定位计数器的值为 `0x80000000`，使得后续的地址以此为基础进行分配
- `.text`, `.rodata`, `.data`, `.bss`：分别定义了代码段，只读数据段，数据段和 BSS 段（存放初始化为 0 或未初始化的全局变量）。各个段的功能如下：
 - 代码段 `text`：存放代码数据
 - 只读数据段 `rodata`：存放只读数据
 - 数据段 `data`：存放一般数据
 - BSS 段 `bss`：存放为 0 的常量或者初始化为 0 的全局变量
- `. = ALIGN(x)`：指定从当前地址开始，按照 `x` 字节对齐
- `PROVIDE(end = .)`：定义了一个符号 `end`，其值为当前定位计数器的值。这标记了整个内核镜像（包括所有段）的结束地址

在源文件编译为目标文件之后，目标文件的各个段（代码段，数据段等）将会按照如上要求排列放在这些段中，从而实现连接

1.2.3 串口驱动uart.c

`uart.c`通过对于内存映射I/O的控制，从而实现对于输出的实现。对于uart硬件的相关机理已经在1.1.2节讲述过，这里不再赘述。本操作系统使用类似于xv6的操作代码，详情请看下文。

1.2.4 锁结构spinlock

自旋锁的结构是一种锁结构，用于保证临界资源的互斥使用，其结构如下：

```
typedef struct spinlock {
    int locked;
    char* name;
    int cpuid;
} spinlock_t;

void push_off();
void pop_off();

void spinlock_init(spinlock_t* lk, char* name);
void spinlock_acquire(spinlock_t* lk);
void spinlock_release(spinlock_t* lk);
bool spinlock_holding(spinlock_t* lk);
```

自旋锁包含其名字，状态，CPU编号信息；同时本系统有众多有关于自旋锁的函数，用于上锁以及解锁，从而实现线程之间的互斥。

1.3 设计决策理由

对于整个操作系统的编译和链接，在Makefile控制编译器将各个文件编译成目标文件`.o`，`.d`后，`kernel.ld`将会控制各个段进行链接，从而保证各个段的地址正确，且对齐完备，能够正常运行。

对于操作系统的启动和初始化，以`entry.S`作为整个系统的开始部分从而分配栈空间，以`start.c`作为硬件初始化的代码进行初始化，以`main.c`进行各个模块的初始化以及提示信息的输出，这样可以使得这个操作系统能够对于各个模块初始化，从而易于使用。

对于操作系统的串口输出，通过`uart.c`控制硬件串口，从而逐个输出字符，进而输出字符串并实现格式化的输出功能，同时利用`uart.c`的串口处理能力，还能进行输入的处理等。

综上所述，这种类似于xv6的结构无论从代码编译上还是从执行善都具有良好的结构性，这个结构能够成功构建项目并且成功运行，输出对应的提示信息。

1.4 与 xv6 对比分析

本结构采用与xv6类似的启动和字符输出流程，包含`entry.s`，`start.c`，`main.c`等函数架构实现了启动功能，而在串口输出流程上，本结构相比于xv6减少了`console.c`的显示，其显示更加方便；且输出结构相比xv6更加简单。其简洁的特点为代码书写提供了便利。

2. 实验过程部分

2.1 实现步骤记录

2.1.1 理解xv6的启动流程

首先，阅读`kernel/entry.S`，了解启动汇编代码的机理，对于思考题的答案如下：

为什么第一条指令是设置栈指针：

- 设置栈指针的目的是为了给每个核分配栈空间，先将栈指针设置为`stack0`从而在后来能够方便计算。
- 当然，每个核的初始栈指针地址计算公式为： $\text{stack0} + ((\text{hartid} + 1) * 4096)$ ，如果只有单核，只需要有`stack0 + 4096`即可

`stack0`在哪个地方定义：

- 在`start.c`中定义，表示栈空间指针

为什么要清零 BSS 段：

- BSS段：未初始化或初始化为0的全局变量和静态变量，没必要放在ROM上，只需记录起始地址和大小，再使用前清零即可
- 为了使得那些初始化为0的静态变量能够为0，需要清零（考虑到内存内可能有污染，部分值初始不为0，如果直接使用，可能使用到了非0数据，导致出错）

如何从汇编跳转到 C 函数：

- 在汇编代码中，`call start`调用在`start.c`中的`start`函数，然后在`start.c`中，使用`mret`功能跳转到`main`函数

其次，阅读`kernel/kernel.ld`，了解文件链接文件的机理，对于思考题的答案如下：

对于整个系统，内存管理很复杂。最小系统所需的组成部分有：

- 代码段：存放操作系统相关代码
- 数据段：存放数据
- 堆栈段：存放放入堆栈的变量等。

`ENTRY(_entry)`的作用：

- 指定程序的入口点（Entry Point）为符号`_entry`。这意味着当模拟器（如 QEMU）或引导加载程序加载内核后，执行的第一条指令就是`_entry`所在的地址。这个符号通常定义在汇编文件（如`entry.S`）中，是内核启动代码的起点

代码段放在地址`0x80000000`的原因：

- `. = 0x80000000;`这个部分设置了定位计数器的值为`0x80000000`。定位计数器可以理解为链接器的“当前指针”，后续段的地址会以此为基础进行分配
- 在许多 RISC-V 系统中（特别是在 QEMU 模拟环境下），`0x80000000`是内核加载的起始物理地址。将代码段的起始地址设置于此，确保了内核能被正确加载和执行

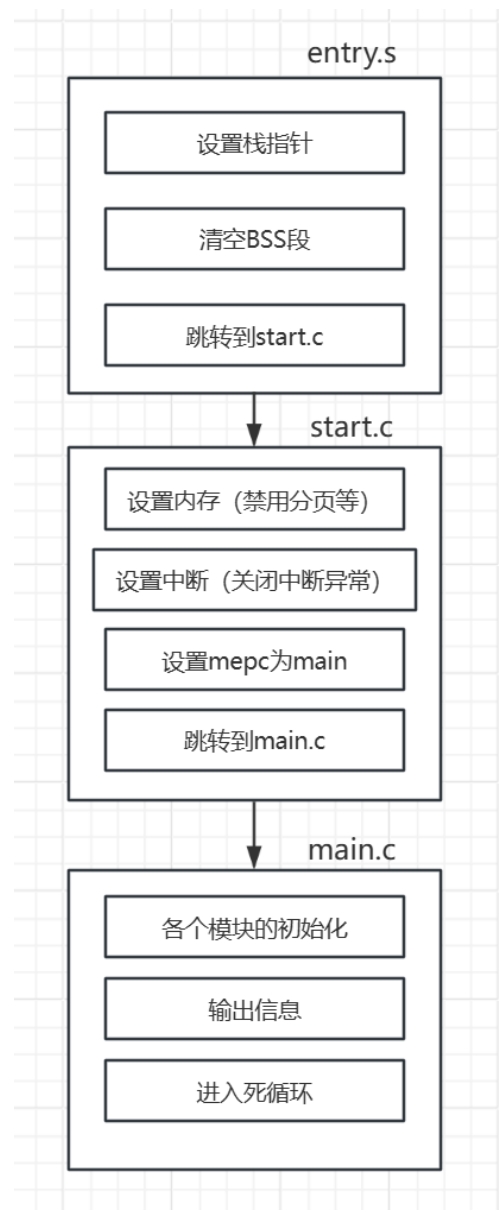
`etext`、`edata`、`end`的用途：

- `etext`：`PROVIDE(etext = .);`：定义了一个符号`etext`，并将其值设置为当前的定位计数器值。这标记了代码段（包括蹦床代码）的结束地址，可以在内核中用于计算代码段的大小或确定其他段的起始位置

- edata：类似于etext。其标记了data段的结束地址，可以在内核中计算数据段的大小，起始位置
- end：PROVIDE(end =.);：定义了一个符号 end，其值为当前定位计数器的值。这标记了整个内核镜像（包括所有段）的结束地址

2.1.2 设计最小启动流程

根据xv6的启动流程，设计自己的操作系统的启动流程，绘制流程图如下：



参考xv6，本操作系统的内存布局方案如下：

- 代码段：存放操作系统的代码
- 只读数据段：存放只读数据
- 数据段：存放一般的数据
- BSS段：存放初始化为0或未初始化的全局数据

其中，栈放在数据段中，每个核分配4096字节；BSS段的初始化交给链接器，不用手动进行BSS段清零。

至于串口输出，由于采用内存映射I/O的方式，则可以使用指针映射从而修改寄存器中的值，具体输出需要配置寄存器RHR，THR，IER，ICR等，具体寄存器的初始化和使用请看2.1.5节。

2.1.3 实现启动汇编代码

参考xv6的实现思路，书写对应的启动汇编代码`kernel/entry.S`：

```
# kernel.ld 将 _entry 作为整个OS的起点放置到0x80000000处
# qemu会自动跳转到0x80000000处并开始执行
# 注意：此时是M-mode

.section .text
_entry:
    # CPU_stack 定义于start.c中
    # sp = CPU_stack + ((hartid + 1) * 4096)
    # 将sp置于当前CPU的内核栈的栈顶
    la sp, CPU_stack
    li a0, 4096
    csrr a1, mhartid
    addi a1, a1, 1
    mul a0, a0, a1
    add sp, sp, a0
    # 跳转到start
    call start

spin:
    j spin
```

2.1.4 编写链接脚本

参考xv6的基本结构，书写链接脚本，从而对于目标文件进行链接，其代码如下：

```
OUTPUT_ARCH( "riscv" )
ENTRY( _entry )

SECTIONS
{
    . = 0x80000000;

    .text : {
        *(.text .text.*)
        . = ALIGN(0x1000);
        /*
        _trampoline = .;
        *(trampsec)
        . = ALIGN(0x1000);
        ASSERT(. - _trampoline == 0x1000, "error: trampoline larger than one page");
        */
        PROVIDE(etext = .);
    }

    .rodata : {
        . = ALIGN(16);
        *(.srodata .srodata.*) /* do not need to distinguish this from .rodata */
    }
```

```
. = ALIGN(16);
*(.rodata .rodata.*)
}

.data : {
. = ALIGN(16);
*(.sdata .sdata.*) /* do not need to distinguish this from .data */
. = ALIGN(16);
*(.data .data.*)
}

.bss : {
. = ALIGN(16);
*(.sbss .sbss.*) /* do not need to distinguish this from .bss */
. = ALIGN(16);
*(.bss .bss.*)
}

PROVIDE(end = .);
}
```

2.1.5 实现串口驱动

参考xv6的uart.c，通过uart的功能实现字符串的输出功能。具体流程如下：

首先，在实现输出前需要对uart进行初始化，初始化相关代码如下：

```
// uart 初始化
void uart_init(void)
{
    // 关闭中断
    WriteReg(IER, 0x00);

    // 进入设置比特率的模式
    WriteReg(LCR, LCR_BAUD_LATCH);

    // 设置比特率的低位和高位，最终设置为38.4K
    WriteReg(0, 0x03);
    WriteReg(1, 0x00);

    // 设置传输字节长度为8bit,不校验
    WriteReg(LCR, LCR_EIGHT_BITS);

    // 清零和使能FIFO模式
    WriteReg(FCR, FCR_FIFO_ENABLE | FCR_FIFO_CLEAR);

    // 使能输出队列和接收队列的中断
    WriteReg(IER, IER_TX_ENABLE | IER_RX_ENABLE);
}
```

如上初始化代码设置了比特率，传输字节长度和输出模式，使得uart能够正确的对终端进行输出。

其次，书写uart输出字符的代码，如下所示：

```
// 单个字符输出
void uart_putc_sync(int c)
{
    push_off();

    while(panicked);

    // 等待TX队列进入idle状态
    while((ReadReg(LSR) & LSR_TX_IDLE) == 0);

    // 输出
    WriteReg(THR, c);

    pop_off();
}
```

这个函数先关中断，之后在无panic的情况下等待TX队列进入闲置状态（需要检查并等待LSR寄存器的IDLE位是否为1，因为需要闲置后才能继续输入），之后向队列输入字符后开启中断。通过开关中断的方式，函数能够实现各个线程的互斥使用

最后，通过此函数实现字符串的输出，代码如下：

```
// 发送一个以 '\0' 结尾的字符串
void uart_puts(char *s) {
    while (*s) {
        uart_putc_sync(*s++);
    }
}
```

如上代码实现了使用uart输出字符，以及输出字符串的功能。

2.1.6 完成C主函数

接下来，编写main.c文件，该文件对于各个功能函数进行初始化操作，并根据当前CPU的不同输出不同的信息，具体代码如下：

```
#include "riscv.h"
#include "lib/print.h"
#include "lib/lock.h"
#include "proc/proc.h"
#include "dev/uart.h"

volatile static int started = 0;
```

```

int main()
{
    // 获取当前CPU的id
    int cpuid = mycpuid();
    if(cpuid == 0){
        uart_init();
        print_init();
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
    }
    else{
        while(started == 0) {};
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);
    }

    while(1) {};
}

```

这个函数获取到了当前CPU的编号，从而依据当前核的不同，输出不同的信息。其中__sync_synchronize实现了同步的功能，从而使得不同核的信息能够有序输出

2.1.7 完善自旋锁的设置

参考xv6的自旋锁模块，实现自旋锁的功能（获取锁，开锁，解锁等）。其中主要函数的代码如下：

```

#include "lib/lock.h"
#include "lib/print.h"
#include "proc/proc.h"
#include "riscv.h"

// 带层数叠加的关中断
void push_off(void)
{
    int old = intr_get();

    intr_off();
    if(mycpu()->noff == 0)
        mycpu()->origin = old;
    mycpu()->noff += 1;
}

// 带层数叠加的开中断
void pop_off(void)
{
    struct cpu *c = mycpu();
    if(intr_get())
        panic("pop_off - interruptible");
}

```

```
    if(c->noff < 1)
        panic("pop_off");
    c->noff -= 1;
    if(c->noff == 0 && c->origin)
        intr_on();
}

// 是否持有自旋锁
// 中断应当是关闭的
bool spinlock_holding(spinlock_t *lk)
{
    int r;
    r = (lk->locked && lk->cpuid == mycpuid());
    return r;
}

// 自旋锁初始化
void spinlock_init(spinlock_t *lk, char *name)
{
    lk->name = name;
    lk->locked = 0;
    lk->cpuid = -1;
}

// 获取自旋锁
void spinlock_acquire(spinlock_t *lk)
{
    push_off(); // disable interrupts to avoid deadlock.
    if(spinlock_holding(lk))
        panic("acquire");

    // On RISC-V, sync_lock_test_and_set turns into an atomic swap:
    //  a5 = 1
    //  s1 = &lk->locked
    //  amoswap.w.aq a5, a5, (s1)
    while(__sync_lock_test_and_set(&lk->locked, 1) != 0)
        ;

    // Tell the C compiler and the processor to not move loads or stores
    // past this point, to ensure that the critical section's memory
    // references happen strictly after the lock is acquired.
    // On RISC-V, this emits a fence instruction.
    __sync_synchronize();

    // Record info about lock acquisition for spinlock_holding() and debugging.
    lk->cpuid = mycpuid();
}

// 释放自旋锁
void spinlock_release(spinlock_t *lk)
{
    if(!spinlock_holding(lk))
        panic("release");
}
```

```

lk->cpuid = -1;

// Tell the C compiler and the CPU to not move loads or stores
// past this point, to ensure that all the stores in the critical
// section are visible to other CPUs before the lock is released,
// and that loads in the critical section occur strictly before
// the lock is released.
// On RISC-V, this emits a fence instruction.
__sync_synchronize();

// Release the lock, equivalent to lk->locked = 0.
// This code doesn't use a C assignment, since the C standard
// implies that an assignment might be implemented with
// multiple store instructions.
// On RISC-V, sync_lock_release turns into an atomic swap:
//   s1 = &lk->locked
//   amoswap.w zero, zero, (s1)
__sync_lock_release(&lk->locked);

pop_off();
}

```

2.1.8 实现printf的功能

在完善自旋锁的设置之后，就可以实现printf函数，从而使得系统能够输出对应的信息。

在printf中，首先定义并初始化当前的锁结构，从而使得输出能够正常执行：

```

// 防止占用的lock
static struct {
    struct spinlock print_lk;
    int locking;
} pr;

void print_init(void)
{
    uart_init();
    spinlock_init(&pr.print_lk, "pr");
    pr.locking = 1;
}

```

接着，定义辅助函数：`printint`用于在%d中的整数输出，`printptr`实现在%p中的指针输出：

```

static char digits[] = "0123456789abcdef";

// 辅助函数
static void
printint(int xx, int base, int sign)
{

```

```

char buf[16];
int i;
uint32 x;

if(sign && (sign = xx < 0))
    x = -xx;
else
    x = xx;

i = 0;
do {
    buf[i++] = digits[x % base];
} while((x /= base) != 0);

if(sign)
    buf[i++] = '-';

while(--i >= 0)
    uart_putc_sync(buf[i]);
}

static void
printptr(uint64 x)
{
    int i;
    uart_putc_sync('0');
    uart_putc_sync('x');
    for (i = 0; i < (sizeof(uint64) * 2); i++, x <<= 4)
        uart_putc_sync(digits[x >> (sizeof(uint64) * 8 - 4)]);
}

```

最后，实现printf，函数功能类似于C语言中的printf，且能识别出%d, %x, %p, %s的标记。注意在输出前后需要分别对输出锁进行上锁和解锁：

```

// Print to the console. only understands %d, %x, %p, %s.
void printf(const char *fmt, ...)
{
    va_list ap;
    int i, c, locking;
    char *s;

    locking = pr.locking;
    if(locking)
        spinlock_acquire(&pr.print_lk);

    if (fmt == 0)
        panic("null fmt");

    va_start(ap, fmt);
    for(i = 0; (c = fmt[i] & 0xff) != 0; i++){
        if(c != '%'){

```

```

        uart_putc_sync(c);
        continue;
    }
    c = fmt[++i] & 0xff;
    if(c == 0)
        break;
    switch(c){
    case 'd':
        printint(va_arg(ap, int), 10, 1);
        break;
    case 'x':
        printint(va_arg(ap, int), 16, 1);
        break;
    case 'p':
        printptr(va_arg(ap, uint64));
        break;
    case 's':
        if((s = va_arg(ap, char*)) == 0)
            s = "(null)";
        for(; *s; s++)
            uart_putc_sync(*s);
        break;
    case '%':
        uart_putc_sync('%');
        break;
    default:
        // Print unknown % sequence to draw attention.
        uart_putc_sync('%');
        uart_putc_sync(c);
        break;
    }
}
va_end(ap);

if(locking)
    spinlock_release(&pr.print_lk);
}

```

于此同时，补充panic和assert函数，用于异常处理：

```

volatile int panicked = 0;

void panic(const char *s)
{
    pr.locking = 0;
    printf("panic: ");
    printf(s);
    printf("\n");
    panicked = 1; // freeze uart output from other CPUs
    for(;;)
        ;
}

```



```
void assert(bool condition, const char* warning)
{
    if(!condition){
        panic(warning);
    }
}
```

2.2 问题与解决方案

2.2.1 常见问题

问题：QEMU启动后无任何输出，仿佛“卡住了”一样

解决方案：对于代码的问题，其中一部分在编译的时候会检查到，从而在终端输出错误信息；然而还有一部分错误（例如引用空指针，除以0等）错误在运行时才会发现，此时qemu在遇到此错误时将会立即结束执行，但不会输出错误信息。（可以使用`ctrl + a`后，按`x`键退出）

所以，需要使用调试的方法对于该项目进行调试，如果发现在运行某行代码后高亮行消失或传送到一个不相关的行，则可知这一行代码可能有问题，需要修正。需要仔细检查代码修正。

对于此例，可能需要检查链接脚本的起始地址，uart基地址以及初始化是否正确，且程序是否正确加载

问题：输出乱码或不完整

解决方案：需要检查uart的初始化是否充分，其次，需要检查发送字符的间隔（可能是没有开关中断，又或者波特率不对），最后需要查看字符是否正确终止。

2.2.2 自己实现的时候遇到的问题

问题：测试输出较大的数字时程序输出的数据很小

解决方案：通过测试发现`xi`设为了`uint8`，需要增大其带宽，使之称为`uint32`即可

问题：正常使用锁的时候发现`panic("acquire")`

解决方案：仔细检查锁的分配程序是否发生错误。同时，由于锁的结构体记录的是`cpuid`，所以在初始化或释放后，其持有者应该设为-1，即无CPU持有。

2.3 源码理解总结

对于项目代码从源代码到可执行程序，首先项目中的`makefile`控制qemu将源文件编译并且输出目标文件`.o`、`.d`，接下来`kernel.ld`链接文件控制系统将目标文件中的各个段按照格式拼接，并控制物理地址，从而形成完整的操作系统程序。

在xv6操作系统启动时，操作系统首先进入`entry.s`中，为各个CPU核开启栈空间；之后进入`start.c`进行硬件相关的初始化（关闭中断，关闭分页等），最后进入`main.c`的`main`函数中，初始化各个功能模块，并输出当前信息。

`uart.c`作为系统和硬件的接口，控制硬件对窗口进行数据的输入输出。在输出前，需要对uart进行初始化，调整波特率，输出位宽，输出模式等，且在输出字符前需要检查并TX队列是否闲置。文件`print.c`调用uart的输出函

数，从而实现printf的功能。

3. 测试验证部分

3.1 功能测试结果

在实现如上功能后，使用`make qemu`对于项目进行编译运行，最终结果如下：

```
riptide@admin:~/whu-oslab-lab1$ make qemu
make[2]: Leaving directory '/home/riptide/whu-oslab-lab1/kernel/proc'
ls: cannot access './*/*/*.o': No such file or directory
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel.ld -o ../kernel-qemu ./boot/entry.o ./boot/main.o ./boot
./lib/spinlock.o ./proc/proc.o
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaulting to 0000000080000000
make[1]: Leaving directory '/home/riptide/whu-oslab-lab1/kernel'
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
```

说明双核程序正常运行并输出了当前信息

3.2 异常测试

为了加深对于锁的理解，设计如下两个有关于锁的测试，通过探讨对于输出的问题，从而加深理解：

3.2.1 并行加法测试

在`main.c`中编写如下代码：

```
#include "riscv.h"
#include "lib/print.h"

volatile static int started = 0;

volatile static int sum = 0;

int main()
{
    int cpuid = r_tp();
    if(cpuid == 0) {
        print_init();
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
        for(int i = 0; i < 1000000; i++)
            sum++;
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    } else {
        while(started == 0);
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);
        for(int i = 0; i < 1000000; i++)
            sum++;
    }
}
```

```
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    }
    while (1);
}
```

由代码可知，我们的预期是后report的cpu应该告诉我们sum = 2000000，但是实际结果可能如下：

```
cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1128497
cpu 1 report: sum = 1143332
```

可知sum++这个操作不是原子操作，当两个CPU核都处理此类操作时，将会产生冲突，从而导致可能的错误。为了实现操作，将sum++提出并作为一个函数，使用锁calculate_lock对其做原子化处理，修改后代码如下：

```
#include "riscv.h"
#include "lib/print.h"
#include "lib/lock.h"
#include "proc/proc.h"

volatile static int started = 0;

volatile static int sum = 0;

static spinlock_t calculate_lock;

void plus1(){
    spinlock_acquire(&calculate_lock);
    sum++;
    spinlock_release(&calculate_lock);
}

void main()
{
    int cpuid = mycpuid();
    if(cpuid == 0){
        print_init();
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;

        for(int i = 0; i < 1000000; i++){
            plus1();
        }
        printf("cpu %d report: sum = %d\n", cpuid, sum);
    }
    else{
        while(started == 0) {};
```

```
    __sync_synchronize();
    printf("cpu %d is booting!\n", cpuid);

    for(int i = 0; i < 1000000; i++){
        plus1();
    }
    printf("cpu %d report: sum = %d\n", cpuid, sum);

}

while(1);
}
```

运行代码后结果为：

```
cpu 0 is booting!
cpu 1 is booting!
cpu 0 report: sum = 1998255
cpu 1 report: sum = 2000000
```

很明显，通过互斥锁的设置，成功实现了临界区的互斥访问。

3.2.2 重叠输出测试

接下来，为了深入了解锁的运作机理，尝试去除printf中的锁从而测试输出。测试的main函数如下：

```
void main()
{
    int cpuid = mycpuid();
    if(cpuid == 0){
        print_init();
        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;

        printf("cpu %d report: The first sentence\n", cpuid);
    }
    else{
        while(started == 0) {};
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);

        printf("cpu %d report: The another sentence\n", cpuid);
    }

    while(1){};
}
```

去除printf函数中对应的锁的模块，输出如下：

```
cpu 0 is booting!  
cpu 0 repocpu 1 is booting!  
rcpu 1 report: The another sentence  
t: The first sentence
```

恢复printf，输出如下：

```
cpu 0 is booting!  
cpu 0 report: The first sentence  
cpu 1 is booting!  
cpu 1 report: The another sentence
```

可见如果没有自旋锁维护printf函数，那么输出将会十分混乱。可见其重要性。

4. 结论与心得

通过本次实验，我收获颇多，具体如下：

首先，我理解了xv6操作系统启动的机理；其次，我了解了xv6中串口uart的基本初始化和输出的基本结构。最后，通过额外任务中的并行加法和重叠输出测试，我加深了对于锁的印象。

实验 2：页表与内存管理

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)，其处于lab2分支。

0. 实验任务和目标

通过深入分析xv6 的内存管理系统，理解虚拟内存的工作原理，独立实现物理内存分配器和虚拟内存的页表管理系统。

1. 系统设计

1.1 架构设计说明

本risc-v系统采用与xv6相同的物理内存和虚拟内存分配系统，其架构如下：

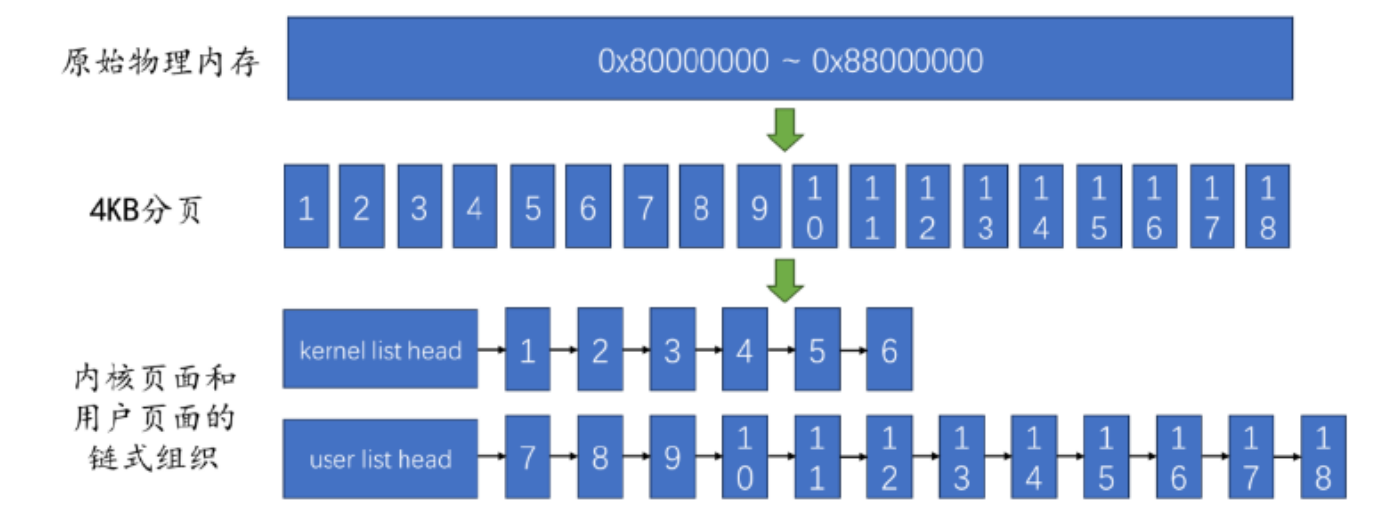
1.1.1 物理内存的分配架构

该操作系统采用和xv6相同的物理页分页存储结构，其将物理内存空间划分为若干大小相同的页面，每个页面大小为4KB（即4096字节）。

故而，该物理内存的逻辑地址结构如下。可以知道，该系统的物理地址的位数为56位（实际上使用64位的uint64存储）



对于该操作系统，物理页的分配将采用空闲物理页链表进行管理，每当需要分配或释放物理页时，在空闲物理页链表头进行物理页的分配和释放。同时为了保障内核运行的安全性，将内核的物理页与用户的物理页分开，从而保证内核物理页不会意外耗尽，从而导致错误。物理内存管理方式的示意图如下：



1.1.2 虚拟内存的分配架构

与xv6相同，本操作系统采用使用RISC-V体系结构中的SV39作为虚拟内存的设计规范，采用Sv39的三级页表机制。

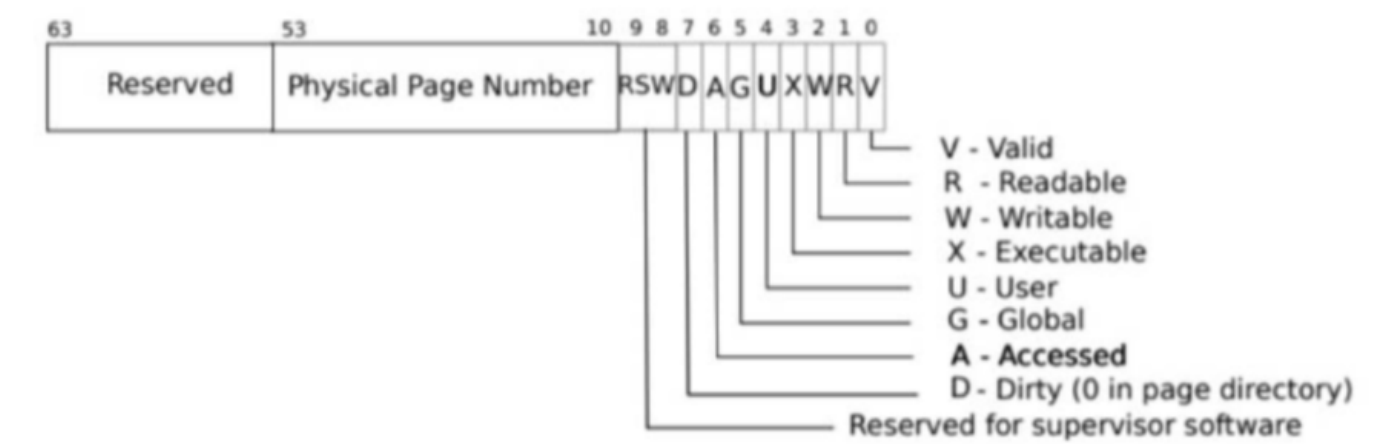
页表放置于物理空间，每一页的大小与物理页相同，为4KB（4096字节）。每一个页表可以视为一个很大的数组，存放每个虚拟地址对应的物理地址以及页表状态位。在页表中每一个页表项占用8字节的空间，共512项。

对于xv6操作系统中的虚拟地址结构如下：



其中，VPN2~VPN0代表虚拟地址在各级页表中的项，占用9位（ $512 = 2^9$ ），offset代表对应页中的页偏移，占用12位（ $4096 = 2^{12}$ ）

在页表中，页表项（pte）的结构如下：



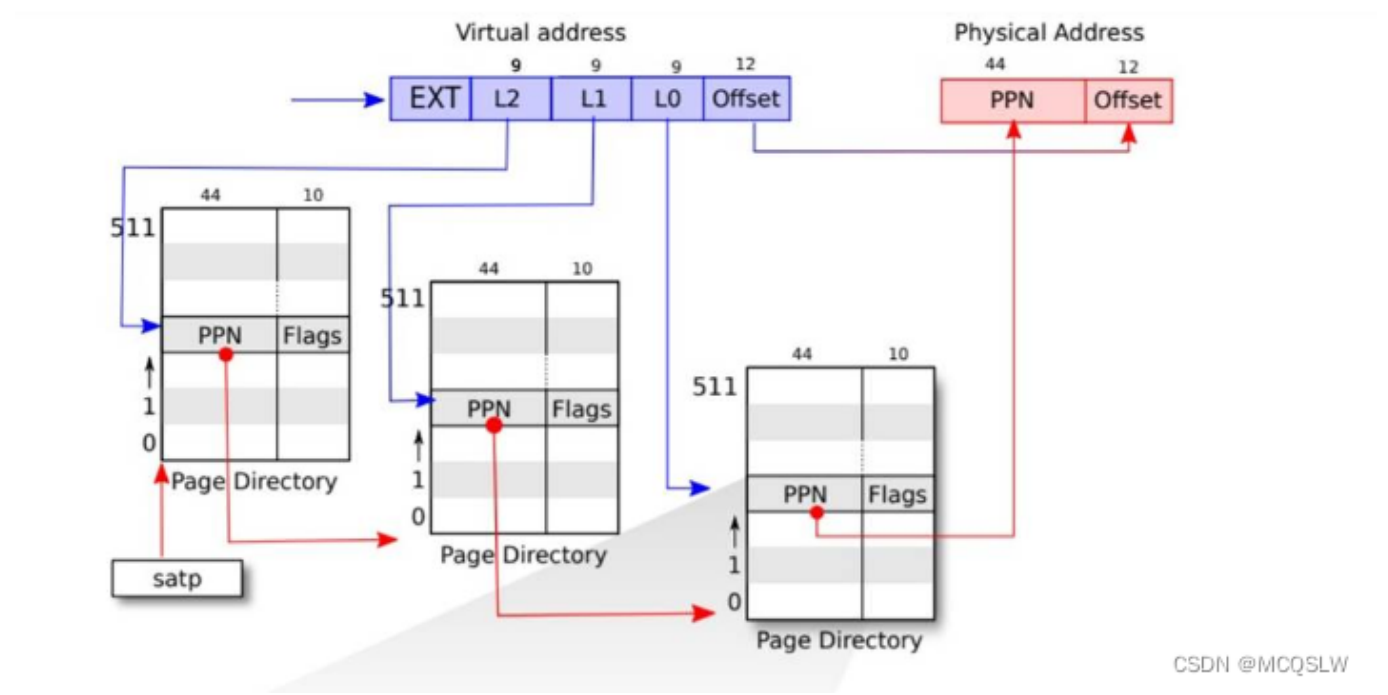
注：此页表项实际有效位为54位，最高10位保留。从而每一个页表项实际占用64位即8字节的空间（使用uint64存储）

其中，物理页号PPN占用44位，表示其对应的物理页号（实际的物理页或者下一级页表），而低10位为各个标志位，其中部分位的作用如下：

- V：合法位（Valid），表示该页是否是一个有效的页
- R, W, X：读（Read），写（Write），执行（Execute）位，分别表示该页是否可读/可写/可执行
- U：用户态位（User），表示用户是否可以访问这个页

在实际使用中，可以通过改变相关位的值，从而控制各个位的控制情况。

当需要访问物理地址时，其先访问首级页表，得到一级页表的物理地址，其次通过二级页表的物理地址访问页表，得到三级页表的物理地址，最后通过访问三级页表从而得到目标页的物理地址。其示意图如下图所示：



1.2 关键数据结构

1.2.1 物理页表结构kalloc.c（对应项目源码pmem.c）

在实际物理页分配中，采用如下节点结构从而维护各个物理页节点：

```
typedef struct page_node{
    struct page_node* next;
} page_node_t;
```

这种结构类似于链表，将各个物理页链接起来。当需要分配或释放物理页时，在表头插入或提取物理页即可。

本系统采用内核物理页和用户物理页分离的机制，故采用如下数据结构从而维护内核态物理页和用户态物理页的信息：

```
// 许多物理页构成一个可分配的区域
typedef struct alloc_region {
    uint64 begin;           // 起始物理地址
    uint64 end;             // 终止物理地址
    spinlock_t lk;          // 自旋锁(保护下面两个变量)
    uint32 allocable;        // 可分配页面数
```



```
    page_node_t list_head; // 可分配链的链头节点
}alloc_region_t;          // 内核和用户可分配的物理页分开

static alloc_region_t kern_region, user_region;
```

这个物理结构包含可分配区域物理地址，自旋锁，可分配链表的头节点等，用于记录各个态相关的物理页信息和可分配物理页链表。对于具体的机制请看下文。

1.2.2 虚拟页表结构vm.c

对于虚拟页表结构，本系统采用三级页表结构存储页表项和链接物理页的信息。由于三级页表机制已经在1.1.2节讲述过，这里不再赘述。

由于页表项直接操控物理地址，所以实际上使用uint64作为页表的类型，其指针的值即为对应的物理地址，而指向的内容即为页内部的内容。如下所示：

```
// 页表项
typedef uint64 pte_t;

// 顶级页表
typedef uint64* pgtbl_t;

pgtbl_t kernel_pagetable; // 页表
```

在实际使用中，将对应的物理页看成一个很大的数组，通过向这个数组内页表项的读写，即可实现对于页表项的管理。

1.2.3 内存管理相关riscv.h

riscv.h作为辅助文件，对于内存管理存放了相关的宏辅助计算以及位掩码等。当需要使用时直接调用宏即可。

对于所有内存管理相关宏的内容以及对应功能，详情请看2.1.5节，这里不再赘述。

1.3 设计决策理由

对于操作系统的物理内存，本操作系统采用分页管理，且采用空闲页链表存储当前的空闲页。操作系统在链表的表头执行物理页的分配和释放，其性能较为高效。

对于操作系统的虚拟内存，本操作系统和xv6一样，采用Sv39作为页表的规范。这种规范能够以较高效率进行页表项的申请和释放，从而能够对于虚拟内存进行有效的管理。

1.4 与 xv6 对比分析

本操作系统的结构与xv6对比如下：

在物理页分配上，本系统和xv6同样采用按页分配且使用空闲页链表分配的方法，但本系统将内核态物理页和用户态物理页区分开来，从而防止用户态过度申请物理页从而导致内核态卡死的情形，保证了安全性。

在虚拟页分配上，本系统和xv6同样采用Sv39作为页表的规范，且使用三级页表的结构。然而，本操作系统额外新增了一个辅助的输出页表功能，从而使得系统能够通过输出页表的方式进行调整。

2. 实验过程部分

2.1 实现步骤记录

2.1.1 字符串相关功能实现

在开始书写有关于内存分配的代码前，首先需要实现字符串的相关功能实现。字符串的部分指针操作功能如memset在内存分配中也有使用，而本操作系统项目不包括任何任何C标准库，所以需要对函数进行相关的实现。

需要实现的字符串相关操作函数有：

```
void* memset(void *dst, int c, uint32 n);
int memcmp(const void *v1, const void *v2, uint32 n);
void* memmove(void *dst, const void *src, uint32 n);

void* memcpy(void *dst, const void *src, uint32 n);
int strncmp(const char *p, const char *q, uint32 n);
char* strncpy(char *s, const char *t, int n);

char* safestrcpy(char *s, const char *t, int n);
int strlen(const char *s);
```

其中，memset函数负责相关内存区域的按字节设置，其代码如下：

```
void*
memset(void *dst, int c, uint32 n)
{
    char *cdst = (char *) dst;
    int i;
    for(i = 0; i < n; i++){
        cdst[i] = c;
    }
    return dst;
}
```

对于其他字符串实现函数，由于其与内存分配关系不大，所以详情请看代码库代码，在这里不再赘述。

2.1.2 深入理解Sv39页表机制

在实现类似于xv6的页表分配机制之前，还需要深入理解Sv39页表机制。此部分在1.1.2节已有阐述，这里不再赘述。

对于该部分思考题的回答如下：

对于39位地址的分解：

每个VPN段的作用是什么？

- 各个VPN段表示在各级表项中的项数，表示在页表中对应第几个页表项。

为什么每级VPN是9位而不是其他位数？

- 可以知道页表中每个页表项占用8字节，共4096字节，所以每个页表有512项，需要使用9位来表示。

对于页表项的格式：

中间级页表项的R/W/X位应该如何设置？

- 在页表项中，R/W/X位主要放置于页表项的低10位中，为了设置这些项，我们需要将在页表中的页表项的低10位进行设置。
- 在实际编程中，为了方便，在`ris cv.h`中设立了一些宏，这些宏定义了对应位对应的位掩码，通过将页表项与位掩码进行位运算，就能启用/禁止相关页表项的功能

物理页号（PPN）的提取方式

- 可知在页表项中前44位为物理页号，低10位为控制位，所以只需要将页表项右移10位即可得到物理页号。

对于“3级页表”：

为什么选择三级页表而不是二级或四级？

- 三级页表将原来的27位页码分为三部分，每部分9位，通过多层次的页表结构来优化内存使用。
- 三级页表相比于二级页表能够访问到更大的虚拟空间，相比于四级页表寻址会更方便快捷。

如何理解“页表也存储在物理内存中”？

- 页表实际上是一种数据结构，其核心作用是记录虚拟页号到物理页框号的映射关系。
- 为了快速的访问所需要的物理页，需要将页表存储在内存上，从而以 $O(1)$ 的时间复杂度查询需要的页表项。代价则为巨大的空间浪费。
- 总之，页表存放在物理内存中实际上是一种时间和空间上的折中。

2.1.3 分析xv6的物理内存分配器

首先，阅读xv6中的`kalloc.c`，了解其核心数据结构：

```
struct run {
    struct run *next;
};
```

此结构类似链表，由大小相等的物理页按序链接而成。其精巧性体现在两方面：首先，定位一个页仅需其起始指针，无需保存页的大小等元数据。其次，由于页是定长且连续的，任何物理地址都能快速“归位”到其所在页的起始地址，这为物理页管理器的初始化等操作带来了极大便利。

其次，阅读`kinit()`函数，掌握初始化进程，其代码如下：

```
void
kinit()
{
    initlock(&kmem.lock, "kmem");
    freerange(end, (void*)PHYSTOP);
}
```

注：这个函数使用了freerange函数，其代码如下：

```
void
freerange(void *pa_start, void *pa_end)
{
    char *p;
    p = (char*)PGROUNDUP((uint64)pa_start);
    for(; p + PGSIZE <= (char*)pa_end; p += PGSIZE)
        kfree(p);
}
```

在这里，kinit()函数调用freerange函数进行页表项的初始化设置，用于分配的内存范围为end到PHYSTOP，从宏定义中可知其大小为128*1024*1024字节。而freerange中，函数使用循环语句，每次将其与页的起始地址对齐，从而逐个将空闲页链表建立。

这个函数按页进行对齐，因为页表项指针所在地址为页的起始地址，且页表项定长且连续，所以需要通过页对齐的方式（步长PGSIZE，即4096字节）从而进行页对齐的初始化。

然后，通过阅读参考代码理解kalloc()和kfree()的实现。代码如下所示：

```
void
kfree(void *pa)
{
    struct run *r;

    if(((uint64)pa % PGSIZE) != 0 || (char*)pa < end || (uint64)pa >= PHYSTOP)
        panic("kfree");

    // Fill with junk to catch dangling refs.
    memset(pa, 1, PGSIZE);

    r = (struct run*)pa;

    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}
```

```
void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);
    r = kmem.freelist;
    if(r)
        kmem.freelist = r->next;
    release(&kmem.lock);

    if(r)
        memset((char*)r, 5, PGSIZE); // fill with junk
    return (void*)r;
}
```

由代码可知，分配算法中，在链表头插入或删除物理页所需时间复杂度为 $O(1)$ ，为物理页填充字节所需时间复杂度为 $O(PGSIZE)$ 即 $O(1)$ ，这样可知分配算法的时间复杂度为 $O(1)$

为了防止重复释放，代码通过检查其物理页的地址从而判断。如果该物理地址未对齐或其不在特定范围内，则函数将会启用panic函数从而报错

这种链表项分配物理页的设计，在数据结构上，很好的利用了物理页等长且连续的特点，除了链表指针项无需其他元数据；同时其物理页的分配和回收时间复杂度低，适合快速需要分配的情形。然而，这种设计将用户态页面和内核态页面统一成一个链表free_list，这样在安全性上可能会发生内核态因过度申请物理页从而卡死的情况，安全性上有待改善。

2.1.4 设计并实现自己的物理内存管理器

参考xv6的代码，设计并实现物理内存管理器。在这里与xv6不同，将用户态页面和内核态页面区分开来，从而保障安全性。

在这里使用物理页节点的链表数据结构page_node如下：

```
// 物理页节点
typedef struct page_node{
    struct page_node* next;
} page_node_t;
```

于此同时，设立一个数据结构alloc_region，用于管理用户态和内核态中的物理页信息。其结构如下：

```
// 许多物理页构成一个可分配的区域
typedef struct alloc_region {
    uint64 begin;           // 起始物理地址
    uint64 end;             // 终止物理地址
    spinlock_t lk;          // 自旋锁(保护下面两个变量)
    uint32 allocable;        // 可分配页面数
    page_node_t list_head;  // 可分配链的链头节点
```

```

}alloc_region_t;           // 内核和用户可分配的物理页分开

static alloc_region_t kern_region, user_region;

```

内存管理器的初始化代码如下：

```

#define KERNEL_PAGES 2048

void pmem_init(void){
    kern_region.begin = (uint64)&ALLOC_BEGIN;
    kern_region.end = (uint64)&ALLOC_BEGIN + KERNEL_PAGES * PGSIZE;
    kern_region.allocable = 0;
    kern_region.list_head.next = NULL;
    spinlock_init(&kern_region.lk, "kern_pmem_lock");
    for (uint64 addr = kern_region.begin; addr < kern_region.end; addr += PGSIZE)
    {
        // 将物理页首地址转换为 page_node_t 指针
        page_node_t *page = (page_node_t *)addr;
        // 使用头插法插入链表
        page->next = kern_region.list_head.next;
        kern_region.list_head.next = page;
        kern_region.allocable++; // 增加可分配页数
    }

    user_region.begin = (uint64)&ALLOC_BEGIN + KERNEL_PAGES * PGSIZE;
    user_region.end = (uint64)&ALLOC_END;
    user_region.allocable = 0;
    user_region.list_head.next = NULL;
    spinlock_init(&user_region.lk, "user_pmem_lock");
    for (uint64 addr = user_region.begin; addr < user_region.end; addr += PGSIZE)
    {
        // 将物理页首地址转换为 page_node_t 指针
        page_node_t *page = (page_node_t *)addr;
        // 使用头插法插入链表
        page->next = user_region.list_head.next;
        user_region.list_head.next = page;
        user_region.allocable++; // 增加可分配页数
    }
}

```

如上代码使用了for循环，从而实现类似于xv6中的物理页分配。其中，使用ALLOC_BEGIN和ALLOC_END从而确定可用的物理内存范围，其在kernel.ld中定义。kernel_pages定义了内核态可分配的页数为2048

对于单个物理页的分配和释放，其代码如下：

```

void* pmem_alloc(bool in_kernel){
    alloc_region_t *region = in_kernel ? &kern_region : &user_region;
    spinlock_acquire(&region->lk); // 上锁

```

```

    if(region->allocable == 0){
        spinlock_release(&region->lk);
        panic("alloc pages not enough");
        return NULL; // 无可分页
    }

    page_node_t *page = region->list_head.next;
    if(page)
        region->list_head.next = page->next;
    region->allocable--;

    spinlock_release(&region->lk); // 解锁

    return (void*)page;
}

void pmem_free(uint64 page, bool in_kernel){
    if((page % PGSIZE) != 0 || (char*)page < ALLOC_BEGIN || (char *)page >=
    ALLOC_END)
        panic("kfree");

    alloc_region_t *region = in_kernel ? &kern_region : &user_region;
    page_node_t *page_ptr = (page_node_t *)page;

    spinlock_acquire(&region->lk); // 上锁

    page_ptr->next = kern_region.list_head.next;
    kern_region.list_head.next = page_ptr;
    region->allocable++;

    spinlock_release(&region->lk); // 解锁
}

```

与xv6代码不同的是，函数参数中新增了一个in_kernel参数，从而判断需要分配/释放的物理页来自哪一个链表头，其他方面类似于xv6代码。

2.1.5 理解xv6的页表管理

接下来，在实现页表管理系统之前需要理解xv6的页表管理，其主要过程如下：

首先，阅读walk()函数，函数内容如下：

```

pte_t *
walk(pagetable_t pagetable, uint64 va, int alloc)
{
    if(va >= MAXVA)
        panic("walk");

    for(int level = 2; level > 0; level--) {
        pte_t *pte = &pagetable[PX(level, va)];
        if(*pte & PTE_V) {

```

```

    pagetable = (pagetable_t)PTE2PA(*pte);
} else {
    if(!alloc || (pagetable = (pde_t*)kalloc()) == 0)
        return 0;
    memset(pagetable, 0, PGSIZE);
    *pte = PA2PTE(pagetable) | PTE_V;
}
}
return &pagetable[PX(0, va)];
}

```

注：这里使用到了`riscv.h`中的一些宏定义如下：

```

#define PGSHIFT 12 // bits of offset within a page
#define PTE2PA(pte) (((pte) >> 10) << 12)
#define PXMASK 0x1FF // 9 bits
#define PXSHIFT(level) (PGSHIFT+(9*(level)))
#define PX(level, va) ((((uint64) (va)) >> PXSHIFT(level)) & PXMASK)

```

这个函数使用了for循环，通过逐级页表项寻找从而找到对应的页表项。在这里，`PX(level, va)`用于从虚拟地址中提取各级的索引，其通过对虚拟地址进行右移指定位数并提取后九位，从而得到对应的索引；`PTE2PA`宏负责将页表项中的物理地址部分提取出来，从而得到下一级页表的地址或者目标页表的地址。

在循环寻找对应的页表项中，函数还会判断页表项是否有效。当遇到无效页表项时，`alloc`将会用于控制“在寻找相关页时是否启用对于无效页表的分配”。如果`alloc`项为1，则函数将会尝试新建一个物理页用于存放页表项，并将其标记为有效并继续寻找；如果分配失败或者`alloc`项为0，则循环停止，返回0，表示没有找到页。

其次，阅读`mappages()`的代码，研究页表项的映射建立：

```

int
mappages(pagetable_t pagetable, uint64 va, uint64 size, uint64 pa, int perm)
{
    uint64 a, last;
    pte_t *pte;

    if(size == 0)
        panic("mappages: size");

    a = PGROUNDDOWN(va);
    last = PGROUNDDOWN(va + size - 1);
    for(;;){
        if((pte = walk(pagetable, a, 1)) == 0)
            return -1;
        if(*pte & PTE_V)
            panic("mappages: remap");
        *pte = PA2PTE(pa) | perm | PTE_V;
        if(a == last)
            break;
        a += PGSIZE;
    }
}

```



```
    pa += PGSIZE;
}
return 0;
}
```

这个函数使用`PGROUNDDOWN`宏，实现地址和分配大小的取整，从而实现对齐，并且使用`for`语句，对于各个页表项进行对应物理表项的分配，从而实现页表项和物理表项的对应关系。

对于页表项中的权限位，由于其体现在页表项的地位，所以其设置是通过页表项的位运算从而实现的。

页表项和物理表项的设置设立了较为完善的处理，当分配物理页失败时，函数返回-1，表示分配失败；当发现新分配的物理页已经是合法页时，则启用`panic`函数，表示该页已被分配，防止重复分配。

同时，需要通过阅读`riscv.h`中的各个宏，理解地址转换定义。用于内存管理的宏定义如下：

```
#define PGSIZE 4096 // bytes per page
#define PGSHIFT 12 // bits of offset within a page

#define PGROUNDUP(sz) (((sz)+PGSIZE-1) & ~(PGSIZE-1))
#define PGROUNDDOWN(a) (((a)) & ~(PGSIZE-1))

#define PTE_V (1L << 0) // valid
#define PTE_R (1L << 1)
#define PTE_W (1L << 2)
#define PTE_X (1L << 3)
#define PTE_U (1L << 4) // user can access

#define PA2PTE(pa) (((uint64)pa) >> 12) << 10
#define PTE2PA(pte) (((pte) >> 10) << 12)
#define PTE_FLAGS(pte) ((pte) & 0x3FF)

#define PXMASK 0x1FF // 9 bits
#define PXSHIFT(level) (PGSHIFT+(9*(level)))
#define PX(level, va) (((uint64)(va)) >> PXSHIFT(level)) & PXMASK

#define MAXVA (1L << (9 + 9 + 9 + 12 - 1))
```

如上各个宏的功能为：

- `PGSIZE`：表示每个页的大小为4096字节
- `PGSHIFT`：业内偏移所占用的位数
- `PGROUNDUP`：将数值向上取整到整数个页
- `PGROUNDDOWN`：将数值向下取整到整数个页
- `PTE_V`、`PTE_R`等：各个控制位的位掩码，用于控制位的读写
- `PA2PTE`：将物理地址转化为页表项
- `PTE2PA`：将页表项转化为物理地址
- `PTE_FLAGS`：提取页表项后10位的控制位
- `PX(level, va)`：提取虚拟地址中的各级索引
- `MAXVA`：虚拟地址的最大值，如果超过，将会视为非法。

2.1.6 设计并实现自己的页表管理系统

参考xv6的参考代码，实现自己的页表管理系统。具体流程如下：

首先，参考xv6的参考代码实现获取页表项的代码，其内容如下。

```
pte_t* vm_getpte(pgtbl_t pgtbl, uint64 va, bool alloc){
    for(int level = 2; level > 0; level--) {
        pte_t *pte = &pgtbl[VA_TO_VPN(va, level)];
        if(*pte & PTE_V) {
            pgtbl = (pgtbl_t)PTE_TO_PA(*pte);
        } else {
            if(!alloc || (pgtbl = (pte_t*)pmem_alloc(true)) == 0)
                return 0;
            memset(pgtbl, 0, PGSIZE);
            *pte = PA_TO_PTE(pgtbl) | PTE_V;
        }
    }
    return &pgtbl[VA_TO_VPN(va, 0)];
}
```

其次，实现页表的分配和释放，具体代码如下：

```
void vm_mappages(pgtbl_t pgtbl, uint64 va, uint64 pa, uint64 len, int perm){
    uint64 a, last;
    pte_t *pte;

    if(len == 0)
        panic("mappages: len");

    a = PG_ROUND_DOWN(va);
    last = PG_ROUND_DOWN(va + len - 1);
    for(;;){
        if((pte = vm_getpte(pgtbl, a, 1)) == 0)
            panic("mappages: pte==0");
        if(*pte & PTE_V)
            panic("mappages: remap");
        *pte = PA_TO_PTE(pa) | perm | PTE_V;
        if(a == last) break;
        a += PGSIZE;
        pa += PGSIZE;
    }
}

void vm_unmappages(pgtbl_t pgtbl, uint64 va, uint64 len, bool freeit){
    uint64 a;
    pte_t *pte;

    if((va % PGSIZE) != 0)
        panic("uvmunmap: not aligned");
}
```

```

int len_rounded = PG_ROUND_UP(len);
for(a = va; a < va + len_rounded; a += PGSIZE){
    if((pte = vm_getpte(pgtbl, a, 0)) == 0)
        panic("uvmunmap: vm_getpte");
    if((*pte & PTE_V) == 0)
        panic("uvmunmap: not mapped");
    if(PTE_FLAGS(*pte) == PTE_V)
        panic("uvmunmap: not a leaf");
    if(freeit){
        uint64 pa = PTE_TO_PA(*pte);
        pmem_free(pa, true);
    }
    *pte = 0;
}
}

```

除此以外，该系统新建了一个`vm_print`函数，其调用辅助递归函数`vm_print_helper`从而输出对应的页表信息。当然，为了防止死循环，该函数在`level < 2`的情况下才会往下递归输出信息。

```

void vm_print_helper(pgtbl_t pgtable, int level){
    for(int i=0; i<512; i++){
        pte_t *pte = &pgtable[i];
        pgtable_t pa = (pgtable_t)PTE_TO_PA(*pte);
        if(pte && pa){
            for(int k=0; k<level; k++) printf("    ");
            printf("%d: pte %p, pa %p\n", i, pte, (PTE_TO_PA(*pte)));
            if(level < 2)
                vm_print_helper(pa, level+1); // 页表项 < 2
        }
    }
}

void vm_print(pgtbl_t pgtable){
    vm_print_helper(pgtbl, 0);
}

```

2.1.7 启用虚拟内存

参考xv6的内核初始化，为自己的操作系统启用虚拟内存。具体流程如下：

首先，研读`kvm_init()`，并实现操作系统中的页表创建。代码如下所示：

```

void kvm_init(){
    kernel_pagetable = kvm_create();
}

// 创建页表并且提前分配
pgtbl_t kvm_create(){

```

```

pgtbl_t kpgtbl;

kpgtbl = (pgtbl_t) pmem_alloc(true);
memset(kpgtbl, 0, PGSIZE);

// UART 寄存器
vm_mappages(kpgtbl, UART_BASE, UART_BASE, PGSIZE, PTE_R | PTE_W);

// 中断控制器CLINT
vm_mappages(kpgtbl, CLINT_BASE, CLINT_BASE, PGSIZE, PTE_R | PTE_W);

// 中断处理PLIC
vm_mappages(kpgtbl, PLIC_BASE, PLIC_BASE, 0x400000, PTE_R | PTE_W);

// 内核代码段KERNEL
vm_mappages(kpgtbl, KERNEL_BASE, KERNEL_BASE, (uint64)etext-KERNEL_BASE, PTE_R
| PTE_X);

// 内核数据区KERNEL_DATA
vm_mappages(kpgtbl, (uint64)etext, (uint64)etext, PHYSTOP-(uint64)etext, PTE_R
| PTE_W);

return kpgtbl;
}

```

可以知道，在如上代码中，UART寄存器、中断控制相关CLINT，PLIC、内核代码段KERNEL、内核数据段KERNEL_DATA都需要映射，并采用恒等映射。因为这些内存区域在禁用页表分配时启用，为了使得在启用页表分配后这些地址仍然能够访问到正确的物理地址，需要将其进行恒等映射初始化。

这个函数调用`vm_mappages`建立虚拟内存以及物理内存的对应关系，对于设备内存的权限设置则采用向`perm`参数中传递位掩码从而实现，这样在建立虚拟页的时候即可将`perm`参数写入控制位，从而控制对应权限。

其次，参考xv6中`kvm_inithart()`的机理，启用操作系统的页表激活。代码如下所示：

```

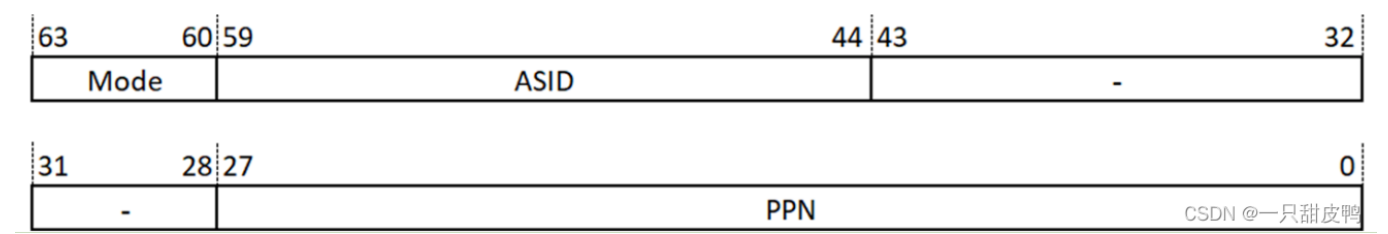
void    kvm_inithart(){
    // wait for any previous writes to the page table memory to finish.
    sfence_vma();

    w_satp(MAKE_SATP(kernel_pagetable));

    // flush stale entries from the TLB.
    sfence_vma();
}

```

在这段代码中，`satp`寄存器的格式如下：



其中，PPN表示第一级页表基地址的物理页号，ASID表示当前进程的ASID号，Mode表示当前模式，其类型如下：

SXLEN=32		
Value	Name	Description
0	Bare	No translation or protection.
1	Sv32	Page-based 32-bit virtual addressing (see Section 5.3).
SXLEN=64		
Value	Name	Description
0	Bare	No translation or protection.
1-7	-	Reserved for standard use
8	Sv39	Page-based 39-bit virtual addressing (see Section 5.4).
9	Sv48	Page-based 48-bit virtual addressing (see Section 5.5).
10	Sv57	Page-based 57-bit virtual addressing (see Section 5.6).
11	Sv64	Reserved for page-based 64-bit virtual addressing.
12-13	-	Reserved for standard use
14-15	-	Designated for custom use

Table 23. Encoding of satp MODE field.

对于本操作系统，采用Sv39作为页表规范，所以在设置的时候，需要将当前模式设置为Sv39，并将当前页表基地址的物理页号写入，下列宏即表示了satp应该设置的内容：

```
#define SATP_SV39 (8L << 60)
#define MAKE_SATP(pagetable) (SATP_SV39 | (((uint64)pagetable) >> 12))
```

于此同时，sfence.vma指令是RISC-V架构中的一个特权指令。它主要用于刷新与地址转换相关的TLB缓存，以确保虚拟地址到物理地址转换的正确性。在激活页表前后，需要使用此指令对于TLB缓存进行刷新，以保证地址转换的正确性。

2.2 问题与解决方案

2.2.1 常见问题

- 问题：启用分页后系统崩溃
- 解决方案：需要检查内核代码，栈空间，设备地址等是否正确映射。在启用前后，可以打印关键地址的映射状态，从而调试。
- 问题：页表映射失败
- 解决方案：需要检查地址是否对齐，内存是否不足，并检查当前页表的权限是否冲突
- 问题：地址转换错误

解决方案：需要检查各个宏的计算是否正确，同时检查PTE的格式是否符合RISC-V规范。如果出错，可能会发生如`panic: remap`的问题

2.2.2 自己遇见的问题

问题：在测试中，很容易产生内核物理表项不足的问题（如`panic: pmem alloc not enough`）

解决方案：需要加大为内核物理页表项的分配个数（实际上，通过调试，可以发现总共可供分配的页表项约为30000个，所以可以适当增大为内核的物理页分配）

2.3 源码理解总结

对于操作系统的物理内存，本操作系统采用分页管理，且采用空闲页链表存储当前的空闲页。由于空闲页等长且连续，所以只需要在页的开头放置链表指针即可组织，无需额外数据。操作系统在链表的表头执行物理页的分配和释放，其性能较为高效。

对于操作系统的虚拟内存，本操作系统和xv6一样，采用Sv39作为页表的规范。本操作系统采用三级页表机制，每张页表有512个页表项，每个页表项包含对应的物理地址和控制位。在查找虚拟地址对应的物理页时，首先利用虚拟地址逐级寻找页表，从而得到物理页号，然后物理页号和业内偏移组合形成物理地址。通过物理地址和状态位的编辑，系统可以建立或取消物理页和虚拟页的链接，从而对于系统的内存进行管理。

物理内存和虚拟内存存在启用前都需要初始化。其中，在系统初始化页表项时，需要对各个物理地址进行恒等映射，使得各个地址映射正确，能找到原来的物理地址；而在激活虚拟分页表项前后，除了写入satp寄存器，还需要在初始化前后刷新TLB缓冲区，从而保证映射正确。

3. 测试验证部分

在编写完成后，为了测试当前系统的有效性，还需要对这个系统进行测试，具体测试如下：

3.1 输出完好性检测

在设置完物理内存以及虚拟内存的初始化之后，理论上原有的输出不应该被改变，故重新运行整个程序，在启用物理和虚拟地址的分配后，输出如下：

```
riptide@admin:~/whu-oslab$ make clean && make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -MD -mmodel=medany -ffreest
ax -I. -fno-stack-protector -fno-pie -no-pie -I ../include -c proc.c
make[2]: Leaving directory '/home/riptide/whu-oslab/kernel/proc'
ls: cannot access './**/*.o': No such file or directory
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel.ld -o ../kernel-qemu ./boot/entry.o ./boot/main.o ./bo
./lib/spinlock.o ./lib/str.o ./mem/kvm.o ./mem/pmem.o ./proc/proc.o
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaulting to 0000000080000000
make[1]: Leaving directory '/home/riptide/whu-oslab/kernel'
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
```

那么可知，程序的物理和虚拟地址分配正常，在启用物理和虚拟内存的管理后系统仍能正常输出系统信息。

3.2 物理内存分配测试

除了输出完好性检测，还需要对于物理内存管理进行测试。对于系统的物理内存分配，测试代码如下：

```
volatile static int started = 0;

volatile static int over_1 = 0, over_2 = 0;

static int* mem[1024];

int main()
{
    int cpuid = r_tp();

    if(cpuid == 0) {

        print_init();
        pmem_init();

        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;

        for(int i = 0; i < 512; i++) {
            mem[i] = pmem_alloc(true);
            memset(mem[i], 1, PGSIZE);
            printf("mem = %p, data = %d\n", mem[i], mem[i][0]);
        }
        printf("cpu %d alloc over\n", cpuid);
        over_1 = 1;

        while(over_1 == 0 || over_2 == 0);

        for(int i = 0; i < 512; i++)
            pmem_free((uint64)mem[i], true);
        printf("cpu %d free over\n", cpuid);

    } else {

        while(started == 0);
        __sync_synchronize();
        printf("cpu %d is booting!\n", cpuid);

        for(int i = 512; i < 1024; i++) {
            mem[i] = pmem_alloc(true);
            memset(mem[i], 1, PGSIZE);
            printf("mem = %p, data = %d\n", mem[i], mem[i][0]);
        }
        printf("cpu %d alloc over\n", cpuid);
        over_2 = 1;

        while(over_1 == 0 || over_2 == 0);

        for(int i = 512; i < 1024; i++)
            pmem_free((uint64)mem[i], true);
        printf("cpu %d free over\n", cpuid);
    }
}
```

```
    }  
    while (1);  
}
```

在main.c上书写测试代码，代码执行的结果如下：


```
mem = 0x0000000080422000, data = 16843009
mem = 0x0000000080421000, data = 16843009
mem = 0x0000000080420000, data = 16843009
mem = 0x000000008041f000, data = 16843009
mem = 0x000000008041e000, data = 16843009
mem = 0x000000008041d000, data = 16843009
mem = 0x000000008041c000, data = 16843009
mem = 0x000000008041b000, data = 16843009
mem = 0x000000008041a000, data = 16843009
mem = 0x0000000080419000, data = 16843009
mem = 0x0000000080418000, data = 16843009
mem = 0x0000000080417000, data = 16843009
mem = 0x0000000080416000, data = 16843009
mem = 0x0000000080415000, data = 16843009
mem = 0x0000000080414000, data = 16843009
mem = 0x0000000080413000, data = 16843009
mem = 0x0000000080412000, data = 16843009
mem = 0x0000000080411000, data = 16843009
mem = 0x0000000080410000, data = 16843009
mem = 0x000000008040f000, data = 16843009
mem = 0x000000008040e000, data = 16843009
mem = 0x000000008040d000, data = 16843009
mem = 0x000000008040c000, data = 16843009
mem = 0x000000008040b000, data = 16843009
mem = 0x000000008040a000, data = 16843009
mem = 0x0000000080409000, data = 16843009
mem = 0x0000000080408000, data = 16843009
mem = 0x0000000080407000, data = 16843009
mem = 0x0000000080406000, data = 16843009
cpu 1 alloc over
cpu 0 alloc over
cpu 1 free over
cpu 0 free over
QEMU: Terminated
```

可以知道，代码运行正常，物理页正常分配和释放，无panic现象。说明程序物理内存管理代码正确，能够正常执行。

3.3 虚拟内存分配测试

除了输出完好性检测，还需要对于虚拟内存管理进行测试。本节采用了多个测试用例对虚拟内存和页表分配进行了测试，具体测试如下：

3.3.1 页表信息写入测试

本测试用例主要对于页表的初始化以及基本的分配功能进行基本的测试。测试代码如下：

```
void test_pagetable_1(void){
    printf("====Pagetable test 1 started====\n\n");

    pgtbl_t pt = (pgtbl_t) pmem_alloc(true);
    memset(pt, 0, PGSIZE);

    // 测试基本映射
    uint64 va = 0x10000000;
    uint64 pa = (uint64)pmem_alloc(true);
    vm_mappages(pt, va, pa, PGSIZE, PTE_R | PTE_W);

    // 测试地址转换
    pte_t *pte = vm_getpte(pt, va, true);
    assert(pte != 0 && (*pte & PTE_V), "Test2");
    assert(PTE_TO_PA(*pte) == pa, "Test3");

    // 测试权限位
    assert(*pte & PTE_R, "Test4");
    assert(*pte & PTE_W, "Test5");
    assert(!(*pte & PTE_X), "Test6");

    printf("====Pagetable test 1 ended====\n\n");
}
```

本测试用例测试了基本映射，地址转换，测试权限位等，采用`assert`从而确认当前项是否正确。在运行时，如果没有报错，则说明功能正常。

运行如上测试代码，其运行结果如下：

```
====Pagetable test 1 started====
====Pagetable test 1 ended====
```

可以知道，本页表基本功能正常。

3.3.2 页表地址分配测试

本测试用例主要对于页表分配和释放中的地址边界进行测试，通过测试在不同地址和分配大小下虚拟页的分配，具体测试代码如下：

```
void test_pagetable_2(void){
    printf("====Pagetable test 2 started====\n\n");

    pgtbl_t test_pgtbl = pmem_alloc(true);
    uint64 mem[5];
    for(int i = 0; i < 5; i++){
        mem[i] = (uint64)pmem_alloc(false);
    }

    printf("\nPart1: allocating pages\n\n");
    vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_R);
    vm_mappages(test_pgtbl, PGSIZE * 10, mem[1], PGSIZE / 2, PTE_R | PTE_W);
    vm_mappages(test_pgtbl, PGSIZE * 512, mem[2], PGSIZE - 1, PTE_R | PTE_X);
    vm_mappages(test_pgtbl, PGSIZE * 512 * 512, mem[3], PGSIZE, PTE_R | PTE_X);
    vm_mappages(test_pgtbl, VA_MAX - PGSIZE, mem[4], PGSIZE, PTE_W);
    vm_print(test_pgtbl); printf("\n");

    printf("\nPart2: unmappages\n\n");
    vm_unmappages(test_pgtbl, 0, PGSIZE, true);
    // 由于下行代码重复分配了，所以需要先解绑再重绑
    vm_mappages(test_pgtbl, 0, mem[0], PGSIZE, PTE_W);
    vm_unmappages(test_pgtbl, PGSIZE * 10, PGSIZE, true);
    vm_unmappages(test_pgtbl, PGSIZE * 512, PGSIZE, true);
    vm_print(test_pgtbl);

    printf("====Pagetable test 2 ended====\n\n");
}
```

可以看到这个测试代码对于五个虚拟页进行了编辑和释放。且每次申请都有不同的地址和申请内存大小（均小于等于1个页）。从而测试分配和释放函数对于不同情况的处理能力。

运行测试代码，其结果如下：

```
====Pagetable test 2 started====

Part1: allocating pages

0: pte 0x00000000807b9000, pa 0x00000000807b8000
  0: pte 0x00000000807b8000, pa 0x00000000807b7000
    0: pte 0x00000000807b7000, pa 0x0000000087fff000
    10: pte 0x00000000807b7050, pa 0x0000000087ffe000
    1: pte 0x00000000807b8008, pa 0x00000000807b6000
      0: pte 0x00000000807b6000, pa 0x0000000087ffd000
1: pte 0x00000000807b9008, pa 0x00000000807b5000
  0: pte 0x00000000807b5000, pa 0x00000000807b4000
    0: pte 0x00000000807b4000, pa 0x0000000087ffc000
255: pte 0x00000000807b97f8, pa 0x00000000807b3000
   511: pte 0x00000000807b3ff8, pa 0x00000000807b2000
     511: pte 0x00000000807b2ff8, pa 0x0000000087ffb000

Part2: unmappages

0: pte 0x00000000807b9000, pa 0x00000000807b8000
  0: pte 0x00000000807b8000, pa 0x00000000807b7000
    0: pte 0x00000000807b7000, pa 0x0000000087fff000
    1: pte 0x00000000807b8008, pa 0x00000000807b6000
1: pte 0x00000000807b9008, pa 0x00000000807b5000
  0: pte 0x00000000807b5000, pa 0x00000000807b4000
    0: pte 0x00000000807b4000, pa 0x0000000087ffc000
255: pte 0x00000000807b97f8, pa 0x00000000807b3000
   511: pte 0x00000000807b3ff8, pa 0x00000000807b2000
     511: pte 0x00000000807b2ff8, pa 0x0000000087ffb000
====Pagetable test 2 ended====
```

可以看到，在第一个部分，五个页面被分配到(0,0,0), (0,0,10), (0,1,0), (1,0,0), (255,511,511)，说明其分配正常。而在释放(0,0,10)以及(0,1,0)这两个虚拟页后，页表中相关表项被删除，说明代码对于页的分配和释放正常。

3.3.3 页表批量分配测试

本测试代码对在不同地址和不同分配大小的情况下，对于页表的分配和释放进行较为批量的测试，代码如下：

```
void test_pagetable_3(void){
    printf("====Pagetable test 3 started====\n\n");

    // 创建页表
```

```
pgtbl_t test_pgtbl = pmem_alloc(true);

// 创建物理页
uint64 pa_list[64];
for(int i=0; i<64; i++){
    pa_list[i] = (uint64)pmem_alloc(false);
    if (pa_list[i] == 0) {
        printf("Failed to allocate physical memory.\n");
        return;
    }
}

// 开始映射
// 第i个项, 虚拟地址为i*64*PGSIZE, 物理地址为pa_list[i], 分配大小(i+1)*PGSIZE
for(int i=0; i<64; i++){
    vm_mappages(test_pgtbl, i*64*PGSIZE, pa_list[i], (i+1)*PGSIZE, PTE_R);
    printf("Id %d: Mapped VA %p to PA %p.\n", i, i*64*PGSIZE, pa_list[i]);
}

// vm_print(test_pgtbl); printf("\n");

// 验证映射是否正确
for(int i=0; i<64; i++){
    pte_t *pte = vm_getpte(test_pgtbl, i*64*PGSIZE, 0);
    if (pte == 0 || (*pte & PTE_V) == 0) {
        panic("Failed to find correct mapping");
    } else {
        uint64 mapped_pa = PTE_TO_PA(*pte);

        printf("VA 0x%p is mapped to PA 0x%p.\n", i*64*PGSIZE, mapped_pa);

        assert(mapped_pa == pa_list[i], "Incorrect mapping");
    }
}

// 清除后32个页面
for(int i=32; i<64; i++){
    vm_unmappages(test_pgtbl, i*64*PGSIZE, (i+1)*PGSIZE, true);
}

vm_print(test_pgtbl); printf("\n");

// 倒序清除前32个页面
for(int i=31; i>=0; i--){
    vm_unmappages(test_pgtbl, i*64*PGSIZE, (i+1)*PGSIZE, true);
}

vm_print(test_pgtbl); printf("\n");

printf("====Pagetable test 3 ended====\n\n");
}
```

该测试用例申请了64个申请地址和申请大小各不相同的页表绑定，并进行批量释放，从而对于页表进行批量测试。

本测试代码先分配64次，后逐渐比对各个页表项和物理表项是否正确分配，如果不匹配则报错。在此之后，将逐个删除后32个页表和前32个页表，并输出页表信息。

该测试代码的结果如下：

在进行绑定页表后，其页表的状态如下：


```
====Pagetable test 3 started====
```

```
Id 0: Mapped VA 0x0000000000000000 to PA 0x0000000087fff000.  
Id 1: Mapped VA 0x0000000000004000 to PA 0x0000000087ffe000.  
Id 2: Mapped VA 0x0000000000008000 to PA 0x0000000087ffd000.  
Id 3: Mapped VA 0x000000000000c000 to PA 0x0000000087ffc000.  
Id 4: Mapped VA 0x0000000000010000 to PA 0x0000000087ffb000.  
Id 5: Mapped VA 0x0000000000014000 to PA 0x0000000087ffa000.  
Id 6: Mapped VA 0x0000000000018000 to PA 0x0000000087ff9000.  
Id 7: Mapped VA 0x000000000001c000 to PA 0x0000000087ff8000.  
Id 8: Mapped VA 0x0000000000020000 to PA 0x0000000087ff7000.  
Id 9: Mapped VA 0x0000000000024000 to PA 0x0000000087ff6000.  
Id 10: Mapped VA 0x0000000000028000 to PA 0x0000000087ff5000.  
Id 11: Mapped VA 0x000000000002c000 to PA 0x0000000087ff4000.  
Id 12: Mapped VA 0x0000000000030000 to PA 0x0000000087ff3000.  
Id 13: Mapped VA 0x0000000000034000 to PA 0x0000000087ff2000.  
Id 14: Mapped VA 0x0000000000038000 to PA 0x0000000087ff1000.  
Id 15: Mapped VA 0x000000000003c000 to PA 0x0000000087ff0000.  
Id 16: Mapped VA 0x0000000000040000 to PA 0x0000000087fef000.  
Id 17: Mapped VA 0x0000000000044000 to PA 0x0000000087fee000.  
Id 18: Mapped VA 0x0000000000048000 to PA 0x0000000087fed000.  
Id 19: Mapped VA 0x000000000004c000 to PA 0x0000000087fec000.  
Id 20: Mapped VA 0x0000000000050000 to PA 0x0000000087feb000.  
Id 21: Mapped VA 0x0000000000054000 to PA 0x0000000087fea000.  
Id 22: Mapped VA 0x0000000000058000 to PA 0x0000000087fe9000.  
Id 23: Mapped VA 0x000000000005c000 to PA 0x0000000087fe8000.  
Id 24: Mapped VA 0x0000000000060000 to PA 0x0000000087fe7000.  
Id 25: Mapped VA 0x0000000000064000 to PA 0x0000000087fe6000.  
Id 26: Mapped VA 0x0000000000068000 to PA 0x0000000087fe5000.  
Id 27: Mapped VA 0x000000000006c000 to PA 0x0000000087fe4000.  
Id 28: Mapped VA 0x0000000000070000 to PA 0x0000000087fe3000.  
Id 29: Mapped VA 0x0000000000074000 to PA 0x0000000087fe2000.  
Id 30: Mapped VA 0x0000000000078000 to PA 0x0000000087fe1000.  
Id 31: Mapped VA 0x000000000007c000 to PA 0x0000000087fe0000.  
Id 32: Mapped VA 0x0000000000080000 to PA 0x0000000087fdf000.  
Id 33: Mapped VA 0x0000000000084000 to PA 0x0000000087fde000.  
Id 34: Mapped VA 0x0000000000088000 to PA 0x0000000087fdd000.  
Id 35: Mapped VA 0x000000000008c000 to PA 0x0000000087fdc000.  
Id 36: Mapped VA 0x0000000000090000 to PA 0x0000000087fdb000.  
Id 37: Mapped VA 0x0000000000094000 to PA 0x0000000087fda000.  
Id 38: Mapped VA 0x0000000000098000 to PA 0x0000000087fd9000.  
Id 39: Mapped VA 0x000000000009c000 to PA 0x0000000087fd8000.  
Id 40: Mapped VA 0x00000000000a0000 to PA 0x0000000087fd7000.  
Id 41: Mapped VA 0x00000000000a4000 to PA 0x0000000087fd6000.
```

```
VA 0x0x0000000000000000 is mapped to PA 0x0x0000000087fff000.
VA 0x0x0000000000004000 is mapped to PA 0x0x0000000087ffe000.
VA 0x0x0000000000008000 is mapped to PA 0x0x0000000087ffd000.
VA 0x0x000000000000c000 is mapped to PA 0x0x0000000087ffc000.
VA 0x0x0000000000010000 is mapped to PA 0x0x0000000087ffb000.
VA 0x0x0000000000014000 is mapped to PA 0x0x0000000087ffa000.
VA 0x0x0000000000018000 is mapped to PA 0x0x0000000087ff9000.
VA 0x0x000000000001c000 is mapped to PA 0x0x0000000087ff8000.
VA 0x0x0000000000020000 is mapped to PA 0x0x0000000087ff7000.
VA 0x0x0000000000024000 is mapped to PA 0x0x0000000087ff6000.
VA 0x0x0000000000028000 is mapped to PA 0x0x0000000087ff5000.
VA 0x0x000000000002c000 is mapped to PA 0x0x0000000087ff4000.
VA 0x0x0000000000030000 is mapped to PA 0x0x0000000087ff3000.
VA 0x0x0000000000034000 is mapped to PA 0x0x0000000087ff2000.
VA 0x0x0000000000038000 is mapped to PA 0x0x0000000087ff1000.
VA 0x0x000000000003c000 is mapped to PA 0x0x0000000087ff0000.
VA 0x0x0000000000040000 is mapped to PA 0x0x0000000087fef000.
VA 0x0x0000000000044000 is mapped to PA 0x0x0000000087fee000.
VA 0x0x0000000000048000 is mapped to PA 0x0x0000000087fed000.
VA 0x0x000000000004c000 is mapped to PA 0x0x0000000087fec000.
VA 0x0x0000000000050000 is mapped to PA 0x0x0000000087feb000.
VA 0x0x0000000000054000 is mapped to PA 0x0x0000000087fea000.
VA 0x0x0000000000058000 is mapped to PA 0x0x0000000087fe9000.
VA 0x0x000000000005c000 is mapped to PA 0x0x0000000087fe8000.
VA 0x0x0000000000060000 is mapped to PA 0x0x0000000087fe7000.
VA 0x0x0000000000064000 is mapped to PA 0x0x0000000087fe6000.
VA 0x0x0000000000068000 is mapped to PA 0x0x0000000087fe5000.
VA 0x0x000000000006c000 is mapped to PA 0x0x0000000087fe4000.
VA 0x0x0000000000070000 is mapped to PA 0x0x0000000087fe3000.
VA 0x0x0000000000074000 is mapped to PA 0x0x0000000087fe2000.
VA 0x0x0000000000078000 is mapped to PA 0x0x0000000087fe1000.
VA 0x0x000000000007c000 is mapped to PA 0x0x0000000087fe0000.
VA 0x0x0000000000080000 is mapped to PA 0x0x0000000087fdf000.
VA 0x0x0000000000084000 is mapped to PA 0x0x0000000087fde000.
VA 0x0x0000000000088000 is mapped to PA 0x0x0000000087fdd000.
VA 0x0x000000000008c000 is mapped to PA 0x0x0000000087fdc000.
VA 0x0x0000000000090000 is mapped to PA 0x0x0000000087fdb000.
VA 0x0x0000000000094000 is mapped to PA 0x0x0000000087fda000.
VA 0x0x0000000000098000 is mapped to PA 0x0x0000000087fd9000.
VA 0x0x000000000009c000 is mapped to PA 0x0x0000000087fd8000.
VA 0x0x00000000000a0000 is mapped to PA 0x0x0000000087fd7000.
VA 0x0x00000000000a4000 is mapped to PA 0x0x0000000087fd6000.
VA 0x0x00000000000a8000 is mapped to PA 0x0x0000000087fd5000.
VA 0x0x00000000000ac000 is mapped to PA 0x0x0000000087fd4000.
```

在第一阶段解绑页表之后，页表的情况如下：

```
0: pte 0x0000000087bd000 pa 0x0000000087bc000
```



```
0: pte 0x00000000807bb000, pa 0x00000000807bc000
0: pte 0x00000000807bc000, pa 0x00000000807bb000
0: pte 0x00000000807bb000, pa 0x00000000807fff000
64: pte 0x00000000807bb200, pa 0x00000000807ffe000
65: pte 0x00000000807bb208, pa 0x00000000807fff000
128: pte 0x00000000807bb400, pa 0x00000000807ffd000
129: pte 0x00000000807bb408, pa 0x00000000807ffe000
130: pte 0x00000000807bb410, pa 0x00000000807fff000
192: pte 0x00000000807bb600, pa 0x00000000807ffc000
193: pte 0x00000000807bb608, pa 0x00000000807ffd000
194: pte 0x00000000807bb610, pa 0x00000000807ffe000
195: pte 0x00000000807bb618, pa 0x00000000807fff000
256: pte 0x00000000807bb800, pa 0x00000000807ffb000
257: pte 0x00000000807bb808, pa 0x00000000807ffc000
258: pte 0x00000000807bb810, pa 0x00000000807ffd000
259: pte 0x00000000807bb818, pa 0x00000000807ffe000
260: pte 0x00000000807bb820, pa 0x00000000807fff000
320: pte 0x00000000807bba00, pa 0x00000000807ffa000
321: pte 0x00000000807bba08, pa 0x00000000807ffb000
322: pte 0x00000000807bba10, pa 0x00000000807ffc000
323: pte 0x00000000807bba18, pa 0x00000000807ffd000
324: pte 0x00000000807bba20, pa 0x00000000807ffe000
325: pte 0x00000000807bba28, pa 0x00000000807fff000
384: pte 0x00000000807bbc00, pa 0x00000000807ff9000
385: pte 0x00000000807bbc08, pa 0x00000000807ffa000
386: pte 0x00000000807bbc10, pa 0x00000000807ffb000
387: pte 0x00000000807bbc18, pa 0x00000000807ffc000
388: pte 0x00000000807bbc20, pa 0x00000000807ffd000
389: pte 0x00000000807bbc28, pa 0x00000000807ffe000
390: pte 0x00000000807bbc30, pa 0x00000000807fff000
448: pte 0x00000000807bbe00, pa 0x00000000807ff8000
449: pte 0x00000000807bbe08, pa 0x00000000807ff9000
450: pte 0x00000000807bbe10, pa 0x00000000807ffa000
451: pte 0x00000000807bbe18, pa 0x00000000807ffb000
452: pte 0x00000000807bbe20, pa 0x00000000807ffc000
453: pte 0x00000000807bbe28, pa 0x00000000807ffd000
454: pte 0x00000000807bbe30, pa 0x00000000807ffe000
455: pte 0x00000000807bbe38, pa 0x00000000807fff000
1: pte 0x00000000807bc008, pa 0x00000000807ba000
0: pte 0x00000000807ba000, pa 0x00000000807ff7000
1: pte 0x00000000807ba008, pa 0x00000000807ff8000
2: pte 0x00000000807ba010, pa 0x00000000807ff9000
3: pte 0x00000000807ba018, pa 0x00000000807ffa000
```

```
4: pte 0x00000000807ba020, pa 0x0000000087ffb000
```

在第二阶段解绑页表之后，页表的情况如下：

```
0: pte 0x00000000807bd000, pa 0x00000000807bc000
  0: pte 0x00000000807bc000, pa 0x00000000807bb000
  1: pte 0x00000000807bc008, pa 0x00000000807ba000
  2: pte 0x00000000807bc010, pa 0x00000000807b9000
  3: pte 0x00000000807bc018, pa 0x00000000807b8000
  4: pte 0x00000000807bc020, pa 0x00000000807b7000
  5: pte 0x00000000807bc028, pa 0x00000000807b6000
  6: pte 0x00000000807bc030, pa 0x00000000807b5000
  7: pte 0x00000000807bc038, pa 0x00000000807b4000
```

实验中无报错，且各阶段输出正常，说明测试用例成功。

4. 结论与心得

在本实验之后，我收获颇多，具体如下：

对于操作系统中的物理内存管理，我了解到了操作系统的分页管理以及空闲链表的管理方式，我明白并实现利用空闲页的特点进行巧妙的初始化，同时我也掌握了空闲页表链表的组织，了解了操作系统如何实现对于物理页进行分配和释放。

对于操作系统中的虚拟内存管理，我深入了解了RISC-V的Sv39页表规范，深入了解操作系统的三级页表机制，并掌握页表初始化、分配、释放等基本流程，同时我也了解到了启用页表管理所需要的基本条件。

同时，对于实验过程中，我还通过调试代码测试代码中的错误，学会通过编写测试用例测试代码状态是否正常等。这些都加强了我自己对于操作系统的认识，并加强了我自己的动手能力。

综上所述，本次操作系统实践课是具有意义的课程。

实验 3：中断处理与时钟管理

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

0. 实验任务和目标

本实验主要通过分析xv6的中断处理机制，理解操作系统如何响应硬件事件，并实现如下目标：

1. 实现完整的操作系统中断处理框架，使得操作系统能够正确响应外部事件
2. 实现xv6的时钟中断响应，使得xv6能够在引发时钟中断时输出信息（如当前计时器的ticks值）
3. 实现xv6的外部中断响应，并通过CLINT，PLIC，uart使得xv6能够进行输入输出。

1. 系统设计

1.1 架构设计说明

本系统的内核开发基于RISC-V指令集架构，因此在中断与异常处理机制上完全遵循RISC-V的设计规范。与xv6教学操作系统类似，本系统内核直接利用RISC-V硬件提供的中断架构进行陷阱处理。

RISC-V中断架构的核心设计可概括如下：

- **统一的陷阱处理机制**：RISC-V架构将中断（Interrupt）和异常（Exception）统称为陷阱。中断来源于处理器外部，由硬件信号异步触发（如定时器中断、设备I/O中断）；异常则源于当前执行的指令，是同步触发的（如非法指令、系统调用ECALL）。本系统采用与硬件协同的设计：所有陷阱最初共享同一处理通道，由硬件完成初步的现场保存与跳转，随后软件陷阱处理程序再根据陷阱原因进行分发处理。这种统一机制简化了硬件设计，并保证了处理流程的一致性。
- **基于控制状态寄存器的硬件支持**：RISC-V提供了一组关键的控制状态寄存器来精确控制陷阱处理流程。本系统内核通过配置这些寄存器来管理中断。其核心作用包括：
 - **陷阱配置**：stvec（监管者陷阱向量基址寄存器）用于设置陷阱处理程序的入口地址。
 - **陷阱原因识别**：发生陷阱时，硬件自动将原因代码写入scause（监管者陷阱原因寄存器），并将出错的地址或指令等附加信息存入stval（监管者陷阱值寄存器）。
 - **现场保护与恢复**：sepc（监管者异常程序计数器）会自动保存发生陷阱的指令地址，以便处理结束后能正确返回。sstatus（监管者状态寄存器）中的中断使能位SIE会被硬件自动关闭，并在执行SRET指令返回时恢复，从而保护陷阱处理过程不被意外打断。
- **高效的中断委托机制**：RISC-V支持从高特权的机器模式将特定中断和异常委托给低特权的监管者模式处理。本系统充分利用了这一机制。在启动阶段，机器模式固件（如OpenSBI）已将时钟中断、外部设备中断以及来自用户模式的系统调用委托给监管者模式。这意味着，本系统内核（运行在S模式）能够直接处理绝大多数陷阱，无需频繁陷入M模式，极大地提升了中断响应和处理效率。

综上所述，本系统的中断处理建立在RISC-V清晰、模块化的硬件基础之上，通过硬件自动化的初步处理与软件灵活的分发处理相结合，实现了高效、可靠的陷阱管理机制。至于RISC-V操作系统中的详情策略（寄存器的功能以及详细的中断处理流程），请看2.1.1节 ~ 2.1.2节

1.2 关键数据结构

本系统通过配置RISC-V硬件中的寄存器以及物理内存（如中断向量）来管理中断，所以对于中断调用，没有新增新的管理用数据结构。除此以外，关键数据结构如下：

1.2.1 计时器timer

与xv6相同，本系统开辟了一份物理空间作为计时器的物理空间。这种计时器类似于80系列的计时器硬件，其原理如下：

1. 物理时钟将会不断进行电平信号的改变（即时钟脉冲`clk`），带动硬件内的寄存器`MTIME`不断自增
2. 除此之外，还有一个比较寄存器`MTIMECMP`与`MTIME`进行比较。一旦两者相同，就会触发一次时钟中断，程序流跳转到`mtvec`寄存器中存储的地址。
3. 可以确定一个滴答数 `INTERVAL`，先初始化 `MTIMECMP = MTIME + INTERVAL`，这样过 `INTERVAL` 个滴答后会触发第一次时钟中断此时，系统将当前的ticks数增加1，之后重新计时。按照这样循环往复，`ticks`数将会不断自增。

在xv6中，时钟中断的代码分散在`start.c`等文件中。这种设计比较便于在`start.c`中直接使用`timerinit`函数进行初始化，然而，这种分散的结构不易于代码的理解，且不利于修改。为了解决此问题，本系统将时钟中断以及计时器模块相关代码存放在`timer.c`中。时钟模块的定义文件`timer.h`如下所示：

```
// 计时器
typedef struct timer {
    uint64 ticks;
    spinlock_t lk;
} timer_t;

// 每隔INTERVAL个单位时间发生一次时钟中断(1e6大约为0.1s)
#define INTERVAL 1000000

void timer_init();           // 时钟初始化(in M-mode)

void timer_create();         // 时钟创建
void timer_update();         // 时钟更新(ticks++)
uint64 timer_get_ticks();    // 获取时钟的tick
```

通过这样的分离式调用，`timer`相关的模块得到了妥善的处理，这样的模块易于理解，且更容易进行修改。对于详细的模块功能实现，请看源代码仓库。

1.3 设计决策理由

RISC-V硬件本身即为陷阱处理提供了标准化的流水线（保存上下文、设置原因寄存器、跳转）。和xv6系统一样，本系统的软件设计选择与硬件紧密协同，遵循“硬件做什么，软件补全什么”的原则，这样，在硬件上，本系统能够充分利用寄存器，实现功能，在软件上，代码的逻辑更佳清晰，且易于理解和调试。

1.4 与 xv6 对比分析

本系统与xv6的对比分析如下：

本系统和xv6采用相同的中断处理机制，均使用RISC-V中的硬件寄存器进行处理从而进行中断的处理。

而相比xv6，本系统有部分简化和处理的部分：中断处理上，由于本系统尚未实现有关于进程分配的功能，所以在具体实现上相比于xv6更加简单。而在代码结构上，本系统采用了诸多操作降低代码的复杂度，提升代码的可读性。例如将timer模块单独提取，合并kerneltrap, devintr函数等。通过如上操作，本系统在不影响中断功能的前提下实现了系统的简化。

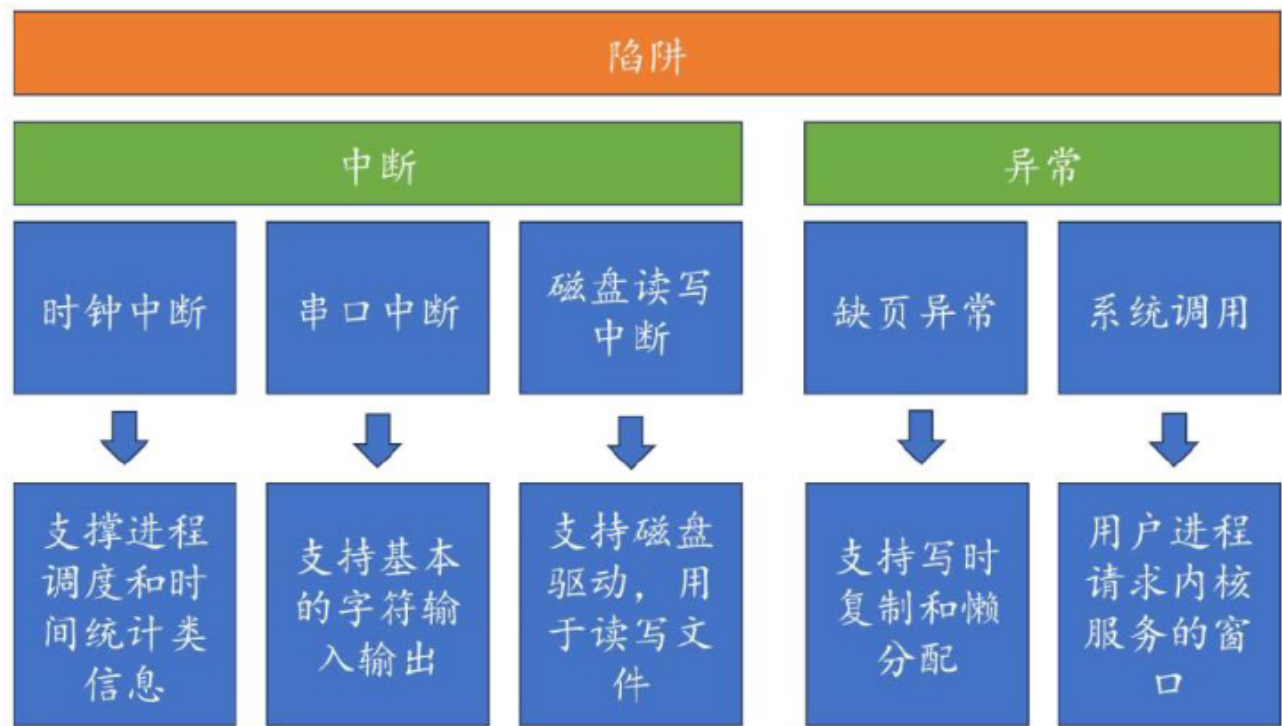
2. 实验过程部分

2.1 实现步骤记录

2.1.1 理解RISC-V中断架构

首先，由于本系统需要使用RISC-V相关的中断架构，所以在书写代码之前，需要理解RISC-V的中断机理。

RISC-V将将中断（Interrupt）和异常（Exception）统称为陷阱（Trap），并使用同一套机制来处理他们。中断来源于处理器外部，由硬件信号异步触发（如定时器中断、设备I/O中断）；异常则源于当前执行的指令，是同步触发的（如非法指令、系统调用ECALL）。示意图如下：



RISC-V相关的中断机理如下：

特权级别与中断入口

RISC-V定义了多个特权级别，最常见的三个是：

- M模式（机器模式）：最高权限，所有RISC-V处理器都必须实现。它是上电后最初进入的模式，负责最底层的硬件管理。
- S模式（监管者模式）：操作系统内核通常运行在此模式。它支持现代多任务操作系统的需求，如虚拟内存。
- U模式（用户模式）：应用程序运行的模式，权限最低。

一般而言，中断和异常可以在任何特权级别发生，并且处理器会跳转到更高特权级别进行处理。

中断特权委托机制

中断委托是RISC-V架构提供的一种硬件机制，允许高特权级别（通常是M模式，机器模式）将特定中断和异常的处理权限“下放”或“委托”给低特权级别（通常是S模式，监管者模式）来处理。

在技术上，RISC-V提供了两个关键的委托寄存器：

- **medeleg**：异常委托寄存器。可以将特定的异常（如ECALL、页面故障）直接委托给S模式处理，而无需经过M模式。
- **mideleg**：中断委托寄存器。可以将特定的中断（如软件中断、定时器中断、外部中断）直接委托给S模式处理。

如果没有中断委托，那么一个发生在S模式或U模式下的中断（比如磁盘读取完成），会先导致CPU切换到最高特权的M模式，再由M模式软件交给S模式的内核，处理完之后在经过两次上下文切换再返回原来的模式，这样恢复开销巨大，效率低下。而通过操作系统内核的委托机制，中断的处理能直接送达S模式进行处理，从而提升性能。同时，这样使得各个模式的分工明确，这使得软件结构更清晰，更易于维护。

所有操作系统内核正常运行所必需的中断，都应该被委托给S模式。例如软件中断，计时器中断，外部中断等。而来自M模式本身的异常等系统的严重错误则在M模式直接处理即可。

其他关键控制状态寄存器（CSR）

RISC-V的中断架构的核心是一组CSR，操作系统通过读写它们来配置和管理中断。除了上文所介绍的委托寄存器，这些寄存器及其功能如下：

1. 中断使能与控制

- **mstatus/sstatus**：全局状态寄存器。其中的MIE/SIE位是全局中断使能位。当该位为0时，对应模式下的所有中断都会被屏蔽。
- **mip/sip**：中断待定寄存器。其中的位表示当前是否有中断正在等待处理（例如，MTIP位表示机器级定时器中断待定）。
- **mie/sie**：中断使能寄存器。你可以通过设置该寄存器的位来选择使能或禁用特定类型的中断（例如，MTIE位用于控制是否允许机器级定时器中断）。

2. 陷阱入口基地址

- **mtvec/stvec**：陷阱向量基地址寄存器。当发生中断或异常时，处理器会跳转到这个寄存器指定的地址开始执行陷阱处理程序。包含如下模式：
 - 直接模式：所有陷阱都跳转到同一个地址。
 - 向量模式：不同的中断/异常原因会跳转到不同的偏移地址，这可以避免一开始就用软件判断原因，从而提高效率。

3. 陷阱处理过程中的CSR

当陷阱发生时，硬件将会自动更新以下CSR，从而记录相关信息：

- `mcause/scause`：记录陷阱发生的原因。最高位是中断位（1表示中断，0表示异常），其余位表示具体的编号（例如，5是加载地址错误异常，7是存储/AMO地址错误异常，9是S模式下的ECALL）
- `mepc/sepc`：保存发生陷阱时即将执行的那条指令的地址。在处理完陷阱后，通过执行SRET指令，CPU会从这里恢复执行。
- `mtval/stval`：提供关于陷阱的附加信息，例如发生页面故障的地址、执行非法指令时的指令编码等。

陷阱处理流程过程概述

对于一个已经产生的中断，首先如果需要，系统会将该中断委托给S模式下的系统，在此之后，系统将进入S模式，硬件和软件的行为如下：

1. 保存现场：将当前pc保存到sepc、将陷阱原因（中断类型或异常代码）保存到scause、将附加信息（如错误地址）保存到stval。
2. 关中断：在此之后，系统将sstatus中的SIE位清零，屏蔽后续的中断，防止处理程序被意外打断。
3. 跳转：跳转至stvec寄存器中的地址，开始执行陷阱处理程序。
4. 保存上下文：在汇编代码中，程序将会将寄存器的值保存在栈中，从而保存完整的上下文。之后程序将会调用中断处理函数。
5. 中断处理程序：在软件中，系统读取寄存器的值，并分析陷阱的类型（中断/异常），调用相关的处理程序进行中断处理
6. 恢复上下文：从内核栈上恢复之前保存的所有寄存器。
7. 返回：执行SRET指令。该指令会将pc恢复为sepc的值，并重新打开中断（恢复sstatus中的SIE位）。

对于思考题，答案如下：

时钟中断为什么在M模式产生，却在S模式处理？

操作系统需要依靠定时器中断来实现时间片轮转调度，维护系统时间，实现高精度定时器等。而这些功能需要时钟中断能够及时响应。如果不委托，则每次时钟滴答需要进入M模式在转换，从而导致效率降低，这些功能会收到影响。所以时钟中断得委托给S模式处理。

如何理解“中断是异步的，异常是同步的”？

中断是外界设备所产生的时间，其发生不随当前时钟而同步，所以是异步的。而异常通常指系统内部产生的错误和事件（例如非法指令等），其运行在内部的CPU中，其与时钟而同步。所以中断是异步的，异常是同步的。

2.1.2 分析xv6的中断处理流程

接下来，阅读xv6的官方代码，了解各个部分的设置内容。

首先，阅读`start.c`，了解机器模式的设置：

```
void
start()
{
    // ...
    // delegate all interrupts and exceptions to supervisor mode.
```

```

w_medeleg(0xffff);
w_mideleg(0xffff);
w_sie(r_sie() | SIE_SEIE | SIE_STIE | SIE_SSIE);

// ...
// ask for clock interrupts.
timerinit();

// ...
}

```

该`start`函数代码将所有中断（包含时钟中断）委托给S模式，并且将中断使能位置1，从而启动中断。与此同时，该函数还对于时钟中断初始化，初始化代码如下：

```

void
timerinit()
{
    // each CPU has a separate source of timer interrupts.
    int id = r_mhartid();

    // ask the CLINT for a timer interrupt.
    int interval = 1000000; // cycles; about 1/10th second in qemu.
    *(uint64*)CLINT_MTIMECMP(id) = *(uint64*)CLINT_MTIME + interval;

    // prepare information in scratch[] for timervec.
    // scratch[0..2] : space for timervec to save registers.
    // scratch[3] : address of CLINT MTIMECMP register.
    // scratch[4] : desired interval (in cycles) between timer interrupts.
    uint64 *scratch = &timer_scratch[id][0];
    scratch[3] = CLINT_MTIMECMP(id);
    scratch[4] = interval;
    w_mscratch((uint64)scratch);

    // set the machine-mode trap handler.
    w_mtvec((uint64)timervec);

    // enable machine-mode interrupts.
    w_mstatus(r_mstatus() | MSTATUS_MIE);

    // enable machine-mode timer interrupts.
    w_mie(r_mie() | MIE_MTIE);
}

```

该初始化代码将计时器的值初始化，并且保存相关寄存器的信息以供中断使用，初始化中断向量，修改状态寄存器的值从而启动时钟中断。

可以知道，由于时钟是在M模式（硬件）下产生的，同时这也和进程管理等功能相关，所以其处理和其他M模式的中断不同，不仅需要委托给S模式处理，并且还需要初始化，记录相关信息地址。

在这里, `timervec`为中断向量, 用于表示时钟中断启动时程序的入口。当中断启动时, 程序将会跳转到对应的处理函数。

其次, 阅读`kernelvec.S`, 了解上下文切换过程。代码如下:

```
.globl kerneltrap
.globl kernelvec
.align 4
kernelvec:
    # make room to save registers.
    addi sp, sp, -256

    # save the registers.
    sd ra, 0(sp)
    sd sp, 8(sp)
    sd gp, 16(sp)
    sd tp, 24(sp)
    sd t0, 32(sp)
    sd t1, 40(sp)
    sd t2, 48(sp)
    sd s0, 56(sp)
    sd s1, 64(sp)
    sd a0, 72(sp)
    sd a1, 80(sp)
    sd a2, 88(sp)
    sd a3, 96(sp)
    sd a4, 104(sp)
    .....

    # call the C trap handler in trap.c
    call kerneltrap

    # restore registers.
    ld ra, 0(sp)
    ld sp, 8(sp)
    ld gp, 16(sp)
    # not tp (contains hartid), in case we moved CPUs
    ld t0, 32(sp)
    ld t1, 40(sp)
    ld t2, 48(sp)
    ld s0, 56(sp)
    ld s1, 64(sp)
    ld a0, 72(sp)
    ld a1, 80(sp)
    ld a2, 88(sp)
    ld a3, 96(sp)
    ld a4, 104(sp)
    .....

    addi sp, sp, 256

    # return to whatever we were doing in the kernel.
```

```

        sret

        #
        # machine-mode timer interrupt.
        #
.globl timervec
.align 4
timervec:
    # start.c has set up the memory that mscratch points to:
    # scratch[0,8,16] : register save area.
    # scratch[24] : address of CLINT's MTIMECMP register.
    # scratch[32] : desired interval between interrupts.

    csrrw a0, mscratch, a0
    sd a1, 0(a0)
    sd a2, 8(a0)
    sd a3, 16(a0)

    # schedule the next timer interrupt
    # by adding interval to mtimecmp.
    ld a1, 24(a0) # CLINT_MTIMECMP(hart)
    ld a2, 32(a0) # interval
    ld a3, 0(a1)
    add a3, a3, a2
    sd a3, 0(a1)

    # arrange for a supervisor software interrupt
    # after this handler returns.
    li a1, 2
    csrw sip, a1

    ld a3, 16(a0)
    ld a2, 8(a0)
    ld a1, 0(a0)
    csrrw a0, mscratch, a0

    mret

```

可以知道，对于一般的内核态中断（kernelvec），其保存了所有必须保存的通用寄存器（如ra, sp）等；而对于时钟中断，由于其产生在M模式下，所以其在M模式下处理之后，将a1=2的信息传递给S模式，综上只需要保存a1~a3寄存器即可。

为了提升效率，保障程序空间，无需将所有的寄存器进行保存，只需要保存通用寄存器保存相关上下文即可，其他的状态寄存器正常工作。

对于栈的使用，其和函数调用中栈的使用类似，先开辟空间，之后有序将各个需要保存的寄存器保存，然后进行中断处理（主要调用处理函数处理），在处理结束后恢复寄存器并缩小栈空间即可。

最后，阅读trap.c，了解中断的处理机制：

```

// interrupts and exceptions from kernel code go here via kernelvec,
// on whatever the current kernel stack is.
void
kerneltrap()
{
    int which_dev = 0;
    uint64 sepc = r_sepc();
    uint64 sstatus = r_sstatus();
    uint64 scause = r_scause();

    if((sstatus & SSTATUS_SPP) == 0)
        panic("kerneltrap: not from supervisor mode");
    if(intr_get() != 0)
        panic("kerneltrap: interrupts enabled");

    if((which_dev = devintr()) == 0){
        printf("scause %p\n", scause);
        printf("sepc=%p stval=%p\n", r_sepc(), r_stval());
        panic("kerneltrap");
    }

    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2 && myproc() != 0 && myproc()->state == RUNNING)
        yield();

    // the yield() may have caused some traps to occur,
    // so restore trap registers for use by kernelvec.S's sepc instruction.
    w_sepc(sepc);
    w_sstatus(sstatus);
}

int
devintr()
{
    uint64 scause = r_scause();

    if((scause & 0x8000000000000000L) &&
        (scause & 0xff) == 9){
        // this is a supervisor external interrupt, via PLIC.

        // irq indicates which device interrupted.
        int irq = plic_claim();

        if(irq == UART0_IRQ){
            uartintr();
        } else if(irq == VIRTIO0_IRQ){
            virtio_disk_intr();
        } else if(irq){
            printf("unexpected interrupt irq=%d\n", irq);
        }

        // the PLIC allows each device to raise at most one
        // interrupt at a time; tell the PLIC the device is

```

```

    // now allowed to interrupt again.
    if(irq)
        plic_complete(irq);

    return 1;
} else if(scause == 0x8000000000000001L){
    // software interrupt from a machine-mode timer interrupt,
    // forwarded by timervec in kernelvec.S.

    if(cpuid() == 0){
        clockintr();
    }

    // acknowledge the software interrupt by clearing
    // the SSIP bit in sip.
    w_sip(r_sip() & ~2);

    return 2;
} else {
    return 0;
}
}

```

可以知道，`kerneltrap`程序对于到来的中断进行处理。在确认当前中断已经委托或已经在S模式且中断已经关闭的情况下，调用`devintr`函数对于中断进行进一步判断。`devintr`函数通过识别相关的状态寄存器从而判断当前状态并予以处理。当为时钟中断返回2，当为其他设备相关的中断返回1，未知trap返回0。而在最后`kerneltrap`还进行了进程管理处理以及状态寄存器恢复等。

对于中断处理中的重入问题，程序将会在中断处理程序运行期间关闭中断，直至中断结束后恢复，从而防止重复进入。

中断处理时间不应过长，如果过长，那么其他的中断将会不能得到及时的解决，从而影响处理效率（例如输入相关的中断会因为其他中断处理时间过长从而不能响应输入等）。

2.1.3 设计中断处理框架

接下来，设计中断处理框架。本系统所使用的中断处理框架如下：

1. 设计中断向量表结构：本系统的中断向量表由riscv执行，当需要写入时，使用`w_stvec`命令即可。
2. 定义中断处理函数接口：当中断发生时，系统将会跳转到`trap.S`的相关处理函数中进行中断处理。
3. 实现中断的注册和注销机制：每个子模块负责对应中断的启动，其注册在对应初始化函数中执行。

设计中断处理程序中的相关代码定义如下：

```

#include "common.h"

// trap的初始化和处理

void trap_kernel_init();
void trap_kernel_inithart();
void trap_kernel_handler();

```

```
void trap_user_handler();
void trap_user_return();

// 辅助函数：外设中断和时钟中断处理
void external_interrupt_handler();
void timer_interrupt_handler();
```

2.1.4 实现上下文保存与恢复

仿照xv6，实现中断入口的实现。相关的汇编代码类似于xv6，如下所示：

```
# 外部函数声明 in trap_kernel.c
.globl trap_kernel_handler

# S-mode 中断处理（包括软件中断和外设中断）
.globl kernel_vector
.align 4
kernel_vector:

    # 准备空间给32个通用寄存器
    addi sp, sp, -256

    # 寄存器状态保存
    sd ra, 0(sp)
    sd sp, 8(sp)
    sd gp, 16(sp)
    sd tp, 24(sp)
    sd t0, 32(sp)
    sd t1, 40(sp)
    sd t2, 48(sp)
    sd s0, 56(sp)
    sd s1, 64(sp)
    sd a0, 72(sp)
    sd a1, 80(sp)
    sd a2, 88(sp)
    sd a3, 96(sp)
    sd a4, 104(sp)
    sd a5, 112(sp)
    sd a6, 120(sp)
    sd a7, 128(sp)
    sd s2, 136(sp)
    sd s3, 144(sp)
    sd s4, 152(sp)
    sd s5, 160(sp)
    sd s6, 168(sp)
    sd s7, 176(sp)
    sd s8, 184(sp)
    sd s9, 192(sp)
    sd s10, 200(sp)
```

```
sd s11, 208(sp)
sd t3, 216(sp)
sd t4, 224(sp)
sd t5, 232(sp)
sd t6, 240(sp)

# trap的处理过程
call trap_kernel_handler

# 寄存器状态恢复
ld ra, 0(sp)
ld sp, 8(sp)
ld gp, 16(sp)
# tp寄存器的值可能不可靠
# ld tp, 24(sp)
ld t0, 32(sp)
ld t1, 40(sp)
ld t2, 48(sp)
ld s0, 56(sp)
ld s1, 64(sp)
ld a0, 72(sp)
ld a1, 80(sp)
ld a2, 88(sp)
ld a3, 96(sp)
ld a4, 104(sp)
ld a5, 112(sp)
ld a6, 120(sp)
ld a7, 128(sp)
ld s2, 136(sp)
ld s3, 144(sp)
ld s4, 152(sp)
ld s5, 160(sp)
ld s6, 168(sp)
ld s7, 176(sp)
ld s8, 184(sp)
ld s9, 192(sp)
ld s10, 200(sp)
ld s11, 208(sp)
ld t3, 216(sp)
ld t4, 224(sp)
ld t5, 232(sp)
ld t6, 240(sp)

# 栈指针恢复
addi sp, sp, 256

# 当前处于S-mode,返回调用者
sret
```

```
# M-mode 中断处理(只包括时钟中断)
.globl timer_vector
.align 4
timer_vector:
```

```

# 暂存寄存器 a0 a1 a2 a3
# 将MSR寄存器 mscratch 放入 a0
csrrw a0, mscratch, a0
sd a1, 0(a0)      # mscratch[0] = a1
sd a2, 8(a0)      # mscratch[1] = a2
sd a3, 16(a0)     # mscratch[2] = a3

# CLINT_MTIMECMP(hartid) = CLINT_MTIMECMP(hartid) + INTERVAL
# 以便响应下一次时钟中断
ld a1, 24(a0)     # a1 = mscratch[3] 里面放了 CLINT_MTIMECMP(hartid)
ld a2, 32(a0)     # a2 = mscratch[4] 里面放了 INTERVAL
ld a3, 0(a1)
add a3, a3, a2
sd a3, 0(a1)

# 引发一个 S-mode software interrupt
li a1, 2
csrw sip, a1

# 恢复寄存器 a0 a1 a2 a3
# 将 mscratch 寄存器恢复
ld a3, 16(a0)
ld a2, 8(a0)
ld a1, 0(a0)
csrrw a0, mscratch, a0

mret

```

2.1.5 实现中断的处理（含时钟中断）

在保存上下文之后，程序需要调用相关的中断处理函数，与xv6不同，书写函数如下：

```

// 在kernel_vector()里面调用
// 内核态trap处理的核心逻辑
void trap_kernel_handler()
{
    uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
    uint64 sstatus = r_sstatus();     // 与特权模式和中断相关的状态信息
    uint64 scause = r_scause();       // 引发trap的原因
    uint64 stval = r_stval();         // 发生trap时保存的附加信息(不同trap不一样)

    // 确认trap来自S-mode且此时trap处于关闭状态
    assert(sstatus & SSTATUS_SPP, "trap_kernel_handler: not from s-mode");
    assert(intr_get() == 0, "trap_kernel_handler: interreput enabled");

    int trap_id = scause & 0xf;

    // 中断异常处理核心逻辑
    if(scause & ((uint64)1 << 63)){

```



```

// 外部中断 (含时钟等)
// 可用于调试的输出测试信息
switch(trap_id){
    case 1:
        timer_interrupt_handler();
        break;
    case 9:
        external_interrupt_handler();
        break;
    default:
        printf("activated interrupt: %s\n", interrupt_info[trap_id]);
        printf("scause %p\n", scause);
        printf("sepc=%p stval=%p\n", sepc, stval);
        panic("kerneltrap: not recognized");
        break;
}
}
else{
    // 异常
    // 可用于调试的输出测试信息
    printf("activated exception: %s\n", exception_info[trap_id]);
    printf("scause %p\n", scause);
    printf("sepc=%p stval=%p\n", sepc, stval);
    panic("kerneltrap: Exception\n");
}
}

```

本函数获取相关寄存器信息，并且根据trap的类型从而进入不同的处理程序。对于中断，其根据中断号从而判断中断。对于已知的中断，具体实现如下：

对于时钟中断，其交给timer_interrupt_handler函数解决，该函数调用timer_update，使得当前的ticks值增加。完成之后承认软件中断，消除中断位

```

// 时钟中断处理 (基于CLINT)
void timer_interrupt_handler()
{
    if(mycpuid() == 0){
        timer_update();
    }
    // 通过清除SSIP位，承认软件中断
    w_sip(r_sip() & ~2);
}

```

对于外部设备中断，则交给external_interrupt_handler解决，代码如下：

```

// 外设中断处理 (基于PLIC)
void external_interrupt_handler()
{
    // irq indicates which device interrupted.
}

```

```

int irq = plic_claim();

switch(irq){
    case 0:
        break;
    case UART_IRQ:
        uart_intr();
        break;
    case VIRTIO0_IRQ:
        // virtio_disk_intr();
        break;
    default:
        printf("unexpected interrupt irq=%d\n", irq);
}

// the PLIC allows each device to raise at most one
// interrupt at a time; tell the PLIC the device is
// now allowed to interrupt again.
if(irq)
    plic_complete(irq);
}

```

该函数判断外设中断的类型并且调用相关功能处理，在完成后同样承认中断，从而实现外部中断的处理。

2.1.6 实现异常处理机制

由代码可知，本系统不处理任何异常，如果发生，则报告调试信息，然后进入panic卡死即可。

2.1.7 其他文件的修改

在完成上列过程之后，为了启动中断处理程序，需要对其他文件做如下改动：

对于`start.c`，在机器启动时，需要关闭中断，并初始化时钟。

```

.....
// 关闭所有中断和异常
w_medeleg(0xffff);
w_mideleg(0xffff);
w_sie(r_sie() | SIE_SEIE | SIE_STIE | SIE_SSIE);

// 请求时钟中断的启动
timer_init();
.....

```

对于`main.c`，需要对每个核进行相关的初始化设置，在设置好之后，开启中断处理程序：

```

void main()
{
    int cpuid = r_tp();

```

```
if(cpuid == 0){
    // 进行初始化的其他部分
    trap_kernel_init();
    trap_kernel_inithart();
    plic_init();
    plic_inithart();

    printf("cpu %d is booting!\n", cpuid);
    __sync_synchronize();
    started = 1;
}
else{
    while(started == 0) {};
    __sync_synchronize();
    printf("cpu %d is booting!\n", cpuid);
    kvm_inithart();
    trap_kernel_inithart();
    plic_inithart();
}
intr_on();
while(1);
}
```

经过如上设置，系统在开机时将会初始化并启动中断处理程序。

2.2 问题与解决方案

2.2.1 常见问题

问题：中断无响应

解决方案：中断无响应，可能是中断被禁止，也可能是中断源配置不良，首先需要检查中断使能位设置，其次需要验证中断向量地址，同时保证程序开始的时候中断已经开启。最后还需要确认中断源是否正确配置

问题：系统在中断处理后崩溃

解决方案：首先需要检查栈指针保存和恢复，其次验证上下文保存的完整，最后确认中断处理函数没有破坏调用约定

问题：中断频率异常

解决方案：首先需要检查时钟设置参数（如**INTERVAL**不应过大过小），同时需要查看时钟中断相关调用是否正确，最后需要确认计算是否溢出。

2.3 源码理解总结

同xv6，本操作系统利用RISC-V拥有的中断处理功能从而进行中断和异常的处理。系统将中断和异常统称为陷阱，在中断处理函数中判别并且分别处理。于此同时，操作系统利用中断委托机制，将各个模式下的陷阱集中到S模式进行处理，从而减少多余的上下文切换次数，提升系统效率。

同xv6，本操作系统利用了RISC-V中的状态寄存器进行对应的陷阱处理，其过程遵循了RISC-V中的中断处理流程：保存现场、关中断、跳转陷阱处理程序、保存上下文、进入进一步的陷阱处理程序（在其中判别中断和异常）、恢复上下文、返回原地。其符合RISC-V操作系统的设计规范。

`trap_kernel_handler`函数作为进一步的陷阱处理程序，其不仅保证了中断和异常的判别，还调用了相关的处理函数对于不同情况进行处理，从而使得中断的处理能够有序的进行。

3. 测试验证部分

3.1 功能测试结果

3.1.1 时钟中断测试

为了测试时钟中断是否能够及时响应，在时钟中断当中增加一个输出语句，每次启用时钟中断时，将会输出当前timer的ticks数目。代码如下所示：

```
void timer_interrupt_handler()
{
    // 输出当前ticks的测试代码部分
    int ticks = timer_get_ticks();
    printf("Current tick is %d.\n", ticks);

    // 其他部分保持不变
}
```

测试结果如下：

```
riptide@admin:~/whu-oslab$ make clean && make qemu
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaultin
g to 0000000080000000
make[1]: Leaving directory '/home/riptide/whu-oslab/kernel'
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M
-smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
Current tick is 0.
Current tick is 0.
Current tick is 1.
Current tick is 2.
Current tick is 2.
Current tick is 2.
Current tick is 3.
Current tick is 3.
Current tick is 4.
Current tick is 4.
Current tick is 5.
Current tick is 5.
Current tick is 6.
Current tick is 7.
Current tick is 7.
Current tick is 7.
Current tick is 8.
Current tick is 8.
Current tick is 9.
Current tick is 9.
Current tick is 10.
Current tick is 10.
```

可以看到，屏幕上逐个出现有关输出当前ticks数的输出信息，且ticks数逐渐增长。（由于实际上有2个核在运行，所以配置了两个时钟，输出的信息略有重叠）修改参数`INTERVAL`的值，可以看到输出的速率发生变化，说明时钟中断处理正常。

3.1.2 外部中断测试

为了测试当前是否满足外部中断，由于外设已经连接上uart，所以在中断查看当前是否已经满足了输入的显示即可。

启动操作系统之后，尝试在键盘上输入字符。测试结果如下：

```
riptide@admin:~/whu-oslab$ make clean && make qemu
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel.ld -o ../kernel-qemu ./boot/entry.o ./bo
mer.o ./dev/uart.o ./lib/print.o ./lib/spinlock.o ./lib/str.o ./mem/kvm.o ./mem/pmem.o ./proc
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaulting to 0000000080000000
make[1]: Leaving directory '/home/riptide/whu-oslab/kernel'
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
cpu 0 is booting!
cpu 1 is booting!
Hello world!
```

可以看到，键盘的输入能够在终端中显示，所以外部终端运行正常。

4. 结论与心得

本实验我收获颇多，具体如下：

我理解了RISC-V中的中断委托机制，理解了mideleg/medeleg寄存器的作用，以及为什么需要将时钟中断从M模式委托到S模式处理；同时，我也知道不同特权级别下的中断需要保存的寄存器数量不同，M模式时钟中断只需保存a0-a3，而S模式中断需要保存全部通用寄存器。同时我也知道系统会通过sstatus.SIE位的自动管理，防止中断处理程序被嵌套打断。

除此以外，我还提升了代码能力，对于操作系统有了更佳深入的了解，更佳懂得如何调试。这些都会在我的未来中有很大作用。

实验 4：首个用户态进程的创建

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

0. 实验任务和目标

1. 通过深入分析xv6的进程管理机制，理解操作系统如何创建和管理进程。
2. 学会如何创建并切换到用户态的第一个进程`proczero`，并将其作为CPU核运行的第一个用户态进程
3. 掌握如何在用户态下处理陷阱（含系统调用，中断，异常等），实现最基本的第一个进程的相关系统调用实现，最终在保证原有中断的情况下实现输出对应的系统调用信息。

1. 系统设计

1.1 架构设计说明

1.1.1 第一个进程的实现

进程（process）是资源分配的基本单位，也是独立运行的基本单位。在操作系统中，各个进程分别运行。从而实现操作系统的各项功能。

本操作系统实现了进程的基本数据结构，同时也对于用户态进程的启动进行了实现。具体实现和机理请看下文。

1.1.2 上下文切换和陷阱处理

为了解决多任务并发并且保障操作系统的稳定性及其他进程的安全，RISC-V定义了多个特权级别从而实现权限的控制和隔离。其中，U模式（用户模式）为应用程序运行的模式，权限最低。它无法直接执行特权指令（如操作 I/O、管理内存映射），也无法随意访问整个物理内存空间。在这个空间内，进程可以安全地运行，其错误会被控制在局部，其行为必须通过操作系统内核的审核。

为了实现从处在内核态S模式下的模式转变成处在U模式下的用户进程，亦或者用户态下产生了陷阱，需要进行处理；都需要进行上下文的切换。具体的机理如下：

从U到S的切换

当执行陷阱时，除了时钟中断，其他的过程和内核态陷阱的处理流程大致相同。具体过程为：

1. 保存现场：将当前pc保存到sepc、将陷阱原因（中断类型或异常代码）保存到scause、将附加信息（如错误地址）保存到stval。
2. 关中断：在此之后，系统将sstatus中的SIE位清零，屏蔽后续的中断，防止处理程序被意外打断。
3. 跳转：跳转至stvec寄存器中的地址，开始执行陷阱处理程序。
4. 保存上下文：在汇编代码中，程序将会将寄存器的值保存在栈中，从而保存完整的上下文。之后程序将会调用中断处理函数。
5. 中断处理程序：在软件中，系统读取寄存器的值，并分析陷阱的类型（中断/异常），调用相关的处理程序进行中断处理

6. 恢复上下文：从内核栈上恢复之前保存的所有寄存器。
7. 返回：执行SRET指令。该指令会将pc恢复为sepc的值，并重新打开中断（恢复sstatus中的SIE位）。

注意：CPU不会自动切换到内核的页表，也不会切换到内核栈，也不会保存除了pc之外的寄存器的值，内核需要自行完成。这也是本操作系统需要实现的一点。本操作系统使用trapframe这个数据结构从而实现。

从S到U的切换

为了实现第一个用户进程的运行，需要将内核态转化为用户态，具体流程如下：

1. 内核态设置：要手动清除sstatus的SPP字段，将其置为零；将sstatus的SPIE字段置为1，启用用户中断；设置sepc为用户进程的PC值。如果启用了页表，就需要还原用户进程的页表，即将用户进程的页表地址写入satp。
2. 恢复上下文：使用context这个数据结构，完成寄存器值的存储和交换。
3. 执行sret命令，使得回到用户进程。

1.2 关键数据结构

1.2.1 进程proc

本系统使用proc作为表示进程的数据结构。由于本系统中只需要完成其中一个进程的创建，不实现进程状态、多进程、时间片、有实际功能的系统调用等。所以进程的实际定义比xv6中的简单，其各个字段和作用如下所示：

```
// 进程定义
typedef struct proc {
    int pid;                // 标识符

    pgtbl_t pgtbl;          // 用户态页表
    uint64 heap_top;        // 用户堆顶(以字节为单位)
    uint64 ystack_pages;    // 用户栈占用的页面数量
    trapframe_t* tf;        // 用户态内核态切换时的运行环境暂存空间

    uint64 kstack;          // 内核栈的虚拟地址
    context_t ctx;          // 内核态进程上下文
} proc_t;
```

1.2.2 改进的CPU模块CPU

相比于之前的实验，CPU新增了“当前正在运行的进程”这一要素，所以除了移动相关模块之外，还需要补充相关字段，从而表示CPU当前运行的进程和上下文暂存（为了便于交换），其定义和相关函数如下所示：

```
typedef struct cpu {
    int noff;               // 关中断的深度
    int origin;             // 第一次关中断前的状态
    proc_t* proc;           // cpu上运行的进程
    context_t ctx;          // 内核上下文暂存
} cpu_t;
```

```
int    mycpuid(void);
cpu_t* mycpu(void);
proc_t* myproc(void);
```

1.2.3 上下文context

为了实现上下文切换，需要使用context进行切换。context字段中保存了被调用者保存寄存器的值，从而易于交换，如下所示：

```
typedef struct context {
    uint64 ra; // 返回地址
    uint64 sp; // 栈指针

    // callee-saved
    uint64 s0;
    uint64 s1;
    uint64 s2;
    uint64 s3;
    uint64 s4;
    uint64 s5;
    uint64 s6;
    uint64 s7;
    uint64 s8;
    uint64 s9;
    uint64 s10;
    uint64 s11;
} context_t;
```

1.2.4 陷阱框trapframe

当在用户态中有陷阱（中断或者异常）产生时，需要从用户态模式转变为内核态模式。这时，需要使用trapframe这个数据结构存储各个寄存器的值，同时存储页表等信息，其定义如下：

```
typedef struct trapframe {
    /* 0 */ uint64 kernel_satp; // kernel page table
    /* 8 */ uint64 kernel_sp;   // top of process's kernel stack
    /* 16 */ uint64 kernel_trap; // usertrap()
    /* 24 */ uint64 epc;        // saved user program counter
    /* 32 */ uint64 kernel_hartid; // saved kernel tp

    /* 40 */ uint64 ra;
    /* 48 */ uint64 sp;
    /* 56 */ uint64 gp;
    /* 64 */ uint64 tp;
    /* 72 */ uint64 t0;
    /* 80 */ uint64 t1;
    /* 88 */ uint64 t2;
    /* 96 */ uint64 s0;
    /* 104 */ uint64 s1;
```

```
/* 112 */ uint64 a0;
/* 120 */ uint64 a1;
/* 128 */ uint64 a2;
/* 136 */ uint64 a3;
/* 144 */ uint64 a4;
/* 152 */ uint64 a5;
/* 160 */ uint64 a6;
/* 168 */ uint64 a7;
/* 176 */ uint64 s2;
/* 184 */ uint64 s3;
/* 192 */ uint64 s4;
/* 200 */ uint64 s5;
/* 208 */ uint64 s6;
/* 216 */ uint64 s7;
/* 224 */ uint64 s8;
/* 232 */ uint64 s9;
/* 240 */ uint64 s10;
/* 248 */ uint64 s11;
/* 256 */ uint64 t3;
/* 264 */ uint64 t4;
/* 272 */ uint64 t5;
/* 280 */ uint64 t6;
} trapframe_t;
```

1.3 设计决策理由

本系统使用和xv6类似的进程结构，这样便于系统对于进程的要点进行详细的管理。同时其用户态和上下文切换过程也符合RISC-V标准。这样就实现了对于用户进程的切换以及陷阱处理。

1.4 与 xv6 对比分析

本系统与xv6的对比分析如下：

与xv6相同，本系统采用类似的进程和CPU结构，且采用相同的上下文切换机制，同样采用RISC-V的U模式作为用户态的模式。除此以外，二者分别有一个在初始化完成后运行的用户态进程。

而相比xv6，由于本系统无系统状态、多进程等，所以在进程的初始化定义上较为简单，进程的数量，进程的操作相比xv6中的少很多，但这也表示其相比xv6有更简洁和简单的结构。

2. 实验过程部分

2.1 实现步骤记录

2.1.1 深入理解进程抽象

首先，阅读代码，分析分析xv6的进程结构体。各个段的展示如注释所示：

```
// Per-process state
struct proc {
```

```

struct spinlock lock;

// 使用前需要p->lock:
enum procstate state;           // 进程状态
void *chan;                     // 若非0, 则在chan上休眠
int killed;                     // 若非0, 则已被杀死
int xstate;                     // 返回给父进程的退出状态
int pid;                        // 进程id号

// 使用前需要wait_lock:
struct proc *parent;            // 父进程

// 每个进程独立的数据, 访问时无需上锁
uint64 kstack;                  // 内核栈的虚拟地址
uint64 sz;                      // 进程内存的大小 (字节)
pagetable_t pagetable;         // 用户页表
struct trapframe *trapframe;    // trampoline.S的数据页
struct context context;         // 对其使用swtch()函数从而运行进程, 上下文
struct file *ofile[NOFILE];     // 已打开的文件
struct inode *cwd;              // 当前目录
char name[16];                  // 进程名 (debug用)
};

```

在这里, 进程的状态类`procstate`定义如下所示。进程的生命周期为UNUSED → USED → RUNNABLE → RUNNING → SLEEPING → ZOMBIE。

```
enum procstate { UNUSED, USED, SLEEPING, RUNNABLE, RUNNING, ZOMBIE };
```

由于本次实验未实现多进程以及进程状态管理, 所以有关于进程状态的详细内容和进程分配机理在此处不再讲述, 详情请看实验六以了解

2.1.2 分析xv6第一个进程的创建过程

接下来, 阅读xv6代码, 了解xv6的第一个进程是如何创建出来的。xv6使用`userinit`函数对于第一个进程进行对应的初始化, 如下所示:

```

// Set up first user process.
void
userinit(void)
{
    struct proc *p;

    p = allocproc();
    initproc = p;

    // allocate one user page and copy initcode's instructions
    // and data into it.
    uvmfirst(p->pagetable, initcode, sizeof(initcode));
    p->sz = PGSIZE;

```

```

    // prepare for the very first "return" from kernel to user.
    p->trapframe->epc = 0;          // user program counter
    p->trapframe->sp = PGSIZE;      // user stack pointer

    safestrcpy(p->name, "initcode", sizeof(p->name));
    p->cwd = namei("/");

    p->state = RUNNABLE;

    release(&p->lock);
}

```

可以知道，此部分首先使用`allocproc`寻找可用进程，之后为其映射页表，修改id编号为0，配置trapframe等。其中，`allocproc`的主要功能为从进程列表中寻找一个可用的进程并将其初始化，其详细的机理详情请看实验六的内容。

2.1.3 设计第一个系统进程

在了解xv6系统进程创建和管理机制过后，接下来就可以设计第一个系统进程了。由于本次实验创建了一个用户进程`proczero`，而没有如多进程和进程状态相关的内容，所以实际上其结构相比xv6简单一些。

首先，创建第一个进程`proczero`，同时参考xv6中的`userinit`函数，为其书写初始化函数。这个函数设计了其pid，页表，栈的设置等。同时准备好陷阱框`trapframe`和上下文`context`以准备切换。在函数的最后，将会调用上下文切换程序从而切换到进程，其具体机制请看2.1.4节

```

// 第一个进程
static proc_t proczero;

void proc_make_first()
{
    // pid 设置
    proczero.pid = 0;

    // pagetable 初始化
    uint64 trapframe_page = (uint64)pmem_alloc(false);
    proczero.pgtbl = proc_pgtbl_init(trapframe_page);

    // ustack 映射 + 设置 ustack_pages
    // proc_mapstacks已经完成了栈的设置和映射
    proczero.kstack = KSTACK(0);
    uint64 ustack_phys = (uint64)pmem_alloc(true);
    vm_mappages(proczero.pgtbl, proczero.kstack - PGSIZE, ustack_phys, PGSIZE,
                PTE_R | PTE_W | PTE_U);

    // data + code 映射
    assert(initcode_len <= PGSIZE, "proc_make_first: initcode too big\n");
    char *mem = (char *)pmem_alloc(false);
    memset(mem, 0, PGSIZE);
    vm_mappages(proczero.pgtbl, 0, (uint64)mem, PGSIZE, PTE_W|PTE_R|PTE_X|PTE_U);
    memmove(mem, initcode, initcode_len);
}

```

```

proczero.ustack_pages = 1;

// 设置 heap_top
proczero.heap_top = (uint64)initcode + PGSIZE;

// tf字段设置
proczero.tf = (struct trapframe *)trapframe_page;
memset(proczero.tf, 0, sizeof(*proczero.tf));
// 设置用户态返回时的关键寄存器
proczero.tf->epc = 0; // 程序计数器, 从虚拟地址0开始执行initcode
proczero.tf->sp = PGSIZE; // 用户栈指针, 设置在用户空间顶部

// 内核字段设置
memset(&proczero.ctx, 0, sizeof(&proczero.ctx));
proczero.ctx.ra = (uint64)trap_user_return;
proczero.ctx.sp = proczero.kstack + PGSIZE;

// 上下文切换
mycpu()->proc = &proczero;
swtch(&mycpu()->ctx, &proczero.ctx);
}

```

注：由于进程只有一个，所以在xv6中相关分配进程的函数在这里被展开。在实验六中，由于新增进程分配函数，将会对此函数进行再一次的修改。

为了使这个进程能够被执行，需要在main函数中添加这一部分的内容。

```

void main()
{
    int cpuid = r_tp();
    if(cpuid == 0){
        // 其他部分保持不变
        // procinit();
        trap_kernel_init();
        trap_kernel_inithart();
        plic_init();
        plic_inithart();
        proc_make_first();

        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
    }
    // 其他部分保持不变
}

```

在此之后，需要注意的是，由于内核态和用户态的核将会使用两套不同的页表，所以需要对于关键的物理地址进行映射，补充的物理地址映射代码如下所示：

```
pgtbl_t kvm_create(){
    pgtbl_t kpgtbl;

    // 其他部分保持不变

    // trampoline, 用于trap中虚拟地址的映射
    vm_mappages(kpgtbl, TRAMPOLINE, (uint64)trampoline, PGSIZE, PTE_R | PTE_X);
    // printf("Trampoline address: va: %x, pa: %x\n", TRAMPOLINE,
    (uint64)trampoline);

    // 为每个进程创建一个栈
    proc_mapstacks(kpgtbl);

    return kpgtbl;
}
```

其中, `proc_mapstacks` 负责对每个进程分配栈空间, 其函数的内容如下所示 :

```
void proc_mapstacks(pgtbl_t kpgtbl){
    char* pa = pmem_alloc(false);
    uint64 va = KSTACK(0);
    vm_mappages(kpgtbl, va, (uint64)pa, PGSIZE, PTE_R | PTE_W);
}
```

在此之后, 设计第一个进程的代码, 并且将其编译。在跳转到这个进程的上下文之后, 内核就在用户态下运行 `initcode` 下的代码。该进程的代码如下 :

```
#include "sys.h"

int main()
{
    syscall(SYS_print);
    syscall(SYS_print);
    while(1);
    return 0;
}
```

在使用 `init` 指令对代码编译之后, 其数据和代码存放在 `initcode` 下, 其包含代码和数据段等信息。从而等待执行。

2.1.4 实现上下文切换机制

接下来需要实现上下文切换机制, 使得内核能够在内核态和用户态之间进行交换。利用 `context` 这一个数据结构, 设计对应的上下文切换机制。

参考 xv6 的 `swtch.S`, 书写对应代码如下 :

```
# void swtch(struct context *old, struct context *new);

.globl swtch
swtch:
    sd ra, 0(a0)
    sd sp, 8(a0)
    sd s0, 16(a0)
    sd s1, 24(a0)
    sd s2, 32(a0)
    sd s3, 40(a0)
    sd s4, 48(a0)
    sd s5, 56(a0)
    sd s6, 64(a0)
    sd s7, 72(a0)
    sd s8, 80(a0)
    sd s9, 88(a0)
    sd s10, 96(a0)
    sd s11, 104(a0)

    ld ra, 0(a1)
    ld sp, 8(a1)
    ld s0, 16(a1)
    ld s1, 24(a1)
    ld s2, 32(a1)
    ld s3, 40(a1)
    ld s4, 48(a1)
    ld s5, 56(a1)
    ld s6, 64(a1)
    ld s7, 72(a1)
    ld s8, 80(a1)
    ld s9, 88(a1)
    ld s10, 96(a1)
    ld s11, 104(a1)

    ret
```

有`context`的数据结构等可知，所有被调用者保存的寄存器需要去寄存，无需保存所有的寄存器。因为其特权级切换，类似于函数调用，调用者保存的寄存器已经在转换成用户态前已经保存好。这样的上下文切换机制类似于函数调用，仅需保存相关的寄存器即可。

2.1.5 实现陷阱与简单的系统调用

在`initcode.c`中，用户态进程将会进行两次系统调用，然后陷入死循环。对应的，需要在用户态中实现对于陷阱的识别，并在识别出系统调用之后，输出对应信息。

有关于用户态下陷阱的捕获和处理函数，其具体机理以及实现细节详情请看实验五以及代码库中的代码，在这里简要阐述用户态陷阱的处理：

在用户态即U模式下，陷阱的处理和S模式下类似。当陷阱触发时，系统由于中断委托机制将中断委托给S模式，之后在内核中，系统会调用在`trampoline`中的中断处理程序，在保存寄存器后跳转到处理函数

`trap_user_handler`，并在该函数下对于陷阱进行识别和处理。处理完之后，由于中断委托给S模式，所以对应的，需要设计`user_return`函数，使得系统能够在陷阱处理完成后恢复到U模式。

目前暂定陷阱处理函数`trap_user_handler`如下。由于当前未实现真正的系统调用，所以在系统调用部分直接输出调试信息即可：

```
void trap_user_handler()
{
    uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
    uint64 sstatus = r_sstatus();     // 与特权模式和中断相关的状态信息
    uint64 scause = r_scause();       // 引发trap的原因
    uint64 stval = r_stval();         // 发生trap时保存的附加信息(不同trap不一样)
    proc_t* p = myproc();

    // 确认trap来自U-mode
    assert((sstatus & SSTATUS_SPP) == 0, "trap_user_handler: not from u-mode");

    // send interrupts and exceptions to kerneltrap(),
    // since we're now in the kernel.
    w_stvec((uint64)kernel_vector);

    // save user program counter.
    p->tf->epc = sepc;

    // 针对scause制定的一系列规则
    int trap_id = scause & 0xf;
    bool isInterrupt = ((scause & ((uint64)1 << 63)) != 0);
    char* info = isInterrupt ? interrupt_info[trap_id] : exception_info[trap_id];

    if(scause == 8){
        // system call
        // if(killed(p)) exit(-1);

        // sepc points to the ecalls instruction,
        // but we want to return to the next instruction.
        p->tf->epc += 4;
        printf("get a syscall from proc %d\n", p->pid);

        // syscall();

        // an interrupt will change sepc, scause, and sstatus,
        // so enable only now that we're done with those registers.
        intr_on();
    } else if (isInterrupt) {
        switch (trap_id)
        {
            case 1:
                // 处理软件中断
                timer_interrupt_handler();
                break;
            case 5:
```

```

        // 处理时钟中断
        // Pass anyway...
        break;
    case 9:
        // 处理外部中断
        external_interrupt_handler();
        break;
    default:
        printf("usertrap(): unexpected interrupt at pid=%d\n", p->pid);
        printf("          trap id: %d trap info: %s\n", trap_id, info);
        printf("          scause %p\n", scause);
        printf("          sepc=%p stval=%p\n", sepc, stval);
        panic("usertrap: unexpected interrupt");
        break;
    }
} else {
    // 异常处理
    printf("usertrap(): exception at pid=%d\n", p->pid);
    printf("          trap id: %d trap info: %s\n", trap_id, info);
    printf("          scause %p\n", scause);
    printf("          sepc=%p stval=%p\n", sepc, stval);
    panic("usertrap: unexpected exception");
    // setkilled(p);
}

trap_user_return();
}

```

2.2 问题与解决方案

由于手册中暂无有关于本次实验的常见问题，在此处写下自己在实践过程当中所遇到的问题。

2.2.1 自己遇见的问题

问题：系统无输出但是卡住，且在gdb调试当中，在上下文切换后汇编代码消失，且发现跳转地址怪异。

解决方案：很大一部分原因在于没有及时将用户进程的页表项配置好，导致在上下文切换之后系统跳转到了错误的地址。为了解决这个问题，需要检查用户态进程的栈空间，上下文和陷阱框是否已经配置完成（注意在kvm文件中需要有用户进程的页表配置）。同时检查进程初始化时，地址空间是否正确；还有就是调用user_return时需要确定参数是否正确。

问题：系统产生异常IMO PAGE FAULT异常

解决方案：同样，可能是地址配置异常所致，解决方案同上。

问题：系统产生unexcepted scause异常

解决方案：需要检查该scause是否已经处理完成。在这里需要指明。注意trap_user_handler虽然属于用户态时中断的处理函数，但其实际所在S模式下，所以仍然需要处理时钟中断和外部中断等。

2.3 源码理解总结

进程是资源分配的基本单位，也是独立运行的基本单位。系统使用一个进程的结构体表示进程，其中保存了一系列相关的信息。在进程初始化进程中，需要对其上下文、陷阱框、页表等进行配置，从而使得系统上下文切换之后，代码可以正常执行。

用户态（U）和内核态（S）的切换需要利用`context`进行上下文切换。其中函数`swtch`保存相关寄存器并且更新页表，这样系统即切换到了用户进程进行处理。

用户态下陷阱的处理委托给S模式下的函数进行处理。处理过程与S模式下陷阱的处理大致相似，在中断处理函数中判别出系统中断并且做相关处理。在中断处理结束之后，程序将会转回U模式，继续执行用户态下的代码。

3. 测试验证部分

3.1 功能测试结果

为了测试功能是否正常，按照上述流程写好代码之后，启动项目，必要时配合gdb进行调试。

根据代码可知，第一个核应该转移到用户态进程`proczero`并运行其中的代码，执行两次系统调用后进入死循环。如果系统转移到内核态运行代码，且成功输出两次系统调用的信息，且其他类型的中断功能保持正常，则本测试通过

启动项目，其结果如下：

```
riptide@admin:~/whu-oslab$ make clean && make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -MD -mcmmodel-medany -ffreestanding -fno-common -nostdlib -mno-relax -I. -fno-stack-protector
-fno-pie -no-pie -I ../include -c trap_user.c
make[2]: Leaving directory '/home/riptide/whu-oslab/kernel/trap'
ls: cannot access './*/*/*.o': No such file or directory
riscv64-linux-gnu-ld -z max-page-size=4096 -T kernel.ld -o ../kernel-qemu ./boot/entry.o ./boot/main.o ./boot/start.o ./dev/plic.o ./dev/timer.o ./dev/uart.o ./lib/print.o
./lib/spinlock.o ./lib/str.o ./mem/kvm.o ./mem/pmem.o ./proc/cpu.o ./proc/proc.o ./proc/swtch.o ./trap/trampoline.o ./trap/trap.o ./trap/trap_kernel.o ./trap/trap_user.o
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaulting to 0000000008000000
make[1]: Leaving directory '/home/riptide/whu-oslab/kernel'
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic
get a syscall from proc 0
get a syscall from proc 0
Hello World! Thank you!!!
```

可以看到，有两个系统调用信息输出，同时外部中断（输入），时钟中断正常。所以功能测试正常。

4. 结论与心得

本实验是目前为止除实验三之外耗时最长的实验，由于在上下文切换期间关闭中断的情况，所以在系统发生错误（例如上文提到的物理内存未在新的页表中映射）时，系统将会死机而不输出任何信息，这就导致了项目的修改十分盲目。但是这次的实验也让我收获颇多：

首先也是最重要的就是我学会如何使用gdb分终端对于操作系统进行调试，且利用gdb的部分指令（如`layout asm`，`si`）等对于操作系统的代码进行调试，从而更好的找到代码错误的产生原因。这次的经历使我对于代码的调试更佳深入，让我在未来能更容易找到系统的错误原因。

其次，我深入了解了RISC-V操作系统的M，S，U模式的区分，同时了解了S/U模式的权限分隔机制和转换机制，了解了如何使用`context`实现上下文切换，且了解了如何在用户态下将陷阱委托给S模式进行处理，理解了简单的系统调用流程。这些让我对于RISC-V操作系统有了更深入的了解

最后，这次实验用时较长，增长了我的耐心，增加了我的代码能力，使我在未来能够更佳好的处理这些任务。

实验 5：系统调用实现

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

0. 实验任务和目标

- 通过分析xv6的系统调用机制，深入理解用户态与内核态的交互方式。
- 参考xv6代码，实现完整的系统调用框架
- 在单进程的情况下，实现一些功能较小的系统调用，如无功能系统调用SYS_test，修改用户进程堆空间的SYS_brk

1. 系统设计

1.1 架构设计说明

1.1.1 用户态下的陷阱处理框架

在实验三中，我们介绍了xv6在内核态（S模式）下的陷阱处理架构。用户态（U模式）下的陷阱处理是其延伸，主要通过中断委托机制将陷阱交给S模式处理。具体流程如下：

陷阱处理的硬件自动阶段（由CPU自动执行）：

当在用户态发生陷阱（中断或异常）时，如果该陷阱已通过medeleg（异常委托）和mideleg（中断委托）寄存器委托给S模式，硬件会自动执行以下操作：

- 保存现场：将当前程序计数器（pc）保存到sepc寄存器，将陷阱原因（中断类型或异常代码）保存到scause寄存器，并将附加信息（如错误地址）保存到stval寄存器。
- 关中断：将sstatus寄存器中的SIE位清零，禁用后续中断，防止处理过程被打断。
- 模式切换：将处理器从U模式切换到S模式。
- 跳转：跳转到stvec寄存器中保存的地址（即陷阱处理程序的入口点）。

陷阱处理的软件阶段（由操作系统代码执行）：

在跳转到stvec指定的地址后，软件陷阱处理程序开始执行：

- 保存上下文：在汇编代码（如xv6的trampoline代码）中，将当前所有寄存器的值保存到内核栈中，以保存完整的用户上下文。然后，调用高级陷阱处理函数。
- 中断处理程序：在C代码（如trap_user_handler函数）中，读取scause等寄存器的值，分析陷阱类型（中断或异常），并调用相应的处理程序（如系统调用处理、页错误处理等）。
- 恢复上下文：处理完成后，从内核栈恢复之前保存的所有寄存器值。
- 返回用户态：执行SRET指令。该指令会将sepc的值恢复为pc（从而返回到用户态的下一条指令），并恢复sstatus中的SIE位（重新启用中断），同时处理器模式从S模式返回U模式。

在xv6的具体实现中，用户态陷阱通过trampoline页面处理，该页面包含用于安全模式切换的汇编代码。陷阱处理函数trap_user_handler负责用户态特定的逻辑，最后通过user_return函数完成返回到用户态的清理工作。

1.1.2 系统调用的实现

在用户态下，系统调用通过ecall指令触发。该指令会产生一个陷阱，其处理流程与中断或其他异常类似，共享同一套底层框架。在xv6中，一个系统调用的完整生命周期如下：

1. 用户程序发起调用：用户程序调用一个封装函数（如 read）。
2. 参数准备与跳转：封装函数通常位于 usys.S（或类似的汇编桩代码文件）中。它的职责是：
 - 将系统调用号存入约定的寄存器（在RISC-V的xv6实现中为a7）。
 - 执行ecall指令。
3. 触发陷阱并切换模式：ecall指令的执行会触发一个从用户态（U模式）到内核态（S模式）的陷阱。硬件自动完成以下操作：
 - 保存当前PC到sepc。
 - 将陷阱原因（环境调用异常）存入 scause。
 - 切换到S模式并跳转到 stvec寄存器所指向的陷阱处理入口。
4. 保存执行上下文：陷阱处理入口代码（在xv6的trampoline页中，通常是 uservec函数）保存所有用户寄存器到当前进程的内核陷阱帧（trapframe）中。
5. 分配内核栈并路由陷阱：随后，控制权转移到C语言编写的通用陷阱处理程序 usertrap。该函数通过检查 scause寄存器来判断陷阱类型。当识别到是系统调用（即scause指示为环境调用异常，在xv6中为8）后，便调用syscall()函数。
6. 分派并执行系统调用：在syscall()函数中：
 - 从陷阱帧中读取a7寄存器保存的系统调用号。
 - 使用该系统调用号作为索引，查询系统调用表，找到对应的内核函数实现。
 - 执行该内核函数。
7. 处理参数与返回值：系统调用的内核实现函数从陷阱帧中获取用户传递的参数（通常来自a0, a1等寄存器），执行具体操作。函数执行完毕后，其返回值会被存回陷阱帧中的返回值寄存器（如a0）位置，以便返回用户态。
8. 返回用户态：系统调用处理完毕，内核最终通过usertrapret()和 trampoline中的代码（如userret）执行返回序列：恢复用户寄存器，将程序计数器设置为sepc+4（指向ecall之后的下一条指令），并执行sret指令切换回用户模式。

本系统的设计与xv6的上述流程基本一致，但在用户态的桩代码实现上进行了简化。

1.2 关键数据结构

1.2.1 系统调用表

为了实现系统调用，除了原有的用户态的陷阱处理程序之外，还需要识别系统调用并且检测系统调用号，从找到并执行对应的内核函数。

在本系统中，系统调用表为如下的一个结构：

```
static uint64 (*syscalls[])(void) = {
    [SYS_test]      sys_test,
    [SYS_print]     sys_print,
    [SYS_brk]       sys_brk,
    [SYS_mmap]      sys_mmap,
    [SYS_munmap]    sys_munmap,
    [SYS_fork]      sys_fork,
    [SYS_wait]      sys_wait,
```

```
[SYS_exit]      sys_exit,  
[SYS_sleep]     sys_sleep,  
[SYS_kill]      sys_kill,  
[SYS_getpid]    sys_getpid,  
};
```

本质上，系统调用表是一个函数指针数组，其中，索引为如下宏定义所示，表示系统调用号：

```
#define SYS_test      0  
#define SYS_print     1  
#define SYS_brk       2  
#define SYS_mmap      3  
#define SYS_munmap    4  
#define SYS_fork      5  
#define SYS_wait      6  
#define SYS_exit      7  
#define SYS_sleep     8  
#define SYS_kill      9  
#define SYS_getpid    10
```

其中的值，则为指向sysfunc.c文件中的不同函数的指针。

在系统调用的分派时，系统将会从a7寄存器中读取进程调用号，之后从系统调用表中找到对应的系统调用实现函数的函数指针，接着调用该系统调用实现函数，从而进行更为详细的操作。

1.3 设计决策理由

本系统与xv6采用相同的系统调用机制，其核心设计思想是在 RISC-V 硬件特性、安全性与隔离性、以及代码的简洁性和清晰度之间取得平衡。具体来说，其主要设计理由如下：

1. 强制进程隔离，构建安全边界 系统调用是用户进程请求内核服务的唯一合法入口。通过 `ecall` 指令触发陷阱，强制处理器从权限较低的用户态（U模式）切换到权限最高的内核态（S模式）。这种硬件级的权限切换是实现进程隔离和安全的基础，确保了用户进程无法任意访问或修改内核数据，从而将不可信的用户代码与关键的内核代码隔离开来。
2. 利用统一的陷阱处理框架，实现关注点分离 系统调用、缺页异常、定时器中断等都被视为“陷阱”。xv6 为所有陷阱设计了一套统一的处理流程（从 `uservec` 到 `usertrap`）。这样做不仅实现了代码的复用，并且使得代码的逻辑更加清晰
3. 通过“蹦床”页面实现安全的上下文切换 `trampoline` 页面的设计是精妙之处。它被映射到所有进程地址空间的相同虚拟地址，同时也在内核地址空间中。这样可以得内核可以在不依赖内核页表的情况下开始执行，进而安全的保存上下文。
4. 清晰的参数传递约定，降低复杂度 设计上规定通过寄存器（如 `a7` 存放系统调用号，`a0-a5` 存放参数）和进程的 `trapframe` 在内核与用户态之间传递参数，这是一种简单高效的方案。它避免了在陷入内核的复杂时刻去解析用户栈等不安全操作，所有用户信息都被集中、安全地保存在 `trapframe` 这个“快照”中供内核查验，降低了安全风险。

5. 面向教学的可读性与简洁性 最终，xv6 是一个教学操作系统。这套设计虽然可能不是性能最优的（例如，每次都会保存/恢复所有寄存器），但其最大的优点是逻辑清晰、易于理解。它将系统调用的完整生命周期——从用户陷入、到内核分发、再到返回用户——的每一步都清晰地展现出来，非常适合学习者深入理解操作系统的核心机制。

总而言之，xv6 的系统调用设计通过硬件陷阱机制和一套精心设计的软件流程，巧妙地实现了权限控制、安全隔离和架构简洁这三大目标。它不仅能够正确工作，更重要的是，其结构清晰地揭示了现代操作系统管理用户请求的核心原理。

1.4 与 xv6 对比分析

在系统调用流程实现以及系统调用表的数据结构来看，本系统的设计与xv6的上述流程基本一致。通过这样的系统调用结构，本系统和xv6一样，实现了权限控制、安全隔离和架构简洁这三大目标。

但是，于此同时，相比于xv6，其系统调用的实现细节上有所不同。例如，本系统的系统调用号与xv6安排不同，但这不影响系统调用功能的实现。除此以外，本系统在用户态的桩代码实现上进行了简化。具体如下：

在xv6下，系统调用的桩代码usys原本为一个usys.pl文件：

```
print "# generated by usys.pl - do not edit\n";

print "#include \"kernel/syscall.h\"\n";

sub entry {
    my $name = shift;
    print ".global $name\n";
    print "${name}:\n";
    print " li a7, SYS_${name}\n";
    print " ecall\n";
    print " ret\n";
}

entry("fork");
entry("exit");
entry("wait");
entry("pipe");
entry("read");
entry("write");
entry("close");

.....
```

该代码定义了entry函数，并在之后调用了众多entry函数，从而生成了各个系统调用函数入口：将寄存器a7的值设为系统调用号并执行ecall指令。

而在本系统调用中，系统调用的启动被转化为各个宏定义文件，具体如下：

在sys.h文件中，定义了基本系统调用的宏定义，并且根据参数的不同，调用不同的系统调用辅助函数：

```

#ifndef __SYSCALL_H__
#define __SYSCALL_H__

#include "syscall_arch.h"
#include "syscall_num.h"

#ifndef __scc
#define __scc(X) ((long)(X))
typedef long syscall_arg_t;
#endif

#define __syscall1(n, a) __syscall1(n, __scc(a))
#define __syscall2(n, a, b) __syscall2(n, __scc(a), __scc(b))
#define __syscall3(n, a, b, c) __syscall3(n, __scc(a), __scc(b), __scc(c))
#define __syscall4(n, a, b, c, d) __syscall4(n, __scc(a), __scc(b), __scc(c),
__scc(d))
#define __syscall5(n, a, b, c, d, e) __syscall5(n, __scc(a), __scc(b), __scc(c),
__scc(d), __scc(e))
#define __syscall6(n, a, b, c, d, e, f) __syscall6(n, __scc(a), __scc(b),
__scc(c), __scc(d), __scc(e), __scc(f))

#define __SYSCALL_NARGS_X(a, b, c, d, e, f, g, h, n, ...) n
#define __SYSCALL_NARGS(...) __SYSCALL_NARGS_X(__VA_ARGS__, 7, 6, 5, 4, 3, 2, 1,
0, )
#define __SYSCALL_CONCAT_X(a, b) a##b
#define __SYSCALL_CONCAT(a, b) __SYSCALL_CONCAT_X(a, b)
#define __SYSCALL_DISP(b, ...) \
    __SYSCALL_CONCAT(b, __SYSCALL_NARGS(__VA_ARGS__)) \
    (__VA_ARGS__)

#define __syscall(...) __SYSCALL_DISP(__syscall, __VA_ARGS__)
#define syscall(...) __syscall(__VA_ARGS__)

#endif // __SYSCALL_H

```

而在syscall_arch文件中，定义了不同参数数量的系统调用函数，这个函数将参数保存并且执行ecall指令：

```

#define __SYSCALL_LL_E(x) (x)
#define __SYSCALL_LL_O(x) (x)

#define __asm_syscall(...) \
    __asm__ __volatile__( "ecall\n\t" \
        : "=r"(a0) \
        : __VA_ARGS__ \
        : "memory"); \
    return a0;

static inline long __syscall0(long n)
{
    register long a7 __asm__("a7") = n;
    register long a0 __asm__("a0");

```



```

    __asm_syscall("r"(a7))
}

static inline long __syscall1(long n, long a)
{
    register long a7 __asm__("a7") = n;
    register long a0 __asm__("a0") = a;
    __asm_syscall("r"(a7), "0"(a0))
}

// ...

```

相比于xv6，该系统避免了每个系统调用都得编写对应桩文件的函数的情形。这样每次需要新增系统调用时仅需扩展系统调用号并实现对应函数即可。该改变使得系统调用功能具有更好的扩展性。

2. 实验过程部分

2.1 实现步骤记录

2.1.1 设计自己的陷阱处理框架

参考xv6中有关于用户陷阱处理函数usertrap和userret，编写对应的用户态陷阱处理函数trap_user_handler和trap_user_return。前者主要用于在用户态陷阱处理以及函数调用的识别和调用，后者主要用户在用户态陷阱结束后返回用户态。由于本系统没有xv6中的如devintr函数，所以实现略有不同。具体函数如下：

```

void trap_user_handler()
{
    uint64 sepc = r_sepc();           // 记录了发生异常时的pc值
    uint64 sstatus = r_sstatus();     // 与特权模式和中断相关的状态信息
    uint64 scause = r_scause();       // 引发trap的原因
    uint64 stval = r_stval();         // 发生trap时保存的附加信息(不同trap不一样)
    proc_t* p = myproc();

    // 确认trap来自U-mode
    assert((sstatus & SSTATUS_SPP) == 0, "trap_user_handler: not from u-mode");

    // send interrupts and exceptions to kerneltrap(),
    // since we're now in the kernel.
    w_stvec((uint64)kernel_vector);

    // save user program counter.
    p->tf->epc = sepc;

    // 针对scause制定的一系列规则
    int trap_id = scause & 0xf;
    bool isInterrupt = ((scause & ((uint64)1 << 63)) != 0);
    char* info = isInterrupt ? interrupt_info[trap_id] : exception_info[trap_id];

    if(scause == 8){
        // system call
    }
}

```

```
// if(killed(p)) exit(-1);

// sepc points to the ecalls instruction,
// but we want to return to the next instruction.
p->tf->epc += 4;
printf("get a syscall from proc %d\n", p->pid);

syscall();

// an interrupt will change sepc, scause, and sstatus,
// so enable only now that we're done with those registers.
intr_on();

} else if (isInterrupt) {
    switch (trap_id)
    {
        case 1:
            // 处理软件中断
            timer_interrupt_handler();
            break;
        case 5:
            // 处理时钟中断
            // Pass anyway...
            break;
        case 9:
            // 处理外部中断
            external_interrupt_handler();
            break;
        default:
            printf("usertrap(): unexpected interrupt at pid=%d\n", p->pid);
            printf("          trap id: %d trap info: %s\n", trap_id, info);
            printf("          scause %p\n", scause);
            printf("          sepc=%p stval=%p\n", sepc, stval);
            panic("usertrap: unexpected interrupt");
            break;
    }
} else {
    // 异常处理
    printf("usertrap(): exception at pid=%d\n", p->pid);
    printf("          trap id: %d trap info: %s\n", trap_id, info);
    printf("          scause %p\n", scause);
    printf("          sepc=%p stval=%p\n", sepc, stval);
    panic("usertrap: unexpected exception");
    // setkilled(p);
}

trap_user_return();
}
```

```
void trap_user_return()
{
```

```

    struct proc *p = myproc();

    // we're about to switch the destination of traps from
    // kerneltrap() to usertrap(), so turn off interrupts until
    // we're back in user space, where usertrap() is correct.
    intr_off();

    // send syscalls, interrupts, and exceptions to uservec in trampoline.S
    uint64 trampoline_uservec = TRAMPOLINE + (user_vector - trampoline);
    w_stvec(trampoline_uservec);

    // set up trapframe values that uservec will need when
    // the process next traps into the kernel.
    p->tf->kernel_satp = r_satp();           // kernel page table
    p->tf->kernel_sp = p->kstack + PGSIZE; // process's kernel stack
    p->tf->kernel_trap = (uint64)trap_user_handler;
    p->tf->kernel_hartid = r_tp();           // hartid for cpuid()

    // set up the registers that trampoline.S's sret will use
    // to get to user space.

    // set S Previous Privilege mode to User.
    unsigned long x = r_sstatus();
    x &= ~SSTATUS_SPP; // clear SPP to 0 for user mode
    x |= SSTATUS_SPIE; // enable interrupts in user mode
    w_sstatus(x);

    // set S Exception Program Counter to the saved user pc.
    w_sepc(p->tf->epc);

    // tell trampoline.S the user page table to switch to.
    uint64 satp = MAKE_SATP(p->pgtbl);

    // jump to userret in trampoline.S at the top of memory, which
    // switches to the user page table, restores user registers,
    // and switches to user mode with sret.
    uint64 trampoline_userret = TRAMPOLINE + (user_return - trampoline);
    ((void (*)(uint64, uint64))trampoline_userret)(TRAPFRAME, satp);
}

```

2.1.2 理解系统调用的完整过程

接下来，对于xv6，理解其系统调用的完整流程。其详细机制已经在1.1.2节中讲述，这里仅作为略微的补充：

对于特权级切换的补充：

- 对于用户栈和内核栈，其分别在各自的页表中映射，所以切换页表就相当于切换栈。
- 对于寄存器状态的保存和恢复，在正常的上下文切换中，使用context实现，而对于用户态陷阱，则使用trapframe进行实现
- 对于页表的切换，在trampoline.S中，在跳转到trap_user_handler()之前，汇编代码将内核页表载入寄存器并切换，而在trap_user_return()函数的末尾，页表转为用户态页表，从而恢复执行程序。

为什么需要陷阱帧(trapframe)?

陷阱帧的作用如下：

- 保存完整的用户态执行上下文：在RISC-V中，寄存器的数量是有限的，当产生中断时，需要保存上下文相关的寄存器，该陷阱帧很好的保存个对应的寄存器并且能够恢复
- 实现安全隔离：xv6利用RISC-V的sscratch寄存器来实现一种安全的“原子交换”操作，从而可靠地获取内核栈指针。这样就可以使得内核和用户态相互分离，系统调用的详细过程不影响用户态下函数的执行。
- 标准化参数传递：陷阱帧作为一个标准化的数据结构，为内核提供了访问用户态参数的统一窗口。从而使得如访问a7等寄存器变得容易

系统调用和中断处理有什么相同和不同？

- 相同点：在系统中均作为“陷阱”进行处理，他们共享同一套底层框架，采用大致相同的处理过程
- 不同点：在陷阱处理程序中，系统调用和一般的中断处理被区分开（由于scause寄存器的值不同），系统调用通过调用syscall函数，从而调用对应系统功能调用；而中断处理直接根据不同的寄存器值直接调用对应的中断处理程序从而处理中断。

2.1.3 分析xv6的系统调用分发机制

接下来，分析xv6的系统调用分发机制。首先，研读syscall.c中的核心分发逻辑：

```
void
syscall(void)
{
    int num;
    struct proc *p = myproc();

    num = p->trapframe->a7;
    if(num > 0 && num < NELEM(syscalls) && syscalls[num]) {
        // Use num to lookup the system call function for num, call it,
        // and store its return value in p->trapframe->a0
        p->trapframe->a0 = syscalls[num]();
    } else {
        printf("%d %s: unknown sys call %d\n",
            p->pid, p->name, num);
        p->trapframe->a0 = -1;
    }
}
```

可以知道，系统调用号保存在陷阱帧的a7寄存器中，而系统调用的返回值保存在陷阱帧的a0寄存器中。当系统调用号超出范围时，输出警告信息，并且将返回值设为-1

其次，分析参数提取函数，如下所示：

```
static uint64
argraw(int n)
{
    struct proc *p = myproc();
```

```
switch (n) {
case 0:
    return p->trapframe->a0;
case 1:
    return p->trapframe->a1;
case 2:
    return p->trapframe->a2;
case 3:
    return p->trapframe->a3;
case 4:
    return p->trapframe->a4;
case 5:
    return p->trapframe->a5;
}
panic("argraw");
return -1;
}

void
argint(int n, int *ip)
{
    *ip = argraw(n);
}

void
argaddr(int n, uint64 *ip)
{
    *ip = argraw(n);
}

int
argstr(int n, char *buf, int max)
{
    uint64 addr;
    argaddr(n, &addr);
    return fetchstr(addr, buf, max);
}
```

可以知道，参数是从当前进程的陷阱帧中获取的（由argraw作为获取函数，其他函数通过调用该函数从而实现获取），对于不同类型的参数，该系统采用地址传输的策略（对于指针还涉及将用户态下的内存导入到内核态）。

对于argstr函数，其对于边界进行了检查，如果拷贝正常则返回字符串长度，否则返回-1.字符串的拷贝使用fetchstr进行控制：

```
fetchstr(uint64 addr, char *buf, int max)
{
    struct proc *p = myproc();
    if(copyinstr(p->pagetable, buf, addr, max) < 0)
        return -1;
}
```

```
    return strlen(buf);  
}
```

接下来, 理解用户内存的访问 :

```
// Copy from kernel to user.  
// Copy len bytes from src to virtual address dstva in a given page table.  
// Return 0 on success, -1 on error.  
int  
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)  
{  
    uint64 n, va0, pa0;  
  
    while(len > 0){  
        va0 = PGROUNDDOWN(dstva);  
        pa0 = walkaddr(pagetable, va0);  
        if(pa0 == 0)  
            return -1;  
        n = PGSIZE - (dstva - va0);  
        if(n > len)  
            n = len;  
        memmove((void *)(pa0 + (dstva - va0)), src, n);  
  
        len -= n;  
        src += n;  
        dstva = va0 + PGSIZE;  
    }  
    return 0;  
}  
  
// Copy from user to kernel.  
// Copy len bytes to dst from virtual address srcva in a given page table.  
// Return 0 on success, -1 on error.  
int  
copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)  
{  
    uint64 n, va0, pa0;  
  
    while(len > 0){  
        va0 = PGROUNDDOWN(srcva);  
        pa0 = walkaddr(pagetable, va0);  
        if(pa0 == 0)  
            return -1;  
        n = PGSIZE - (srcva - va0);  
        if(n > len)  
            n = len;  
        memmove(dst, (void *)(pa0 + (srcva - va0)), n);  
  
        len -= n;  
        dst += n;  
        srcva = va0 + PGSIZE;  
    }  
}
```

```
    return 0;
}
```

这两个函数主要是用户将用户态的内存或者内核态的内存进行拷贝转换。为了保证用户态和内核态相分离，用户态和内核态采用的是不同的页表，所以某个状态下无法直接访问另一个状态的内存，必须经过直接拷贝。为了防止用户指针恶意传递，此处会检查是否合法。

2.1.4 设计自己的系统调用框架

在了解xv6的系统调用流程之后，进行系统调用的实现。

首先，实现系统调用表和设定系统号对应，此部分已经在1.2.1节讲述过，这里不再赘述。

其次，实现系统调用分发器，本系统系统分发器和xv6相同，不再赘述

然后，实现参数提取辅助函数如下：

```
// 读取 n 号参数，作为正常的 int 存储
void arg_int(int n, int* ip)
{
    *ip = arg_raw(n);
}

// 读取 n 号参数，作为 uint32 存储
void arg_uint32(int n, uint32* ip)
{
    *ip = arg_raw(n);
}

// 读取 n 号参数，作为 uint64 存储
void arg_uint64(int n, uint64* ip)
{
    *ip = arg_raw(n);
}

// 读取 n 号参数指向的字符串到 buf，字符串最大长度是 maxlen
void arg_str(int n, char* buf, int maxlen)
{
    proc_t* p = myproc();
    uint64 addr;
    arg_uint64(n, &addr);

    uvm_copyin_str(p->pgtbl, (uint64)buf, addr, maxlen);
}
```

至于系统调用详细的框架，请看源代码部分。

2.1.5 设计自己的系统调用

接下来，实现自己的系统调用。

由于当前进程只是单进程，所以能够实现的系统调用功能十分有限。具体包含空系统调用，输出信息系统调用，修改堆大小系统调用等。实现如下：

```
// 系统调用测试（无功能）
// 输出一行调试信息，然后返回0
uint64 sys_test()
{
    printf("System call test here.\n");
    return 0;
}

// 输出对应的信息
// char* message 需要输出的信息（无格式）
// 输出message，之后返回0
uint64 sys_print()
{
    char* message = "";
    arg_str(0, message, 256);
    printf(message);
    return 0;
}

// 堆伸缩
// uint64 new_heap_top 新的堆顶（如果是0代表查询，返回旧的堆顶）
// 成功返回新的堆顶 失败返回-1
uint64 sys_brk()
{
    uint64 new_heap_top;
    uint64 old_heap_top;
    uint64 res;
    pgtbl_t pgtbl = myproc()->pgtbl;

    arg_uint64(0, &new_heap_top);
    old_heap_top = myproc()->heap_top;

    if(new_heap_top == 0){
        // printf("堆未改变，堆查询结果: %x\n", old_heap_top);
        return old_heap_top;
    }
    else if(new_heap_top > old_heap_top){
        // 需要增大堆空间
        res = uvm_heap_grow(pgtbl, old_heap_top, new_heap_top - old_heap_top);
        // printf("增大的新堆: %x\n", res);
        return res;
    }
    else{
        // 需要缩小堆空间
        res = uvm_heap_ungrow(pgtbl, old_heap_top, old_heap_top - new_heap_top);
        // printf("缩小的新堆: %x\n", res);
        return res;
    }
}
```


2.1.6 实现用户态系统调用接口

首先，参考xv6的usys.pl，了解并实现桩代码生成机制。

这部分的代码已经在1.4节中讲述，这里不再赘述

其次，考虑用户库中的系统调用声明，在实际实现的时候，为了简便，仅实现了如下的函数调用。在未来可能会新增函数包装：

```
syscall(SYS_print, "\nTesting blank system call:\n");
syscall(SYS_test);
```

2.2 问题与解决方案

2.2.1 自己遇见的问题

在实现过程中，可能遇见了部分问题：

系统调用报错

解决方案：按照系统调用流程逐个检查系统调用的机制，从而查找对应的原因。

2.3 源码理解总结

xv6在用户态（U模式）下的陷阱处理与内核态（S模式）下的陷阱处理架构相类似，其主要通过中断委托机制将陷阱交给S模式处理，在S模型中进行中断的处理和类型的识别。

系统调用和中断的处理流程大致相同，但是其调用了syscall()实现了系统调用函数的分配和实现。不同的系统调用函数实现了不同的功能，但是其均使用arg系列函数从trapframe中获取参数，返回值存放在trapframe的a0（和a1）中。

3. 测试验证部分

3.1 功能测试

存放在initcode.c的测试代码如下：

```
#include "sys.h"

int main()
{
    syscall(SYS_test);

    long long heap_top = syscall(SYS_brk, 0);

    heap_top = syscall(SYS_brk, heap_top + 4096 * 10);
```

```
    heap_top = syscall(SYS_brk, heap_top - 4096 * 5);  
  
    while(1);  
    return 0;  
}
```

启动项目，其结果如下：

```
riptide@admin:~/whu-oslab$ make clean && make qemu  
art.o ./lib/print.o ./lib/spinlock.o ./lib/str.o ./mem/kvm.o ./mem/mmap.o ./mem/pmem.o ./mem/uvm.o ./proc/  
yscall.o ./syscall/sysfunc.o ./trap/trampoline.o ./trap/trap.o ./trap/trap_kernel.o ./trap/trap_user.o  
riscv64-linux-gnu-ld: warning: cannot find entry symbol _entry; defaulting to 0000000080000000  
make[1]: Leaving directory '/home/riptide/whu-oslab/kernel'  
qemu-system-riscv64 -machine virt -bios none -kernel kernel-qemu -m 128M -smp 2 -nographic  
System call test here.  
堆未改变，堆查询结果: -7fffa6f0  
增大的新堆: -7fff0000  
缩小的新堆: -7fff5000  
QEMU: Terminated
```

说明其运行功能正常。

4. 结论与心得

通过这次实验，我得知了系统调用的主要流程，并且实现了一些系统调用。在未来，本系统将会实现更大的系统调用扩展。

实验 6：多进程和系统调用扩展

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

0. 实验任务和目标

1. 进一步分析xv6的进程管理机制，理解操作系统如何创建和管理进程，实现完整的进程声明周期管理和简单的调度算法，从而实现多进程进行。
2. 利用多进程的功能，进一步扩充实现常用系统调用功能，例如`SYS_fork`，`SYS_exit`等。

1. 系统设计

1.1 架构设计说明

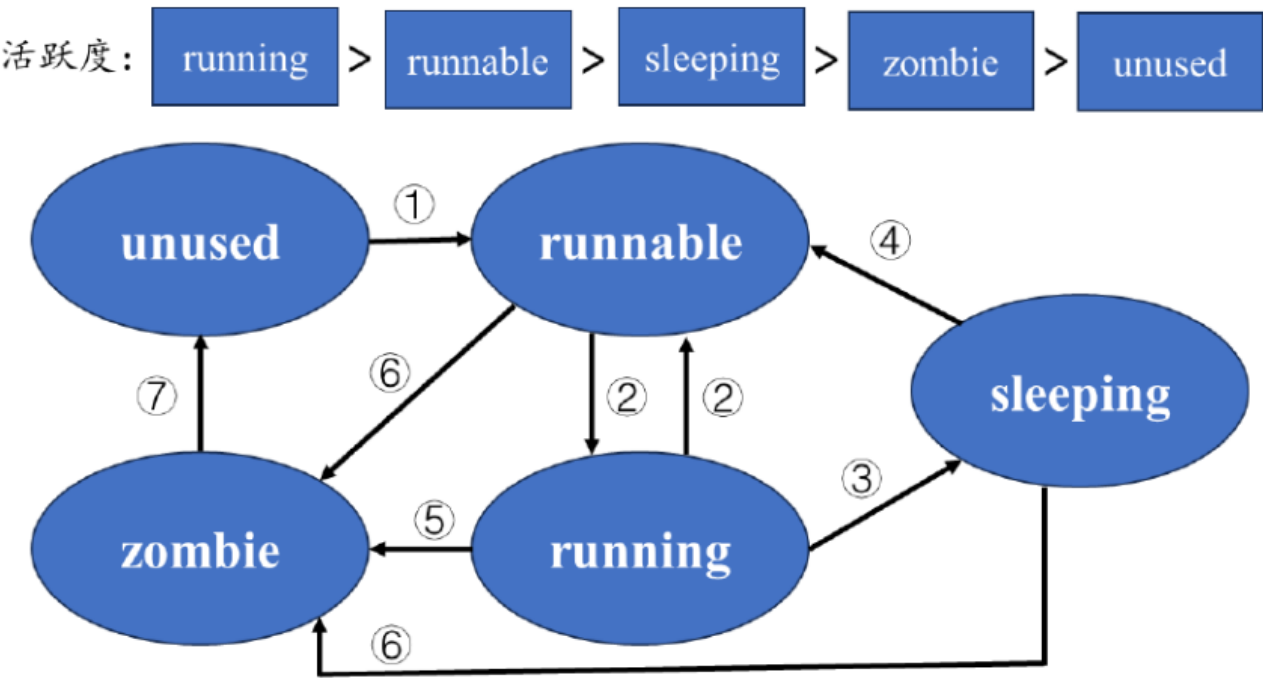
由于进程的基本状态，系统调用的流程等在先前的实验报告中已有讲述，所以在这里不再赘述，详情请看实验四、五的内容

1.1.1 进程状态及其转换

本操作系统的进程状态有如下几种：

- `unused`：表示进程尚未被使用，且未被分配部内存空间
- `runnable`：进程已经创建且分配对应的空间，等待CPU调度进行
- `running`：CPU正在运行该进程
- `sleeping`：该进程由于部分原因处于睡眠状态，等待重新唤醒
- `zombie`：本进程已被退出，即将被回收

进程状态的转化图如下：



其中：

- 1. 空闲进程结构体被取出和初始化
- 2. 进程调度：从可运行进程中选出一个换掉正在执行的进程
- 3. 进程等待某种资源或某个事件，进入睡眠状态
- 4. 进程得到所等待的东西后被唤醒
- 5. 进程执行完毕，主动杀死自己
- 6. 处于可执行状态或睡眠状态的进程被正在执行的进程杀死
- 7. 进程的父进程处理后事，释放濒死进程的资源

通过设置进程的状态，即可实现不同的进程操作。

1.1.2 多进程调度方法

与xv6相同，本操作系统使用抢占式时间片轮转调度（RR）算法进行调度。它将 CPU 时间划分为固定长度的时间片（通常为 10~100ms），并按循环队列的顺序为每个就绪进程分配一个时间片执行。若进程在时间片内未完成，则会被抢占并放到就绪队列尾部，等待下一轮调度。

具体对于实现而言，本操作系统与xv6相同。CPU核心会不断从进程表中循环查找可以运行的进程，并在找到可运行进程（runnable）之后运行该进程。同时，每过一个时钟中断（按照当前设置，约0.1s），系统调用yield函数，强制放下当前正在进行的进程，从而重新回到查找可运行进程的过程。

这种方法在交互式系统中表现优异，能保证所有进程及时响应，但在计算密集型任务中可能导致平均等待时间较长。

1.2 关键数据结构

1.2.1 改进后的进程proc

相比实验四，由于多进程调度的存在，所以需要扩充进程proc的结构体：

```
enum proc_state {
    UNUSED,        // 未被使用
    RUNNABLE,      // 准备就绪
    RUNNING,       // 运行中
    SLEEPING,      // 睡眠等待
    ZOMBIE,        // 濒临死亡
};

// 进程定义
typedef struct proc {
    spinlock_t lk;        // 自旋锁

    // 使用前需要持有p->lock才能访问
    int pid;              // 标识符
    enum proc_state state; // 进程状态
    void* sleep_space;    // 睡眠是为在等待什么
    int xstate;           // 进程退出时的状态
    bool killed;          // 是否已被杀死（功能有待扩展）

    // 使用前需要持有wait_lock才能访问
    struct proc *parent;  // 父进程

    // 每个进程独立的数据，访问时无需上锁
    pgtbl_t pgtbl;        // 用户态页表
    uint64 heap_top;      // 用户堆顶(以字节为单位)
    uint64 ystack_pages;  // 用户栈占用的页面数量
    trapframe_t* tf;      // 用户态内核态切换时的运行环境暂存空间

    uint64 kstack;        // 内核栈的虚拟地址
    context_t ctx;        // 内核态进程上下文
} proc_t;
```

可以看到，相比实验四中的定义，其增加了进程状态以及相关的内容，同时采用自旋锁的方式去维护这些变量。

而对于多进程的存储，本系统使用一个数组去存储这些进程：

```
proc_t proc[NPROC];    // 数组
```

1.3 设计决策理由

首先，为了实现多进程，必须对进程的状态进行管理和调度，这样方可使得进程的运行有序。

其次，本系统采用一个进程数组去存储进程，这样使得进程的分配调度显得有序，且这样的存储方式，使得时间片分布算法变得容易实现。

1.4 与 xv6 对比分析

在进程的状态分析上，本系统相比xv6少了一个状态`unused`，这样一定程度简化了系统进程状态修改的复杂程度。同时当系统回收时将从`zombie`修改为`unused`，也体现回收更具体。

本系统使用和XV6相同的进程调度算法，使用时间片调度算法，且在每个时钟中断切换进程，这样不仅容易实现，同时也保证了在多进程调度时系统的公平性，使得系统能够即使响应。

在扩展系统调用上，情况与第五次实验报告展示的类似，只不过对应的系统功能调用号和系统调用实现函数有新的改变与扩充。

2. 实验过程部分

2.1 实现步骤记录

2.1.1 分析并实现进程创建机制

接下来，阅读xv6代码，分析xv6的进程创建机制。

首先，研读`allocproc`函数，其代码如下：

```
static struct proc*
allocproc(void)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        acquire(&p->lock);
        if(p->state == UNUSED) {
            goto found;
        } else {
            release(&p->lock);
        }
    }
    return 0;

found:
    p->pid = allocpid();
    p->state = USED;

    // Allocate a trapframe page.
    if((p->trapframe = (struct trapframe *)kalloc()) == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }

    // An empty user page table.
    p->pagetable = proc_pagetable(p);
    if(p->pagetable == 0){
        freeproc(p);
        release(&p->lock);
        return 0;
    }
```

```

    }

    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->context, 0, sizeof(p->context));
    p->context.ra = (uint64)forkret;
    p->context.sp = p->kstack + PGSIZE;

    return p;
}

```

由代码可知，该函数逐个遍历进程表，直至找到状态为`unused`的进程或者遍历完成但未找到。如果找到进程的空闲槽位，则使用`allocpid`函数为这个新的进程设置ID号。接着为这个子进程分配页表，并为用户栈和陷阱帧分配空间，之后调整上下文，使得`ra`为`forkret`，`sp`为`p->kstack + PGSIZE`

其次，阅读`fork`函数，了解进程是如何分支出新的进程。代码如下：

```

int
fork(void)
{
    int i, pid;
    struct proc *np;
    struct proc *p = myproc();

    // Allocate process.
    if((np = allocproc()) == 0){
        return -1;
    }

    // Copy user memory from parent to child.
    if(uvmcopy(p->pagetable, np->pagetable, p->sz) < 0){
        freeproc(np);
        release(&np->lock);
        return -1;
    }
    np->sz = p->sz;

    // copy saved user registers.
    *(np->trapframe) = *(p->trapframe);

    // Cause fork to return 0 in the child.
    np->trapframe->a0 = 0;

    // increment reference counts on open file descriptors.
    for(i = 0; i < NOFILE; i++)
        if(p->ofile[i])
            np->ofile[i] = filedup(p->ofile[i]);
    np->cwd = idup(p->cwd);

    safestrcpy(np->name, p->name, sizeof(p->name));
}

```

```

pid = np->pid;

release(&np->lock);

acquire(&wait_lock);
np->parent = p;
release(&wait_lock);

acquire(&np->lock);
np->state = RUNNABLE;
release(&np->lock);

return pid;
}

```

这个函数首先分配了新进程结构，然后复制内存和陷阱帧到子进程中，之后设置返回值，然后标记为 runnable。为了分辨父子进程，需要有不同的返回值（父进程返回子进程的id，子进程返回0）；内存复制依靠调用 `uvmcopy` 函数从而实现，当失败时系统将会收回原本分配的资源，防止资源泄露。

然后，分析进程的等待和退出机制，阅读 `wait` 函数以及 `exit` 函数如下：

```

void
exit(int status)
{
    struct proc *p = myproc();

    if(p == initproc)
        panic("init exiting");

    // Close all open files.
    for(int fd = 0; fd < NOFILE; fd++){
        if(p->ofile[fd]){
            struct file *f = p->ofile[fd];
            fileclose(f);
            p->ofile[fd] = 0;
        }
    }

    begin_op();
    iput(p->cwd);
    end_op();
    p->cwd = 0;

    acquire(&wait_lock);

    // Give any children to init.
    reparent(p);

    // Parent might be sleeping in wait().
    wakeup(p->parent);
}

```



```
    acquire(&p->lock);

    p->xstate = status;
    p->state = ZOMBIE;

    release(&wait_lock);

    // Jump into the scheduler, never to return.
    sched();
    panic("zombie exit");
}

int
wait(uint64 addr)
{
    struct proc *pp;
    int havekids, pid;
    struct proc *p = myproc();

    acquire(&wait_lock);

    for(;;){
        // Scan through table looking for exited children.
        havekids = 0;
        for(pp = proc; pp < &proc[NPROC]; pp++){
            if(pp->parent == p){
                // make sure the child isn't still in exit() or swtch().
                acquire(&pp->lock);

                havekids = 1;
                if(pp->state == ZOMBIE){
                    // Found one.
                    pid = pp->pid;
                    if(addr != 0 && copyout(p->pagetable, addr, (char *)&pp->xstate,
                                            sizeof(pp->xstate)) < 0) {
                        release(&pp->lock);
                        release(&wait_lock);
                        return -1;
                    }
                    freeproc(pp);
                    release(&pp->lock);
                    release(&wait_lock);
                    return pid;
                }
                release(&pp->lock);
            }
        }
    }

    // No point waiting if we don't have any children.
    if(!havekids || killed(p)){
        release(&wait_lock);
        return -1;
    }
}
```

```

    // Wait for a child to exit.
    sleep(p, &wait_lock); //DOC: wait-sleep
}
}

```

在这里，`wait`函数寻找子进程并且主动等待子进程运行完成（如ZOMBIE状态），而`exit`函数收起进程的资源，然后关闭进程并提醒父进程。这种协作关系保证了操作系统中进程的有序管理。

2.1.2 设计并实现进程管理系统

接下来，讲基于上述讲述到的进程管理机制，进行进程管理系统的设置。

例如，进程分配函数`allocproc`代码如下：

```

// 创建一个进程
proc_t* proc_alloc(void)
{
    proc_t *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        spinlock_acquire(&p->lk);
        if(p->state == UNUSED) {
            goto found;
        } else {
            spinlock_release(&p->lk);
        }
    }
    return 0;

found:
    p->pid = allocpid();
    p->state = RUNNABLE;

    // Allocate a trapframe page.
    if((p->tf = (struct trapframe *)pmem_alloc(false)) == 0){
        proc_free(p);
        spinlock_release(&p->lk);
        return 0;
    }

    // An empty user page table.
    p->pgtbl = proc_pgtbl_init((uint64)p->tf);
    if(p->pgtbl == 0){
        proc_free(p);
        spinlock_release(&p->lk);
        return 0;
    }

    // Set up new context to start executing at forkret,
    // which returns to user space.
    memset(&p->ctx, 0, sizeof(p->ctx));

```

```
p->ctx.ra = (uint64)fork_return;
p->ctx.sp = p->kstack + PGSIZE;

return p;
}
```

同时，由于现在有一整个进程表而不是单个进程，所以考虑将首个进程生成函数`proc_make_first`简化如下：

```
void proc_make_first()
{
    struct proc *p;

    // 给其分配一个进程
    p = proc_alloc();
    proczero = p;

    // 重写id
    proczero->pid = 0;

    // ustack 映射 + 设置 ustack_pages
    // proc_mapstacks已经完成了栈的设置和映射
    proczero->kstack = KSTACK(0);
    uint64 ustack_phys = (uint64)pmem_alloc(true);
    vm_mappages(proczero->pgtbl, proczero->kstack - PGSIZE, ustack_phys, PGSIZE,
                PTE_R | PTE_W | PTE_U);

    // data + code 映射
    assert(initcode_len <= PGSIZE, "proc_make_first: initcode too big\n");
    char *mem = (char *)pmem_alloc(false);
    memset(mem, 0, PGSIZE);
    vm_mappages(proczero->pgtbl, 0, (uint64)mem, PGSIZE, PTE_W|PTE_R|PTE_X|PTE_U);
    memmove(mem, initcode, initcode_len);
    proczero->ustack_pages = 1;

    // 设置 heap_top
    proczero->heap_top = PGSIZE;

    // 设置用户态返回时的关键寄存器
    proczero->tf->epc = 0; // 程序计数器，从虚拟地址0开始执行initcode
    proczero->tf->sp = PGSIZE; // 用户栈指针，设置在用户空间顶部

    // 修改其状态并释放该锁
    proczero->state = RUNNABLE;
    spinlock_release(&proczero->lk);
}
```

本部分的进程生成，退出等代码类似于xv6，具体请看代码仓库中的代码，这里不再赘述。

2.1.3 分析并实现调度器

接下来，对于多进程，我们需要去实现具体的调度器。

首先，分析`scheduler()`函数：

```
void
scheduler(void)
{
    struct proc *p;
    struct cpu *c = mycpu();

    c->proc = 0;
    for(;;){
        // Avoid deadlock by ensuring that devices can interrupt.
        intr_on();

        for(p = proc; p < &proc[NPROC]; p++) {
            acquire(&p->lock);
            if(p->state == RUNNABLE) {
                // Switch to chosen process.  It is the process's job
                // to release its lock and then reacquire it
                // before jumping back to us.
                p->state = RUNNING;
                c->proc = p;
                swtch(&c->context, &p->context);

                // Process is done running for now.
                // It should have changed its p->state before coming back.
                c->proc = 0;
            }
            release(&p->lock);
        }
    }
}
```

这个函数的基本流程如下：

1. CPU进行初始化，将当前执行的进程设为空
2. CPU开始永远的遍历进程表，如果发现有一个状态为`RUNNABLE`的进程，就将其设为`RUNNING`，且将当前核心执行的进程设为它，并切换上下文，表示开始执行进程

通过如上的方式，CPU对于整个进程表不断进行遍历，从而实现进程的调度。为了防止长时间的长等待，xv6采用了抢占式时间片轮转调度（RR）从而进行系统调度。同时，为了防止系统进入死锁，该函数开启中断，从而使得外部设备能够进行中断。

其次，阅读`yield`函数，理解调度的时机：

```
void
yield(void)
{
    struct proc *p = myproc();
```

```

    acquire(&p->lock);
    p->state = RUNNABLE;
    sched();
    release(&p->lock);
}

void
sched(void)
{
    int intena;
    struct proc *p = myproc();

    if(!holding(&p->lock))
        panic("sched p->lock");
    if(mycpu()->noff != 1)
        panic("sched locks");
    if(p->state == RUNNING)
        panic("sched running");
    if(intr_get())
        panic("sched interruptible");

    intena = mycpu()->intena;
    swtch(&p->context, &mycpu()->context);
    mycpu()->intena = intena;
}

```

这个函数`yield`是一种抢占式调度。该函数运行时，其将当前进程的状态修改为`RUNNABLE`，并且调用`sched()`函数进行上下文切换，从而抢占式的暂停当前进程，使得核返回到scheduler中重新选择当前执行的进程。

如上`yield`函数主要放置在时钟中断的处理流程中，即每个时钟中断执行一次。例如如下函数，在这里`which_dev == 2`指的是时钟中断：

```

void
usertrap(void)
{
    // ...
    // give up the CPU if this is a timer interrupt.
    if(which_dev == 2)
        yield();
    // ...
}

```

参考如上代码，实现自己的调度器。本系统采用核xv6相同的调度器以及抢占式操作，所以代码部分略。详情请查看实验仓库中的代码。

注：除此以外，调度器`scheduler()`已经拥有死循环以及开中断的功能，所以为了开启调度，`main`函数可以去尾的死循环和`intr_on`，切换成`proc_scheduler()`开启调度流程，具体如下所示：

```
void main()
{
    int cpuid = r_tp();
    if(cpuid == 0){
        // 其他代码不变
        proc_init();
        // 其他代码不变
        proc_make_first();

        printf("cpu %d is booting!\n", cpuid);
        __sync_synchronize();
        started = 1;
    }
    else{
        // 其他代码不变
    }

    proc_scheduler();
}
```

2.1.4 实现进程同步原语

接下来，我们实现进程同步原语。基于xv6的sleep/wakeup机制，实现代码如下：

```
// 进程睡眠在sleep_space
void proc_sleep(void* sleep_space, spinlock_t* lk)
{
    struct proc *p = myproc();

    // Must acquire p->lk in order to
    // sleep_space p->state and then call sched.
    // Once we hold p->lk, we can be
    // guaranteed that we won't miss any wakeup
    // (wakeup locks p->lk),
    // so it's okay to release lk.

    spinlock_acquire(&p->lk); //DOC: sleeplock1
    spinlock_release(lk);

    // Go to sleep.
    p->sleep_space = sleep_space;
    p->state = SLEEPING;

    proc_sched();

    // Tidy up.
    p->sleep_space = 0;

    // Reacquire original lock.
    spinlock_release(&p->lk);
```

```

    spinlock_acquire(&lk);
}

// 唤醒所有在sleep_space沉睡的进程
void proc_wakeup(void* sleep_space)
{
    struct proc *p;

    for(p = proc; p < &proc[NPROC]; p++) {
        if(p != myproc()){
            spinlock_acquire(&p->lk);
            if(p->state == SLEEPING && p->sleep_space == sleep_space) {
                p->state = RUNNABLE;
            }
            spinlock_release(&p->lk);
        }
    }
}

// 唤醒一个进程
static void proc_wakeup_one(proc_t* p)
{
    assert(spinlock_holding(&p->lk) == 0, "proc_wakeup_one: lock");
    if(p->state == SLEEPING && p->sleep_space == p) {
        p->state = RUNNABLE;
    }
}

```

进程睡眠功能的实现主要是通过`proc_sleep`实现，该函数在获取相关锁（维护功能）之后，将该进程状态设为`SLEEPING`，并且设置`sleep_space`参数。并调用`proc_sched()`进行上下文切换。

对于进程唤醒，本系统有两个函数实现：`proc_wakeup`函数可以唤醒所有满足某个`sleep_space`的进程，而`proc_wakeup_one`函数可以唤醒某一个特定的进程。

2.1.5 扩展系统调用功能

最后，我们将所实现的进程管理包装成系统调用，使得用户态代码能够通过系统从而进行进程管理，使用相关功能。具体实现的系统功能调用如下：

```

static uint64 (*syscalls[])(void) = {
    // 其他的系统调用功能不变
    [SYS_fork]      sys_fork,
    [SYS_wait]      sys_wait,
    [SYS_exit]      sys_exit,
    [SYS_sleep]     sys_sleep,
    [SYS_kill]      sys_kill,
    [SYS_getpid]    sys_getpid,
};

```

如上大部分函数调用原来的系统功能即可，具体代码请看源代码仓库。另外`sys_sleep`调用的过程有所不同，具体实现如下：

```
uint64 sys_sleep()
{
    int n;
    uint64 ticks0;

    arg_int(0, &n);
    spinlock_acquire(&timer_get()->lk);
    ticks0 = timer_get_ticks();
    while(timer_get_ticks() - ticks0 < n){
        if(proc_killed(myproc())){
            spinlock_release(&timer_get()->lk);
            return -1;
        }
        proc_sleep(&timer_get()->ticks, &timer_get()->lk);
    }
    spinlock_release(&timer_get()->lk);
    return 0;
}
```

该函数的参数为睡眠的ticks数（一般是毫秒）。本函数通过不断的让系统进入睡眠状态并醒来，从而使得进程卡在该函数中直至获取的ticks数大于等于所需睡眠ticks数。如果发现该进程已被杀死，则返回-1。

为了保证进程睡眠后能够时刻唤醒检查ticks数，在本系统的时钟中断处理中添加了`proc_wakeup`以唤醒进程：

```
// 时钟中断处理（基于CLINT）
void timer_interrupt_handler()
{
    if(mycpuid() == 0){
        timer_update();
        proc_wakeup(&timer_get()->ticks);
    }
    // 通过清除SSIP位，承认软件中断
    w_sip(r_sip() & ~2);
}
```

2.2 问题与解决方案

2.2.1 自己遇见的问题

快速系统功能调用的问题：在用户态代码不断的进行输出相关系统功能调用时会出现报错：`panic: trap_kernel_handler: not from s-mode`

解决方案：需要在`trap_kernel_handler`的末尾增加如下代码，从而恢复寄存器，放置高频率调用使得输出冲突


```
// yield() 可能已经导致了一些陷阱的发生
// 所以为了kernelvec.S的sepc指令使用，需要恢复陷阱寄存器。
w_sepc(sepc);
w_sstatus(sstatus);
```

2.3 源码理解总结

为了实现多进程调用并且妥善的管理进程，系统使用一个进程数组维护所有可用的进程，每个进程有其对应的状态等，且不同状态可以相互切换，系统通过查看并边界不同进程的状态从而对于进程进行妥善的管理。

进程的功能有创建，分支，退出，等待等，不同的函数有不同的实现方法。在实现这些进程功能之后，我们可以将其包装成系统功能调用，从而在用户态代码中使用这些进程相关功能。

3. 测试验证部分

3.1 测试代码

本节为进程的测试以及系统调用提供了一系列测试，测试代码放在用户态代码中。具体代码如下：

```
#include "sys.h"

// 与内核保持一致
#define VA_MAX      (1ul << 38)
#define PGSIZE      4096
#define MMAP_END     (VA_MAX - 34 * PGSIZE)
#define MMAP_BEGIN   (MMAP_END - 8096 * PGSIZE)

char *str1, *str2;

int main()
{
    // 用户进程的开始
    syscall(SYS_print, "\nUser begin\n");

    // 空白系统调用测试
    syscall(SYS_print, "\nTesting blank system call:\n");
    syscall(SYS_test);

    // 进程号调用测试（基本）
    syscall(SYS_print, "\nTesting getting pid:\n");
    int id = syscall(SYS_getpid);
    char* str = "pid: _\n";
    str[5] = '0' + id;
    syscall(SYS_print, str);

    // 输出系统调用测试（输出所有非控制ascii字符，16个1行）
    syscall(SYS_print, "\nTesting printing ascii:\n");
    for(int i=0x20; i<0x7F; i++){
        char* str = "_ ";
```

```
    str[0] = i;
    syscall(SYS_print, str);
    if((i - 0x20) % 16 == 15){
        syscall(SYS_print, "\n");
    }
}
syscall(SYS_print, "\n");

// 测试HEAP区域
syscall(SYS_print, "\nTesting heap:\n");
long long top = syscall(SYS_brk, 0);
str2 = (char*)top;
syscall(SYS_brk, top + PGSIZE);
str2[0] = 'H';
str2[1] = 'E';
str2[2] = 'A';
str2[3] = 'P';
str2[4] = '\n';
str2[5] = '\0';
syscall(SYS_print, str2);

// 测试进程的fork, wait, exit
syscall(SYS_print, "\nTesting forking and waiting:\n");
int pid = syscall(SYS_fork);
if(pid == 0) { // 子进程
    for(int i = 0; i < 100000000; i++){
        syscall(SYS_print, "child: hello\n");
        syscall(SYS_exit, 1);
        syscall(SYS_print, "child: never back\n");
    }
} else { // 父进程
    int exit_state;
    int sonid = syscall(SYS_wait, &exit_state);
    if(sonid == pid && exit_state == 1)
        syscall(SYS_print, "parent: hello\n");
    else
        syscall(SYS_print, "parent: error\n");
}

// 测试进程的杀死
syscall(SYS_print, "\nTesting Killing:\n");
int sons = 5;
int pids[sons];
for(int i=0; i<sons; i++){
    int pid = syscall(SYS_fork);
    if(pid == 0){
        syscall(SYS_print, "child: hello\n");
        while(1) {}
    }
    else{
        pids[i] = pid;
    }
}
for(int i = 0; i < 100000000; i++){
    for(int i=0; i<sons; i++){
```

```
int res = syscall(SYS_kill, pids[i]);
if(res == 0){
    syscall(SYS_print, "parent: kill success\n");
}
else{
    syscall(SYS_print, "parent: kill failed\n");
}
}

// 测试进程的sleep
syscall(SYS_print, "\nTesting Sleeping:\n");
for(int i=0; i<10; i++){
    syscall(SYS_sleep, 10);
    syscall(SYS_print, "z");
}
syscall(SYS_print, "\nSlepped for 10s succeed.\n");

// 快速系统调用压力测试
syscall(SYS_print, "\nTesting Fast syscall:\n");
int times = 100000;
for(int i=0; i<times; i++){
    // syscall(SYS_test);
    syscall(SYS_sleep, 0);
    syscall(SYS_getpid);

    //char* c = (i % 10000 == 0) ? "#" : "";
    //syscall(SYS_print, c);
    syscall(SYS_print, ".");
}
syscall(SYS_print, "\nFast syscall test succeed.\n");

while(1);

return 0;
}
```

此处，所有的进程功能调试都是通过系统调用间接使用的。本测试代码包含如下测试：

- 空白系统功能调用测试：测试空白系统功能调用功能（即除了系统调用框架其他均无）是否正常
- 进程号调用测试：测试系统调用号是否能够获取
- 输出系统调用测试：通过输出所有非控制ascii字符，16个1行从而测试功能是否正常
- 堆区域测试：测试堆增大/减小的系统调用是否正常
- fork, wait, exit测试：测试这些功能是否正常
- 杀死测试：通过杀死5个死循环子进程从而测试功能是否正常
- sleep测试：通过测试系统睡眠从而判断功能是否正常
- 压力测试：通过快速重复进行系统调用，从而测试系统抗压能力

3.2 测试结果

如上代码编译并运行后，结果如下：

首先，对于功能测试，系统运行正常，各项功能正常运行。

```
cpu 0 is booting!

User begin

Testing blank system call:
System call test here.

Testing getting pid:
pid: 0

Testing printing ascii:
! " # $ % & ' ( ) * + , - . /
0 1 2 3 4 5 6 7 8 9 : ; < = > ?
@ A B C D E F G H I J K L M N O
P Q R S T U V W X Y Z [ \ ] ^ _
` a b c d e f g h i j k l m n o
p q r s t u v w x y z { | } ~

Testing heap:
HEAP

Testing forking and waiting:
child: hello
parent: hello

Testing Killing:
child: hello
child: hello
child: hello
child: hello
child: hello
parent: kill success
parent: kill success
parent: kill success
parent: kill success
parent: kill success

Testing Sleeping:
zzzzzzzzzz
Slept for 10s succeed.

Testing Fast syscall:
.....
.....
```

其次，在压力测试中，系统无报错，说明其确实能够处理高速重复系统调用的局面：

综上，说明系统功能测试成功。

通过此次实验，我从单进程扩展到了多进程，这使得我对于代码的印象加深了。

同时，修改代码的过程也使得我有更多耐心。这在我的未来中会帮助我克服困难。

实验7：文件系统

本实验报告为操作系统实践课的实验报告。

本实验所用源代码请看仓库：[Whu-OSlab仓库链接](#)

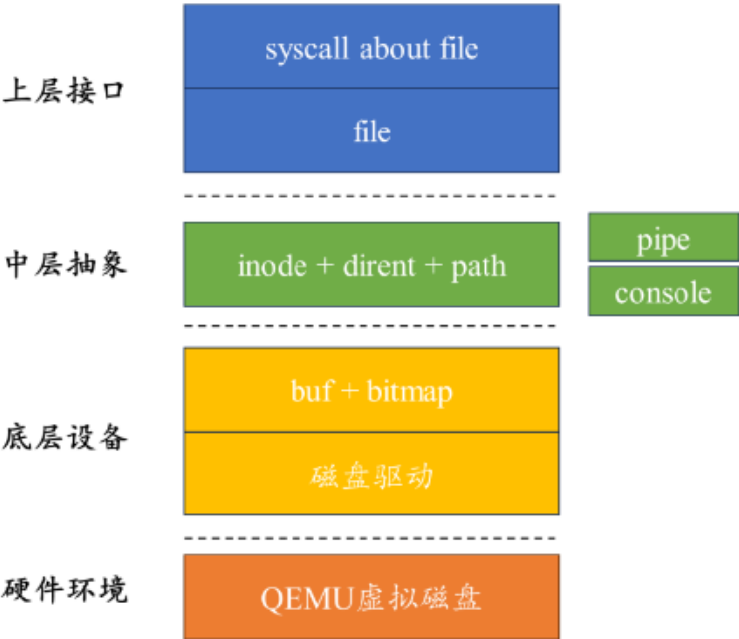
0. 实验任务和目标

通过深入分析xv6 的简化文件系统，理解现代文件系统的核心概念和实现原理，独立实现一个功能完整的文件系统。

1. 系统设计

1.1 架构设计说明

本实验的文件系统架构直接采用了与xv6相同的架构，其结构如下图所示：



其中包括：

- 硬件环境：使用QEMU的虚拟硬盘从而模拟磁盘，使用磁盘镜像fs.img存储硬盘内的文件
- 底层设备：使用bio.h中的硬件操作宏直接对硬盘进行读写，将读写结果存储在buf中。于此同时，通过编辑bitmap从而对于磁盘块的数据进行控制
- 中层抽象：使用inode用于磁盘块控制的抽象化，同时创建path从而实现对于路径的相关分析。
- 上层接口：使用file这个数据结构对于文件进行抽象，并提供读写相关功能，用户可以通过调用相关的系统调用从而对于文件进行创建，读写等。

1.2 关键数据结构

本实现的相关关键数据结构有：

超级块superblock

超级块用于操作系统初始化时魔数匹配，块数统计和控制等，其结构如下：

```
// 磁盘上的超级块结构
struct superblock {
    uint64 magic;           // 魔数，用于验证文件系统
    uint64 size;            // 文件系统大小（块数）
    uint64 nblocks;         // 数据块数量
    uint64 ninodes;         // inode 数量
    uint64 nlog;            // 日志块数量
    uint64 logstart;        // 日志起始块号
    uint64 inodestart;      // inode 区起始块号
    uint64 bmapstart;       // 位图起始块号
};
```

缓冲区buf

缓冲区主要用于将磁盘块内容暂存到内存，从而便于操作管理。采用双向链表对其内容进行缓存。其结构如下：

```
typedef struct buf {
    int valid;             // 缓冲区是否包含有效数据
    int disk;              // 缓冲区内容是否需要写回磁盘
    uint64 dev;            // 设备号
    uint64 blockno;        // 块号
    uint64 refcnt;         // 引用计数
    struct buf *prev;      // LRU 链表前驱
    struct buf *next;      // LRU 链表后继
    uint8 data[BSIZE];    // 数据
} buf_t;
```

inode

inode作为磁盘块的中层抽象，其实现了磁盘块的管理，读写，权限控制等。其结构如下：

```
struct inode {
    uint64 dev;            // 设备号
    uint64 inum;           // inode 号
    int ref;               // 引用计数
    int valid;             // inode 是否已从磁盘加载

    short type;            // 文件类型
    short major;           // 主设备号
    short minor;           // 次设备号
    short nlink;           // 硬链接数
    uint32 size;           // 文件大小（字节）
};
```

```
uint32_t addrs[NDIRECT + 1]; // 数据块地址数组
};
```

对于其具体结构，详情请看下文。

目录项dirent

目录项dirent主要用于管理文件的目录结构，其结构如下：

```
struct dirent {
    uint64_t inum; // inode 号
    char name[DIRSIZ]; // 文件名
};
```

文件File

文件File为文件系统中用户可以编辑的直接单元，其包括不同的类型，引用计数等。用户可以通过系统调用从而创建，读写这些文件。

```
struct File {
    enum EnumFileType type;
    int ref; // 引用计数
    char readable;
    char writable;
    char append; // 追加模式
    struct inode *ip; // 文件对应的 inode
    uint64_t off; // 文件偏移量
    short major; // 设备号（设备文件）
};
```

日志log

日志系统主要用于书写日志，从而保证磁盘输入输出的原子化，其结构如下：

```
struct logheader {
    int n;
    int block[LOGSIZE];
};

struct log {
    int dev;
    int start;
    int size;
    int outstanding; // 正在进行的文件系统操作数
    int committing; // 是否正在提交
    spinlock_t lock;
```



```
    struct logheader lh;  
} log_ctx;
```

1.3 设计决策理由

在本项目中，我们选择实现一个基于 xv6 设计理念的文件系统，因为它在简洁性、可靠性和教学价值上达到了绝佳的平衡。这一选择赋予了本系统一系列宝贵的特性：

- 简洁优雅: 继承了 Unix "一切皆文件" 的设计哲学，整体代码量小，逻辑清晰，便于快速掌握和二次开发。
- 分层设计: 系统自上而下分为系统调用、路径名、Inode、日志、缓冲区缓存和磁盘驱动等多个层次，各层职责分明，高度解耦，易于维护和学习。
- 基于 Inode: 以 Inode 作为文件系统的基本数据结构，高效地管理文件的元信息，并支持直接与间接寻址，以应对不同大小的文件。
- 崩溃安全: 引入了写前日志（Write-Ahead Log）机制。所有磁盘修改操作在写入最终位置前，都会先记录在日志中，从而保证了操作的原子性，确保了系统在断电等意外情况下的快速恢复和数据完整性。
- 性能优化: 通过在内存中开辟缓冲区缓存（Buffer Cache），缓存了频繁访问的磁盘块，大大降低了物理磁盘的读写次数，从而获得了显著的性能提升。

通过复现 xv6 这套成熟的方案，我们不仅在有限的开发周期内构建了一个功能完备的文件系统，更重要的是，确保了系统在数据管理层面的稳定性和鲁棒性。这种设计选择，使得上层应用的开发和数据存取都建立在一个坚实的基石之上。

1.4 与xv6对比分析

本实验所实现的文件系统基本上基于xv6，但是其有所不同：

- 将xv6中所有的数据类型修改为自己项目的数据类型
- 在file的读写实现中简化掉了除INODE种类以外的其他类型，一定程度上简化了代码结构
- 在文件系统初始化阶段增加了对于超级块的检查部分，从而易于调试

除此以外，在如下方面对于系统功能进行修改扩展：

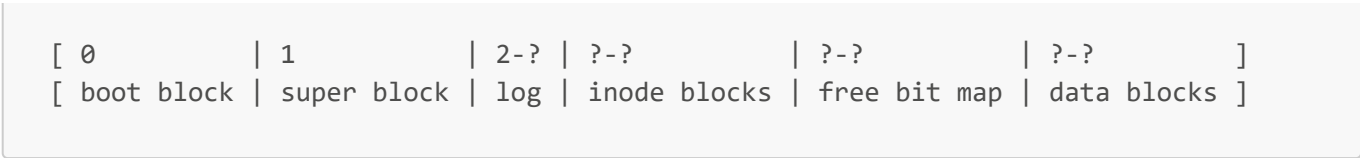
- 实现了第一个用户进程中代码部分的动态扩展，使得其能容纳在initcode中的更大的测试用例（在本实验的测试部分起到装载作用）
- 对于系统调用进行了相关的包装，从而一定程度上简化系统调用功能的填写，具体包括一般的包装（user.h）以及用户态printf实现（printf.h）

2. 实验过程部分

2.1 实现步骤记录

2.1.1 理解xv6文件系统布局

首先，分析磁盘布局结构。xv6的磁盘文件布局结构如下：



磁盘中各个区域的作用如下：

- **boot block**：启动块，它包含了引导加载程序（Bootloader），操作系统的启动代码。
- **super block**：即超级块，描述了文件系统的全局信息。没有它，操作系统就无法理解磁盘上的数据。
- **log**：用于存放日志。这是 xv6 实现崩溃恢复（Crash Recovery）的关键。文件系统操作（如创建文件、写入数据）在实际写入磁盘前，会先将操作记录在日志区。
- **inode blocks**：存储inode数据的磁盘块。它存储了文件的所有元数据，除了文件名和路径。
- **free bit map**：用来跟踪数据块 (Data Blocks) 的使用情况。位为1就是对应的块已分配，否则就是未分配
- **data blocks**：存放数据的磁盘块，用于存储文件的实际内容和目录的条目。

xv6 磁盘布局的顺序（引导块 -> 超级块 -> 日志 -> Inodes -> 位图 -> 数据块）并不是随意的，它遵循了性能、管理便利性和启动逻辑相结合的原则。我们可以将其看作一个“从元到实，从总到分”的过程：

1. 启动引导优先 (Bootstrapping First)
- **引导块 (Block 0)**：必须在磁盘的起始位置。这是硬件（BIOS/UEFI）的规定。计算机启动时，会固定地加载磁盘的第一个扇区并执行其中的代码。这个位置是雷打不动的，文件系统必须服从这个规则。
2. 元数据集中管理 (Centralized Metadata)
- **超级块 (Block 1)**：它是整个文件系统的“地图”和“说明书”。操作系统挂载文件系统时，第一件事就是读取超级块以了解文件系统的全局布局（总共有多少块？Inode 在哪里？等等）。把它放在一个固定的、紧随引导块的位置，使得 OS 能以最小的代价找到它。
 - **日志、Inodes、位图**：这三者都属于**元数据 (Metadata)**，即“关于数据的数据”。将它们物理上聚集在一起有两大好处：
 - **提高性能**：文件操作（如创建、删除、修改文件）常常需要同时修改多个元数据。例如，创建一个文件可能需要：分配一个新 Inode、修改位图（标记新数据块为已用）、修改父目录的数据块（添加新文件名和 Inode 号的对应关系）。如果这些元数据块在物理上彼此靠近，磁盘磁头移动的距离（寻道时间）就会大大缩短，从而显著提升 I/O 性能。这被称为**局部性原理 (Locality of Reference)** 的应用。
 - **管理简单**：将所有元数据放在磁盘的头部，数据放在后面，使得布局非常清晰。操作系统可以很容易地知道元数据区的范围，便于进行管理和维护。

3. 数据与元数据分离 (Separation of Metadata and Data)
- **数据块**：这是文件系统中最大、也是最“廉价”的部分。它只存储纯粹的内容。将它放在最后，使得前面的元数据区可以形成一个连续、紧凑的管理区域。

各部分的大小是在**创建文件系统时（格式化时，例如运行 `mkfs` 程序）** 确定的，并且这是一个充满了**权衡 (Trade-off)** 的过程。一旦确定，通常就无法再更改。

1. 引导块和超级块
- 大小固定，各占 **1 个块**。它们的结构简单且信息量固定，一个块足以容纳。

2. 日志区 (Log)

- 这是一个重要的**性能与安全性的权衡**。
- **如何确定**: xv6 中, 日志大小 (LOGSIZE) 是一个编译时常量。它的设定必须**大于等于任何一个原子系统调用可能修改的块的最大数量**。例如, create 系统调用最坏情况下可能修改: 1个位图块 + 1个inode块 + 1个父目录的数据块。所以日志大小必须至少能容纳这么多块的写操作。

3. Inode 区

- 这是最经典的权衡:**最大文件数量 vs. 可用数据空间**。
- Inode 的总数 (NINODES) 直接决定了你**最多能创建多少个文件和目录**。
- **如何确定**: mkfs 程序通常会根据磁盘总大小, 按照一个预设的“每 X KB 空间分配一个 Inode”的比例来计算 Inode 的总数。这个比例反映了设计者对“平均文件大小”的预期。

4. 位图区 (Bitmap)

- 它的大小是**被动确定的**, 而不是主动选择的。
- 位图需要用 1 个 bit 来代表 1 个数据块。因此, 位图的大小取决于数据块的总数。
- 计算公式: **位图大小 (块) = ceil(数据块总数 / 每块的位数)**。
- 在 xv6 的简化设计中, 它被假定为 **1 个块**, 这意味着一个块 (例如 4096 字节) 的位数 ($4096 * 8 = 32768$ 位) 必须足以覆盖所有的数据块。这也间接限制了 xv6 文件系统的最大大小。

5. 数据区 (Data Blocks)

- 它的大小也是**被动确定的**。
- 计算公式: **数据区大小 = 磁盘总大小 - 引导块大小 - 超级块大小 - 日志区大小 - Inode区大小 - 位图区大小**。
- 它是文件系统中最大的一部分, 是所有元数据最终服务的对象。

其次, 理解超级块的作用。超级块的结构如下:

```
struct superblock {
    uint64 magic;           // 魔数, 用于验证文件系统
    uint64 size;            // 文件系统大小 (块数)
    uint64 nblocks;         // 数据块数量
    uint64 ninodes;         // inode 数量
    uint64 nlog;            // 日志块数量
    uint64 logstart;        // 日志起始块号
    uint64 inodestart;      // inode 区起始块号
    uint64 bmapstart;       // 位图起始块号
};
```

超级块存放了各个部分的关键数据, 对于软件如何使用这些磁盘块数据具有指导作用, 其就像磁盘空间的“总清单和布局图”, 如果没有超级块中的这些元数据, 操作系统就像一个失忆的人面对一个塞满东西的仓库: 它能看到一堆东西 (磁盘上的二进制数据), 但完全不知道这些东西是什么、有多少、放在哪里、如何使用。

为了保持超级块的一致性, 其主要通过如下机制呈现:

- 几乎只读: 超级块的内容在文件系统被创建 (格式化) 后, 在正常的运行期间是几乎不变的。
- 通过日志系统进行原子更新: 如果真正需要对于超级块进行改写, 则使用日志系统保证原子性修改。

最后，深度理解inode结构。磁盘中inode的结构如下：

```
struct dinode {
    short type;           // 文件类型
    short major;          // 主设备号 (设备文件)
    short minor;          // 次设备号 (设备文件)
    short nlink;          // 硬链接数
    uint32 size;           // 文件大小 (字节)
    uint32 addrs[NDIRECT + 1]; // 数据块地址数组
};
```

每个inode结构中包含一个地址数组，由12个直接指针和1个一级间接指针组成。直接指针直接指向存储文件内容的数据块地址；而一级间接指针则指向一个“间接块”，该间接块内部存储了更多的数据块地址。为了支持较大的文件，该系统实现了间接块机制，通过间接指针从而映射到更多的数据块地址，进而实现大文件。

除此以外，该系统还实现了硬链接机制。inode数据结构中的nlink表示了硬链接数，在其被文件名绑定时引用+1，其解除引用时引用-1，只有当最后一个文件名被删除时，文件才会被真正地磁盘中移除。

如下是深度思考问题及其答案：

问题一：为什么选择这种简单的布局？

xv6 的布局（引导块 -> 超级块 -> 日志 -> Inodes -> 位图 -> 数据块）是**为教学而生的刻意选择**。它的首要设计目标是**简洁明了**，而不是极致的性能或空间效率。

1. 易于理解和实现:

- 这种线性的、分区的布局非常直观。学生在阅读 `mkfs.c` (创建文件系统的工具) 的源码时，可以清晰地看到文件系统就是按顺序一块一块地被初始化和写入磁盘的。
- 内核代码在访问不同区域时，计算地址也相对简单：`inodestart + inode_num`, `bmapstart` 等，逻辑清晰。

2. 体现核心概念:

- 它虽然简单，但五脏俱全，完整地展示了一个经典 Unix 文件系统的所有核心组件：用于启动的**引导块**，描述全局的**超级块**，保证一致性的**日志**，文件元数据**Inode**，管理分配的**位图**，以及存储内容的数据块。这使得学生可以专注于理解这些基本概念，而不会被复杂的优化技巧分散注意力。

3. 性能上的合理性 (在小尺度上):

- 将所有元数据 (Inode, Bitmap) 物理上聚集在磁盘的开头部分，利用了**局部性原理**。当进行创建文件、列出目录等涉及大量元数据读写的操作时，磁盘磁头可以在一个相对较小的区域内移动，减少了昂贵的寻道时间。对于 xv6 设计时所面向的小规模磁盘来说，这是一种简单而有效的性能优化。

总而言之，这种布局是**为了教学清晰度而对复杂性做出的权衡**。它不是最优的，但对于学习者来说是最合适的。

问题二：如何提高空间利用率？

xv6 的简单设计导致了一些空间浪费，主要体现在以下几个方面，对应的改进方法如下：

1. 问题：内部碎片 (Internal Fragmentation)

- **现象:** xv6 中，一个文件无论多小（哪怕只有1个字节），都必须占用至少一个完整的数据块（例如 4KB）。一个10KB的文件需要3个数据块，但第三个块中大部分空间都被浪费了。
- **改进方法:**
 - **块内打包/尾部合并 (Tail Packing/Merging):** 对于多个小文件，或者大文件的最后一个“尾巴”块，可以将它们打包存储在同一个物理数据块中。这需要更复杂的块管理和寻址机制。
 - **块次级分配 (Block Suballocation):** 将一个大块（如4KB）在逻辑上划分为多个更小的单元（如512B），小文件可以按需分配这些小单元，从而减少浪费。

2. 问题：固定的 Inode 数量

- **现象:** 在格式化时，Inode 的总数就被固定下来。如果用户存储大量小文件，可能会出现磁盘空间还很充足，但因为 Inode 耗尽而无法创建新文件的窘境。反之，如果存储少量大文件，大量的预分配 Inode 空间就被浪费了。
- **改进方法:**
 - **动态 Inode 分配 (Dynamic Inode Allocation):** 不再使用一个固定的、连续的 Inode 表。而是将 Inode 本身也当作一种数据，存储在普通的数据块中。当需要新 Inode 时，从数据区中分配一个块来存储它。这样 Inode 的数量就可以根据实际需求动态增长。

3. 问题：间接块的开销

- **现象:** 对于一个刚好比直接指针能索引的容量大一点点的文件（例如，在 xv6 中第13个块），也必须分配一整个间接块来存储这一个额外的地址。这个间接块本身占用了宝贵的空间。
- **改进方法:**
 - **Extents (区段):** 这是针对间接块最有效的改进。见下一个问题的详细解答。

问题三：现代文件系统有什么改进？

现代文件系统（如 Linux 的 ext4, Btrfs, ZFS, macOS 的 APFS）在 xv6 的基础上进行了大量演进，核心改进方向是**规模化、性能和数据安全**。

1. 从“单一布局”到“块组化” (From Monolithic to Block Groups)

- **xv6:** 整个文件系统是一个大的整体，所有 Inode 在一起，所有数据块在另一边。
- **现代改进:** 将整个磁盘划分为多个**块组 (Block Groups)** 或**分配组 (Allocation Groups)**。每个组可以看作一个“微型xv6”，包含自己的超级块副本、Inode位图、数据块位图、一部分Inode表和数据块。
- **优势:**
 - **局部性:** 文件系统会尝试将一个文件的 Inode 和它的数据块分配在同一个块组内，极大地减少了磁盘寻道，提升了性能。
 - **可靠性:** 超级块等关键元数据有多个副本分布在不同块组，一个损坏了可以从其他副本恢复。
 - **并行化:** 可以在多个块组上并行执行分配操作。

2. 从“间接指针”到“Extents (区段)”

- **xv6:** 使用直接/间接指针，一个指针对应一个块，对于大文件，地址列表会变得非常庞大和低效。
- **现代改进:** 使用 **Extents**。一个 Extent 是一个指针，但它描述的是一段**连续的数据块**，形式为（**起始块号，块的数量**）。

- **优势:** 对于一个 100MB 的大文件，如果它是连续存放的，在 xv6 中可能需要数万个地址指针（通过多级间接块）；而在使用 Extent 的系统中，**只需要一个 Extent 条目**即可描述。这极大地减少了元数据的大小，并加快了大文件的访问速度。

3. 从“简单日志”到“日志/写时复制” (Journaling / Copy-on-Write)

- **xv6:** 使用简单的写前日志来保证崩溃一致性。
- **现代改进:**
 - **更精细的日志 (Journaling):** ext4 等系统提供了更灵活的日志模式，可以选择只记录元数据（性能高），或同时记录元数据和文件数据（安全性最高）。
 - **写时复制 (Copy-on-Write, CoW):** ZFS, Btrfs, APFS 的核心机制。当修改数据时，从不覆盖旧数据。而是将新数据写入一个全新的位置，然后以原子方式更新指向它的上层指针。
- **CoW 的优势:**
 - **永不崩溃不一致:** 因为指针的切换是原子的，所以文件系统状态永远是一致的，不再需要传统 **fsck** 这样的检查工具。
 - **快照 (Snapshots):** 创建文件系统某个时间点的只读副本几乎是零成本的，因为只需保留旧数据的指针不被回收即可。
 - **数据校验 (Checksums):** CoW 机制天然适合加入数据校验。在写入新数据时计算其校验和，并将校验和与指向它的指针存储在一起。读取时重新计算并比对，可以检测并修复静默的数据损坏（位衰减）。

总结来说，现代文件系统在 xv6 所展示的基本原理之上，通过块组化、Extents、写时复制等技术，在性能、可扩展性和数据安全性上实现了质的飞跃，以应对TB甚至PB级别的存储需求和对数据完整性的更高要求。

2.1.2 分析xv6的inode管理机制

首先，研读inode缓存管理。inode的数据结构如下所示：

```
struct inode {
    uint64 dev;           // 设备号
    uint64 inum;          // inode 号
    int ref;              // 引用计数
    int valid;            // inode 是否已从磁盘加载

    short type;           // 文件类型
    short major;          // 主设备号
    short minor;          // 次设备号
    short nlink;          // 硬链接数
    uint32 size;          // 文件大小（字节）
    uint32 addrs[NDIRECT + 1]; // 数据块地址数组
};
```

内存inode和磁盘inode的关系可以理解为“缓存副本与磁盘本体”的关系。磁盘上的 inode (**struct dinode**) 是持久化存储的最终数据，而内存中的 inode (**struct inode**) 是为了性能而创建的、在内存中的、带有额外管理信息的副本。磁盘上的inode主要是为了持久化存储文件的元数据，只关心数据本身的信息，结构紧凑；而内存中的Inode作为磁盘Inode在内存中的高速缓存，其附加了管理信息，从而方便内核进行操作

对于引用计数，**ref** 字段是 inode缓存管理的核心，它的作用是防止一个正在被内核使用的内存 inode 被从缓存中回收或重用：

- 当内核的任何部分（如一个系统调用的实现）需要开始使用一个 inode 时，它必须调用 `iget()`。
`iget()` 会找到或加载该 inode，并将其 `ref` 计数 **加 1**。
- 当该部分代码使用完这个 inode 后（例如，系统调用结束），它必须调用 `iput()`。`iput()` 会将 `ref` 计数 **减 1**。

在需要缓存的时候，其将会选择引用计数为0的缓存进行释放。从而实现inode空间的安全分配。

缓存一致性要解决两个问题：如何确保内存中的数据能反映磁盘的最新状态？如何确保内存中被修改的数据能安全地写回磁盘？xv6 通过**锁**、**valid 位**和**日志系统**协同工作来保证一致性。

1. 读一致性 (从磁盘到内存)

- **valid 标志**: 如前所述，`valid` 标志确保了 inode 的数据只在第一次真正需要时才从磁盘加载，避免了读取陈旧或未初始化的数据。
- **锁 (ilock)**: 在检查 `valid` 位和从磁盘读取数据期间，内核会锁住这个内存 inode。这可以防止另一个进程同时尝试访问这个正在“填充内容”的 inode，避免了竞争条件。

2. 写一致性 (从内存到磁盘)

- **修改只在内存中进行**: 当内核修改一个 inode 时（例如，`write()` 系统调用增加了文件大小 `size`），它首先只修改内存中的 `struct inode` 副本。这非常快。
- **通过日志原子写回**: 这种修改不会立即直接写回磁盘上的 `struct dinode`。直接写盘如果在中途断电，会导致磁盘数据损坏。取而代之的是，当需要将变更持久化时（通常在 `iput()` 中，或者文件关闭 `close()` 时），内核会调用 `iupdate()` 函数。
- `iupdate()` 的工作流程是：
 1. 开启一个日志事务 (`begin_op`)。
 2. 将整个被修改的内存 inode (`struct inode`) 的内容，作为一个数据块**写入到日志区域** (`log_write`)。
 3. 提交事务 (`end_op`)。
- 日志系统稍后会负责将日志区域中的这个新 inode 版本，原子地“安装”到它在磁盘上的最终位置。这个过程是崩溃安全的。如果在安装过程中崩溃，系统重启后会重做日志，保证写入的完整性。

其次，分析inode分配算法。其代码如下：

```
struct inode*
ialloc(uint dev, short type)
{
    int inum;
    struct buf *bp;
    struct dinode *dip;

    for(inum = 1; inum < sb.ninodes; inum++){
        bp = bread(dev, IBLOCK(inum, sb));
        dip = (struct dinode*)bp->data + inum%IPB;
        if(dip->type == 0){ // a free inode
            memset(dip, 0, sizeof(*dip));
            dip->type = type;
            log_write(bp); // mark it allocated on the disk
            brelse(bp);
            return iget(dev, inum);
        }
    }
}
```

```

    }
    brelse(bp);
}
printf("ialloc: no inodes\n");
return 0;
}

```

该结构中由于没有有关于inode的bitmap，所以不得不逐个在磁盘块中查找（每个block中有IPB个inode），一直寻找直到找到`type == 0`的数据块。

如果分配失败，将会给予警告信息`ialloc: no inodes\n`，之后返回0，表示没有找到块。

为了实现并发同步机制，其使用了`log_write`从而保证磁盘读写的原子性，而且在`iget`函数中，通过对inode数组上锁，从而对于数据进行保护。

最后，理解文件数据块管理。代码如下：

```

static uint
bmap(struct inode *ip, uint bn)
{
    uint addr, *a;
    struct buf *bp;

    if(bn < NDIRECT){
        if((addr = ip->addrs[bn]) == 0){
            addr = balloc(ip->dev);
            if(addr == 0)
                return 0;
            ip->addrs[bn] = addr;
        }
        return addr;
    }
    bn -= NDIRECT;

    if(bn < NINDIRECT){
        // Load indirect block, allocating if necessary.
        if((addr = ip->addrs[NDIRECT]) == 0){
            addr = balloc(ip->dev);
            if(addr == 0)
                return 0;
            ip->addrs[NDIRECT] = addr;
        }
        bp = bread(ip->dev, addr);
        a = (uint*)bp->data;
        if((addr = a[bn]) == 0){
            addr = balloc(ip->dev);
            if(addr){
                a[bn] = addr;
                log_write(bp);
            }
        }
    }
    brelse(bp);
}

```



```

    return addr;
}

panic("bmap: out of range");
}

```

该函数实现了逻辑块号到物理块号的转换，其规则为：

- 如果逻辑块号小于NDIRECT，则直接在inode的addrs数组中寻找对应物理块号
- 如果逻辑块号大于等于NDIRECT，则需要加载间接块（块号NDIRECT），之后从NDIRECT中获取所需的物理块号。

间接块的实现机制即为，在地址数据中添加一个间接指针，指向存储若干个物理块地址的数据块，需要查找间接块的时候，先访问间接数据块中的内容，然后再由间接数据块寻找对应的数据块地址。

间接块的实现，增加了inode能够表示的数据块数，同时扩展了文件的大小。

如下是思考题的答案：

inode 缓存的替换策略是什么？

- xv6 的 inode 缓存替换策略可以被称为“扫描首个未被引用的插槽”，`iget()`会从头到尾线性扫描整个 inode 缓存数组，直至找到第一个 `ref == 0` 的缓存槽。一旦找到这个槽，`iget()`就会占用他，并从磁盘加载，搜索即停止
- 这种策略实现简单，开销低，但是其效率低下，容易导致缓存抖动。现代操作系统普遍采用经典的LRU算法，从而保留热数据，提高缓存命中率。

如何防止inode泄漏？

- 对于内存泄漏，xv6采用严格的iget/iput协议，即`iget()`和`iput()`必须成对出现，在内核函数使用时引用数+1，结束时-1，这样即使在函数出错需要提前返回的情况下，错误处理路径也必须确保调用 `iput()` 来释放已经获取的 inode。从而保证内存不泄露

大文件的性能问题如何解决？

- xv6对于大文件采用单级间接寻址，通过直接块和间接块对于inode的数据块进行管理。
- 这种设计方式对于很大的文件并不友好，现代文件系统通常采用多级间接指针以及区段的方式从而进行大文件管理

2.1.3 设计你的文件系统布局

在此之后，实现自己的文件系统布局。参考xv6的文件布局，基本宏定义如下：

```

// 文件系统基本参数
#define BSIZE 1024           // 块大小
#define FSSIZE 1000         // 文件系统块数
#define LOGSIZE 30          // 日志块数
#define ROOTDEV 1           // 根设备号
#define ROOTINO 1           // 根目录 inode 号
#define FSMAGIC 0x10203040  // 文件系统魔数

```

对于inode结构的设计，与xv6相同，详情请看源代码仓库。

2.1.4 实现块缓存系统

参考xv6的bio.c，实现对于块缓存的管理。在这里使用和xv6相同的buf结构。在1.2节已经阐述，这里不再赘述。

对于缓冲区的读写，实现如下函数：

```
// 读取块
struct buf* bread(uint64 dev, uint64 blockno) {
    struct buf *b;

    b = bget(dev, blockno);

    if(!b->valid) {
        virtio_disk_rw(b, 0);
        b->valid = 1;
    }

    return b;
}

// 写入块到磁盘
void bwrite(struct buf *b) {
    if(b->refcnt < 1)
        panic("bwrite");

    virtio_disk_rw(b, 1);
}

// 释放块
void brelse(struct buf *b) {
    if(b->refcnt < 1)
        panic("brelse");

    b->refcnt--;

    if(b->refcnt == 0) {
        // 移动到 LRU 链表头部
        b->next->prev = b->prev;
        b->prev->next = b->next;
        b->next = bcache.head.next;
        b->prev = &bcache.head;
        bcache.head.next->prev = b;
        bcache.head.next = b;
    }
}
```

这些函数实现了块的读写以及释放。

2.1.5 实现日志系统

xv6 的日志系统是其最精妙、也最重要的部分之一，是保证文件系统在意外崩溃后不会损坏的关键。其唯一的目的就是**让这一组相关的更新操作变成一个原子操作**：要么全部成功，要么就像从未发生过一样。

日志包含如下部分：

1. 写前日志 (Write-Ahead Logging, WAL)

这是整个机制的核心思想，可以用一句话概括：**在修改任何实际的文件系统数据（Inode、位图、数据块）之前，先把“打算怎么改”的计划写到一个专门的日志区域里。**

这个“计划”就是日志。

xv6 的实现：

- **日志区**: 磁盘上有一块连续的区域被专门划出来用作日志。
- **日志头 (Log Header)**: 日志区的第一个块是日志头。它非常关键，包含一个整数 `n`，表示这个日志里记录了多少个块的修改；以及一个数组 `block[]`，记录了这 `n` 个块在磁盘上的**最终地址**。
- **日志数据**: 跟在日志头后面的是日志数据块。如果 `n=5`，那么这里就会有5个块，分别对应 `block[0]` 到 `block[4]` 的新内容。

工作流程:

1. **不直接写盘**: 当文件系统需要修改磁盘块 `B` 时，它不会直接去写 `B`。
2. **写入日志**: 它会把 `B` 的新内容写入到日志区的一个空闲槽位里，并在**内存中的**日志头里记下 `B` 的地址。
3. **重复**: 对一个事务中所有需要修改的块都执行步骤2。
4. **提交**: 当所有修改都准备好后，才把**内存中的**日志头（包含 `n` 和所有块地址）**一次性写入**到磁盘上的日志头块。
5. **安装**: 最后，再根据日志头中的地址，把日志数据区里的新内容逐一复制到它们在磁盘上的最终位置（例如，把新的位图内容写到真正的位图块）。

这个“先写日志，再写数据”的原则，就是 WAL 的精髓。

2. 事务的原子性保证

原子性是通过一个巧妙的“**提交点 (Commit Point)**”来实现的。在 xv6 中，这个提交点就是**将日志头写入磁盘的那一刻**。

让我们把一个事务（比如前面创建文件的例子）的生命周期分解开：

1. **begin_op()**: 事务开始。系统会获取日志系统的锁，确保同一时间只有一个事务在进行。
2. **log_write()**: 在事务期间，每次对磁盘块的修改都会被 `log_write()` 截获。`log_write()` 只是把新数据写入日志区，并在内存中记录下块号。**此时，磁盘上的日志头 `n` 仍然是 0**。从磁盘的角度看，什么都没发生。
3. **end_op()**: 事务结束。这个函数会调用 `commit()`，这是最关键的环节。
 - **commit() 第一步：写入日志头**。它将内存中记录的 `n` 和块地址列表写入磁盘上的日志头块。**这就是原子提交点**。一旦这个写操作完成，这个事务就被认为是“已提交”的。
 - **commit() 第二步：安装日志**。它循环 `n` 次，将日志数据区的内容复制到其最终的磁盘位置。
 - **commit() 第三步：清除日志**。所有数据都安全落地后，它再次向磁盘上的日志头块写入 `n=0`。这表示事务已彻底完成，日志区可以被下一个事务使用了。

原子性如何体现？

- **崩溃在提交点之前**: 如果在 `commit()` 写入日志头之前崩溃, 那么磁盘上的日志头 `n` 仍然是 0。系统重启后, 看到 `n=0`, 就认为上次没有未完成的事务, 直接忽略日志区的内容。文件系统状态保持在事务开始前的样子, 就像什么都没发生过。
- **崩溃在提交点之后**: 如果在 `commit()` 写入日志头之后、但在清除日志之前任何时刻崩溃 (比如正在“安装”数据时), 磁盘上的日志头 `n` 是一个大于 0 的值。系统重启后, 看到 `n > 0`, 就明白有一个已提交但未完成的事务。

3. 崩溃恢复机制

xv6 的崩溃恢复机制非常简单, 可以称为“**前滚 (Roll Forward)**”。

当文件系统启动时, 它会执行以下恢复流程:

1. **读取日志头**: 系统首先读取磁盘上日志头块的内容。
2. **检查 `n` 的值**:
 - 如果 `n == 0`: 说明上次关机前所有事务都已完美结束。一切正常, 无需恢复。
 - 如果 `n > 0`: 说明系统在一个已提交的事务执行期间崩溃了。此时, 磁盘上的文件系统可能处于不一致状态 (有些块可能已经“安装”, 有些则没有)。
3. **重做日志 (Redo Logging)**:
 - 恢复代码会完全相信日志头里的信息。它会像正常的 `commit()` 第二步一样, **重新执行一遍“安装”过程**。
 - 它根据日志头里记录的块地址, 将日志数据区里的内容再次逐一复制到它们的最终位置。
 - 这个操作是**幂等 (idempotent)** 的, 意思是重复执行多次和只执行一次的效果是一样的。即使崩溃前已经有部分块被安装了, 用同样的内容再覆盖写一遍也无妨。
4. **清除日志**:
 - 在完成所有块的“安装”后, 恢复代码会向日志头写入 `n=0`, 将文件系统置于一个干净、一致的状态, 然后才开始正常的运行。

xv6 的日志系统就像一个严谨的会计。它不直接在账本 (文件系统主体) 上涂改, 而是先在草稿纸 (日志区) 上写下所有要做的账目变更。只有当所有变更都在草稿纸上写完后, 它才在草稿纸的顶端签上字 (提交, 即写日志头)。然后, 它才开始一项一项地把草稿纸上的内容誊抄到主账本上。

如果中途被打断 (崩溃), 任何人回来后, 只要看到签过字的草稿纸, 就可以继续完成誊抄工作, 保证账本的最终正确性。如果草稿纸还没签字就被打断, 那直接把草稿纸扔掉就行, 主账本分毫未动。

参考xv6的log.c, 实现日志的关键事务处理函数如下:

初始化日志:

```
void log_init(int dev, struct superblock *sb) {
    if(sizeof(struct logheader) >= BSIZE)
        panic("log_init: too big logheader");

    spinlock_init(&log_ctx.lock, "log"); // 确保锁也被初始化了
    log_ctx.dev = dev;
    log_ctx.start = sb->logstart;
    log_ctx.size = sb->nlog;
```

```
// printf("log_init: zeroing log\n");
for (int i = 0; i < log_ctx.size; i++) {
    struct buf *bp = bread(log_ctx.dev, log_ctx.start + i);
    memset(bp->data, 0, BSIZE);
    bwrite(bp); // <-- 使用 bwrite 进行直接、非日志的写操作
    brelse(bp);
}
// printf("log_init: zeroing log done\n");

recover_from_log();
}
```

开始文件系统操作：

```
void begin_op(void) {
    while(1) {
        if(log_ctx.committing) {
            // 等待提交完成
            continue;
        } else if(log_ctx.lh.n + (log_ctx.outstanding + 1) * LOGSIZE > LOGSIZE) {
            // 日志空间不足
            continue;
        } else {
            log_ctx.outstanding++;
            break;
        }
    }
}
```

结束文件系统操作：

```
void end_op(void) {
    int do_commit = 0;

    log_ctx.outstanding--;

    if(log_ctx.committing)
        panic("log_ctx.committing");

    if(log_ctx.outstanding == 0) {
        do_commit = 1;
        log_ctx.committing = 1;
    }

    if(do_commit) {
        commit();
        log_ctx.committing = 0;
    }
}
```

```

    }
}

```

将修改后的块写入日志：

```

void log_write(struct buf *b) {
    int i;

    if(log_ctx.lh.n >= LOGSIZE || log_ctx.lh.n >= log_ctx.size - 1)
        panic("too big a transaction");
    if(log_ctx.outstanding < 1)
        panic("log_write outside of trans");

    // 查找块是否已在日志中
    for(i = 0; i < log_ctx.lh.n; i++) {
        if(log_ctx.lh.block[i] == b->blockno)
            break;
    }

    log_ctx.lh.block[i] = b->blockno;
    if(i == log_ctx.lh.n) {
        bpin(b);
        log_ctx.lh.n++;
    }
}

```

从日志中恢复所需的读写：

```

static void recover_from_log(void) {
    read_head();
    install_trans(1); // 如果已提交,则安装
    log_ctx.lh.n = 0;
    write_head(); // 清除日志
}

```

2.1.6 实现目录和路径更新

参考xv6的目录实现，实现目录机制。目录的数据结构在上文已经讲述过，这里不再赘述。

接下来，实现目录解析算法如下：

```

static struct inode* namex(char *path, int nameiparent, char *name) {
    struct inode *ip, *next;

    if(*path == '/')
        ip = iget(ROOTDEV, ROOTINO);
    else

```

```

        ip = idup(iget(ROOTDEV, ROOTINO)); // 简化: 当前目录就是根目录

        while((path = skipelem(path, name)) != 0) {
            ilock(ip);
            if(ip->type != T_DIR) {
                iunlockput(ip);
                return 0;
            }
            if(nameiparent && *path == '\\0') {
                iunlock(ip);
                return ip;
            }
            if((next = dirlookup(ip, name, 0)) == 0) {
                iunlockput(ip);
                return 0;
            }
            iunlockput(ip);
            ip = next;
        }
        if(nameiparent) {
            iput(ip);
            return 0;
        }
        return ip;
    }

```

实现目录操作接口如下，这两个函数分别对本目录和父目录进行查找：

```

// 查找路径对应的 inode
struct inode* namei(char *path) {
    char name[DIRSIZ];
    return namex(path, 0, name);
}

// 查找路径的父目录 inode
struct inode* nameiparent(char *path, char *name) {
    return namex(path, 1, name);
}

```

2.2 问题与解决方案

问题：实现inode时，inode写入与读出的结果不一致

解决：检查发现是没有调用对应写入磁盘的操作，强制调用磁盘写操作或实现日志系统后解决。

问题：初始编译时出现struct dinode未定义的错误。

解决：检查发现是头文件包含顺序有误，调整#include顺序后解决。

问题：在实现日志提交时，系统在崩溃后无法正确恢复。

解决：经过调试发现日志头的提交状态标记没有在事务完成后立即写入磁盘。修改了log_write和log_commit的逻辑，确保状态标记的写入顺序符合协议。

问题：对LRU缓存置换算法的理解不清，导致实现错误。

解决：重新阅读了教材相关章节，并用简单的伪代码梳理了流程，最终正确实现了brelse时更新块最近访问时间的逻辑。

2.3 源码理解总结

本次实验的核心源码集中在以下几个数据结构：

- **buf**：实现了块缓存，所有磁盘读写都通过该数据结构进行，其防止了由于频繁进行磁盘读写而导致的性能问题。
- **bitmap**：可以通过查找位图从而快速查找到所需的块的分配情况。
- **inode**：文件管理的主要操作模块，基本文件的读写即对应inode的读写而进行
- **dirent**：目录操作项，在本操作系统中实现了目录操作以及路径解析。
- **log**：实现了完整的日志系统，是保证崩溃一致性的关键。
- **file**：提供了文件描述符层的操作接口。可以通过系统调用从而实现文件的读和写

由于本系统采用和xv6相同的系统，所以此处采用4个文件**fs.c**、**log.c**、**bio.c**、**file.c**从而存放所有所需的数据结构和功能代码，这些功能，构成了一个虽然小但是功能齐全的文件系统。

3. 测试验证部分

3.1 文件系统基本功能测试

本测试主要测试文件系统的基本功能，包括文件写入，读取以及目录创建功能测试，测试代码如下所示：

```
print( "=== 测试文件系统 ===\n\n" );

cleanup_test_artifacts();

// 测试 1: 创建文件并写入
print( "测试 1: 创建文件并写入\n" );
fd = open( "/test", O_CREATE | O_RDWR );
print( "打开文件, fd=" );
print_int(fd);
print( "\n" );

if(fd >= 0) {
    const char* msg = "Hello FS";
    int len = 8;
    print( "写入内容: " );
    print( msg );
    print( "\n" );
    int n = write(fd, msg, len);
    print( "写入 " );
    print_int(n);
    print( " 字节\n" );
```



```
        close(fd);
        print( "文件已关闭\n" );
    }
    print("\n");

// 测试 2: 重新打开并读取
print( "测试 2: 读取文件\n" );
fd = open( "/test", O_RDONLY );
if(fd >= 0) {
    char buf[32];
    for(int i=0; i<32; i++) buf[i]=0;
    int n = read(fd, buf, 20);
    print( "读取 " );
    print_int(n);
    print( " 字节: " );
    print(buf);
    print( "\n" );
    close(fd);
}
print("\n");

// 测试 3: 创建目录
print( "测试 3: 创建目录\n" );
int r = mkdir("/dir");
print( "mkdir 返回: " );
print_int(r);
print( "\n\n" );

print( "所有文件系统测试完成! \n\n" );
```

测试结果如下图所示：

=== 测试文件系统 ===

清理测试生成的文件和目录...

测试 1: 创建文件并写入

打开文件, fd=0

写入内容: Hello FS

写入 8 字节

文件已关闭

测试 2: 读取文件

读取 8 字节: Hello FS

测试 3: 创建目录

mkdir 返回: 0

所有文件系统测试完成!

说明文件系统的基本功能正常

3.2 文件系统完整性测试

本测试主要在上一个测试的基础上, 测试文件系统的完整性, 测试代码如下所示:

```
print( "=== 文件系统完整性测试 ===\n\n" );

// 创建测试文件
print( "创建测试文件...\n" );
fd = open( "/testfile", O_CREATE | O_RDWR );
if ( fd < 0 ) {
    print( "错误: 无法创建文件\n" );
    while(1){}
    return -1;
}
print( "文件创建成功\n" );

// 写入数据
const char* test_buffer = "Hello, filesystem!";
int buffer_len = 0;
while ( test_buffer[buffer_len] ) buffer_len++;

print( "写入内容: " );
```

```
print( test_buffer );
print( "\n" );
int bytes_written = write( fd, test_buffer, buffer_len );
print( "写入了 " );
print_int( bytes_written );
print( " 字节\n" );
close( fd );

// 重新打开并验证
print( "重新打开文件并验证...\n" );
fd = open( "/testfile", O_RDONLY );
if ( fd < 0 ) {
    print( "错误: 无法打开文件\n" );
    while(1){}
    return -1;
}

char read_buffer[64];
for ( int i = 0; i < 64; i++ ) read_buffer[i] = 0;
int bytes_read = read( fd, read_buffer, sizeof(read_buffer) - 1 );
read_buffer[bytes_read] = '\0';

print( "读取了 " );
print_int( bytes_read );
print( " 字节: " );
print( read_buffer );
print( "\n" );

// 验证数据一致性
int data_match = 1;
for ( int i = 0; i < buffer_len; i++ ) {
    if ( test_buffer[i] != read_buffer[i] ) {
        data_match = 0;
        break;
    }
}

if ( data_match ) {
    print( "数据验证通过! \n" );
} else {
    print( "错误: 数据验证失败\n" );
    while(1){}
    return -1;
}

close( fd );

// 删除文件
print( "删除文件...\n" );
int unlink_result = unlink( "/testfile" );
if ( unlink_result == 0 ) {
    print( "文件删除成功\n" );
} else {
    print( "错误: 文件删除失败\n" );
}
```

```
while(1){}  
return -1;  
}  
  
print( "文件系统完整性测试通过! \n\n" );
```

测试结果如下图所示：

```
=== 文件系统完整性测试 ===  
  
创建测试文件...  
文件创建成功  
写入内容: Hello, filesystem!  
写入了 18 字节  
重新打开文件并验证...  
读取了 18 字节: Hello, filesystem!  
数据验证通过!  
删除文件...  
文件删除成功  
文件系统完整性测试通过!
```

说明该文件系统功能完整，运行正常

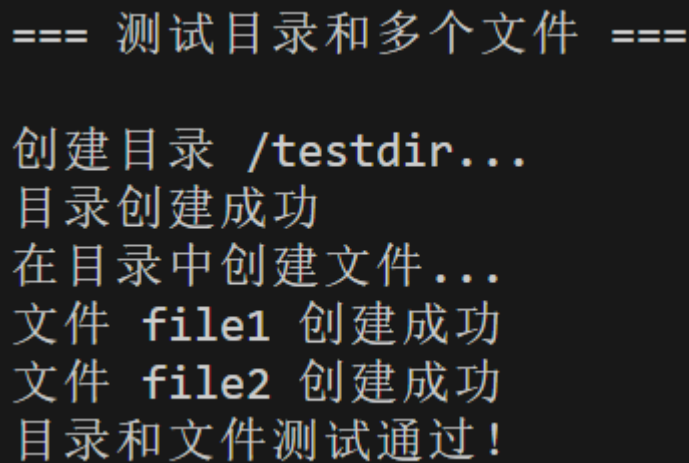
3.3 测试目录和多个文件

本测试主要测试系统对于目录和多个文件的处理情况，测试代码如下所示：

```
print( "=== 测试目录和多个文件 ===\n\n" );  
  
print( "创建目录 /testdir...\n" );  
int mkdir_result = mkdir( "/testdir" );  
if ( mkdir_result < 0 ) {  
    print( "错误: 无法创建目录\n" );  
    while(1){}  
    return -1;  
}  
print( "目录创建成功\n" );  
  
// 在目录中创建多个文件  
print( "在目录中创建文件...\n" );  
fd = open( "/testdir/file1", O_CREATE | O_RDWR );  
if ( fd < 0 ) {  
    print( "错误: 无法在目录中创建文件\n" );  
    while(1){}  
    return -1;  
}
```

```
}  
write( fd, "File 1 content", 14 );  
close( fd );  
print( "文件 file1 创建成功\n" );  
  
fd = open( "/testdir/file2", O_CREATE | O_RDWR );  
if ( fd >= 0 ) {  
    write( fd, "File 2 content", 14 );  
    close( fd );  
    print( "文件 file2 创建成功\n" );  
}  
  
print( "目录和文件测试通过! \n\n" );
```

测试结果如下图所示：



```
=== 测试目录和多个文件 ===  
  
创建目录 /testdir...  
目录创建成功  
在目录中创建文件...  
文件 file1 创建成功  
文件 file2 创建成功  
目录和文件测试通过!
```

说明本系统可以进行多个目录和文件的创建和读写，功能正常

3.4 并发访问测试

本测试主要测试在并发访问的情况下功能是否正常，测试代码如下所示：

```
print( "=== 并发访问测试 ===\n\n" );  
print( "创建共享文件...\n" );  
fd = open( "/shared_file", O_CREATE | O_RDWR );  
if ( fd >= 0 ) {  
    const char* initial_msg = "Initial";  
    print( "写入初始内容: " );  
    print( initial_msg );  
    print( "\n" );  
    write( fd, initial_msg, 7 );  
    close( fd );  
    print( "共享文件创建成功\n" );  
}  
  
print( "创建两个子进程同时写入文件...\n" );  
for ( int i = 0; i < 2; i++ ) {
```

```
int fork_result = fork();
if ( fork_result == 0 ) {
    // 子进程
    print( "子进程 " );
    print_int( i + 1 );
    print( " 开始写入\n" );

    // 使用 O_APPEND 模式打开文件，确保追加写入
    fd = open( "/shared_file", O_RDWR | O_APPEND );
    if ( fd >= 0 ) {
        char write_data[20];
        for ( int j = 0; j < 20; j++ ) {
            write_data[j] = 'A' + i;
        }
        write_data[10] = '\0';

        print( "子进程 " );
        print_int( i + 1 );
        print( " 写入内容: " );
        print( write_data );
        print( " (追加5次)\n" );

        // 多次写入以测试并发追加
        for ( int j = 0; j < 5; j++ ) {
            write( fd, write_data, 10 );
        }

        close( fd );
        print( "子进程 " );
        print_int( i + 1 );
        print( " 写入完成\n" );
    }

    exit( 0 );
}

// 父进程等待子进程完成
print( "等待子进程完成...\n" );
for ( int i = 0; i < 2; i++ ) {
    int status;
    wait( &status );
}

// 验证文件内容
print( "验证共享文件内容...\n" );
fd = open( "/shared_file", O_RDONLY );
if ( fd >= 0 ) {
    char verify_buf[128];
    for ( int i = 0; i < 128; i++ ) verify_buf[i] = 0;
    int n = read( fd, verify_buf, 127 );
    verify_buf[n] = '\0';
    print( "读取到 " );
}
```

```
    print_int( n );
    print( " 字节: " );
    print( verify_buf );
    print( "\n" );
    close( fd );
}
print( "并发访问测试完成\n\n" );
```

测试结果如下图所示：

```
=== 并发访问测试 ===
创建共享文件...
写入初始内容: Initial
共享文件创建成功
创建两个子进程同时写入文件...
等待子进程完成...
子进程 1 开始写入
子进程 1 写入内容: AAAAAAAAAA (追加5次)
子进程 1 写入完成
子进程 2 开始写入
子进程 2 写入内容: BBBBBBBBBB (追加5次)
子进程 2 写入完成
验证共享文件内容...
读取到 107 字节: InitialAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAABBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
BBBBBB
并发访问测试完成
```

说明系统能够在并发环境下正常处理文件的读写操作。

3.5 崩溃恢复测试

本测试主要测试日志模块的恢复能力。通过对于崩溃恢复的读写进行测试。测试代码如下所示：

```
print( "=== 崩溃恢复测试 ===\n\n" );
print( "测试场景：文件写入过程中的数据一致性\n" );

// 创建测试文件并写入多次
print( "创建测试文件并进行多次写入...\n" );
fd = open( "/crash_test", O_CREATE | O_RDWR );
if ( fd >= 0 ) {
    for ( int i = 0; i < 10; i++ ) {
        char crash_data[32];
        for ( int j = 0; j < 31; j++ ) {
            crash_data[j] = '0' + i;
        }
        crash_data[31] = '\n';

        int written = write( fd, crash_data, 32 );
        print( "第 " );
        print_int( i + 1 );
        print( " 次写入: " );
        print_int( written );
        print( " 字节\n" );

        // 在这里如果系统崩溃，日志系统应该能保证数据一致性
    }
}
```

```
        close( fd );
    }

    // 重新打开并验证
    print( "重新打开文件并验证数据一致性...\n" );
    fd = open( "/crash_test", O_RDONLY );
    if ( fd >= 0 ) {
        char crash_verify[512];
        for ( int i = 0; i < 512; i++ ) crash_verify[i] = 0;
        int n = read( fd, crash_verify, 511 );
        print( "读取到 " );
        print_int( n );
        print( " 字节 (预期 320 字节)\n" );

        if ( n == 320 ) {
            print( "数据大小验证通过\n" );
        } else {
            print( "警告: 数据大小不符\n" );
        }

        close( fd );
    }
    print( "崩溃恢复测试完成\n\n" );
```

测试结果如下图所示：

=== 崩溃恢复测试 ===

测试场景：文件写入过程中的数据一致性
创建测试文件并进行多次写入...

第 1 次写入：32 字节

第 2 次写入：32 字节

第 3 次写入：32 字节

第 4 次写入：32 字节

第 5 次写入：32 字节

第 6 次写入：32 字节

第 7 次写入：32 字节

第 8 次写入：32 字节

第 9 次写入：32 字节

第 10 次写入：32 字节

重新打开文件并验证数据一致性...

读取到 320 字节 (预期 320 字节)

数据大小验证通过

崩溃恢复测试完成

说明崩溃恢复功能正常

3.6 性能测试

本测试主要测试当前文件系统的性能，通过小文件批量创建，大文件书写和读取机型测试。测试代码如下所示：

```
print( "=== 性能测试 ===\n\n" );

// 测试 1: 大量小文件创建 (减少到30个以避免资源耗尽)
print( "测试 1: 创建 30 个小文件\n" );
print( "开始创建...\n" );

int small_file_count = 0;
for ( int i = 0; i < 30; i++ ) {
    // 手动构造文件名 /small_XX
    char filename[32];
    int pos = 0;
    filename[pos++] = '/';
    filename[pos++] = 's';
    filename[pos++] = 'm';
    filename[pos++] = 'a';
    filename[pos++] = 'l';
```

```
filename[pos++] = 'l';
filename[pos++] = '_';

if ( i >= 10 ) {
    filename[pos++] = '0' + (i / 10);
}
filename[pos++] = '0' + (i % 10);
filename[pos] = '\\0';

fd = open( filename, O_CREATE | O_RDWR );
if ( fd >= 0 ) {
    write( fd, "test", 4 );
    close( fd );
    small_file_count++;
} else {
    print( "警告: 无法创建文件 " );
    print( filename );
    print( "\\n" );
    break;
}

    print( "已创建 " );
    print_int( i + 1 );
    print( " 个文件\\n" );
}

print( "成功创建 " );
print_int( small_file_count );
print( " 个小文件\\n\\n" );

// 测试 2: 大文件写入
print( "测试 2: 大文件写入 (写入 100 个块)\\n" );
fd = open( "/large_file", O_CREATE | O_RDWR );
if ( fd >= 0 ) {
    char large_buffer[512];
    for ( int i = 0; i < 512; i++ ) {
        large_buffer[i] = 'L';
    }

    int total_written = 0;
    for ( int i = 0; i < 100; i++ ) {
        int written = write( fd, large_buffer, 512 );
        total_written += written;

        // 每 20 次写入打印一次进度
        if ( (i + 1) % 20 == 0 ) {
            print( "已写入 " );
            print_int( i + 1 );
            print( " 个块\\n" );
        }
    }

    close( fd );
    print( "总共写入 " );
```

```
    print_int( total_written );
    print( " 字节\n\n" );
}

// 测试 3: 顺序读取性能
print( "测试 3: 顺序读取大文件\n" );
fd = open( "/large_file", O_RDONLY );
if ( fd >= 0 ) {
    char read_buffer[512];
    int total_read = 0;
    int read_count = 0;

    while ( 1 ) {
        int n = read( fd, read_buffer, 512 );
        if ( n <= 0 ) break;
        total_read += n;
        read_count++;

        if ( read_count % 20 == 0 ) {
            print( "已读取 " );
            print_int( read_count );
            print( " 个块\n" );
        }
    }

    close( fd );
    print( "总共读取 " );
    print_int( total_read );
    print( " 字节\n\n" );
}

print( "性能测试完成\n\n" );
```

测试结果如下图所示，程序很快就完成了测试，说明该文件系统性能较好：

=== 性能测试 ===

测试 1: 创建 30 个小文件

开始创建...

已创建 1 个文件

已创建 2 个文件

已创建 3 个文件

已创建 4 个文件

已创建 5 个文件

已创建 6 个文件

已创建 7 个文件

已创建 8 个文件

已创建 9 个文件

已创建 10 个文件

已创建 11 个文件

已创建 12 个文件

已创建 13 个文件

已创建 14 个文件

已创建 15 个文件

已创建 16 个文件

已创建 17 个文件

已创建 18 个文件

已创建 19 个文件

已创建 20 个文件

已创建 21 个文件

已创建 22 个文件

已创建 23 个文件

已创建 24 个文件

```
已创建 22 个文件  
已创建 23 个文件  
已创建 24 个文件  
已创建 25 个文件  
已创建 26 个文件  
已创建 27 个文件  
已创建 28 个文件  
已创建 29 个文件  
已创建 30 个文件  
成功创建 30 个小文件
```

```
测试 2：大文件写入（写入 100 个块）  
已写入 20 个块  
已写入 40 个块  
已写入 60 个块  
已写入 80 个块  
已写入 100 个块  
总共写入 51200 字节
```

```
测试 3：顺序读取大文件  
已读取 20 个块  
已读取 40 个块  
已读取 60 个块  
已读取 80 个块  
已读取 100 个块  
总共读取 51200 字节
```

```
性能测试完成
```

3.7 日志系统恢复测试

本测试主要测试日志系统的恢复，测试代码如下所示：

```
print("=== 日志系统恢复测试 ===\n");  
print("说明：此测试验证日志系统的崩溃恢复能力\n");  
print("步骤:\n");  
print("1. 创建文件并写入多次(模拟多个事务)\n");
```

```
print("2. 每次事务提交后,日志都会被正确写入\n");
print("3. 如果系统崩溃,下次启动时会从日志恢复\n\n");

int logs_exist = 1;
int check_fd = open("/log_00", O_RDONLY);
if(check_fd < 0) {
    logs_exist = 0;
} else {
    close(check_fd);
}

if(!logs_exist) {
    print("测试: 创建10个文件,每个文件写入32字节\n");
    for(int i = 0; i < 10; i++) {
        char filename[32];
        // 简单的整数转字符串
        filename[0] = '/';
        filename[1] = 'l';
        filename[2] = 'o';
        filename[3] = 'g';
        filename[4] = '_';
        filename[5] = '0' + (i / 10);
        filename[6] = '0' + (i % 10);
        filename[7] = '\0';

        int wfd = open(filename, O_CREATE | O_RDWR);
        if(wfd < 0) {
            print("创建文件失败: ");
            print(filename);
            print("\n");
            continue;
        }

        char content[32];
        for(int j = 0; j < 31; j++) {
            content[j] = 'A' + (i % 26);
        }
        content[31] = '\n';

        int n = write(wfd, content, 32);
        close(wfd);

        print("创建文件 ");
        print(filename);
        print(", 写入 ");
        print_int(n);
        print(" 字节\n");
    }
} else {
    print("检测到已有日志文件, 跳过创建阶段\n");
}

print("\n验证文件内容:\n");
for(int i = 0; i < 10; i++) {
```

```
char filename[32];
filename[0] = '/';
filename[1] = 'l';
filename[2] = 'o';
filename[3] = 'g';
filename[4] = '_';
filename[5] = '0' + (i / 10);
filename[6] = '0' + (i % 10);
filename[7] = '\\0';

int fd = open(filename, O_RDONLY);
if(fd < 0) {
    print("打开文件失败: ");
    print(filename);
    print("\\n");
    continue;
}

char content[33];
int n = read(fd, content, 32);
content[n] = '\\0';
close(fd);

print(filename);
print(": ");
print_int(n);
print(" 字节 - ");
print(content);
}

print("\\n日志系统测试完成!\\n");
print("说明: 如果系统重启,这些文件应该仍然存在且内容正确\\n");
print("这证明了日志系统正确实现了崩溃恢复\\n\\n");
```

测试结果如下图所示：

```
=== 日志系统恢复测试 ===
说明：此测试验证日志系统的崩溃恢复能力
步骤：
1. 创建文件并写入多次(模拟多个事务)
2. 每次事务提交后,日志都会被正确写入
3. 如果系统崩溃,下次启动时会从日志恢复

检测到已有日志文件，跳过创建阶段

验证文件内容：
/log_00: 32 字节 - AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA
/log_01: 32 字节 - BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB
/log_02: 32 字节 - CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC
/log_03: 32 字节 - DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD
/log_04: 32 字节 - EEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEEE
/log_05: 32 字节 - FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
/log_06: 32 字节 - GGGGGGGGGGGGGGGGGGGGGGGGGGGGGGGG
/log_07: 32 字节 - HHHHHHHHHHHHHHHHHHHHHHHHHHHHHHHH
/log_08: 32 字节 - IIIIIIIIIIIIIIIIIIIIIIIIIIIIIIIII
/log_09: 32 字节 - JJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJJ

日志系统测试完成！
说明：如果系统重启,这些文件应该仍然存在且内容正确
这证明了日志系统正确实现了崩溃恢复
```

说明该日志系统的恢复功能正常。

4. 结论与心得

通过本次实验，我理解了文件系统是如何将“文件”这一抽象概念，通过inode、目录项、数据块多层结构映射到物理磁盘扇区的，并深刻体会到日志和缓存这两个机制对提升文件系统可靠性和性能的关键作用。